

**NUTRIGUARDIAN : SMART HEALTH
MONITORING SYSTEM WITH IOT AND AI**

AHMAD NAQUIDDIN BIN MOHAMAD

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

BORANG PENGESAHAN STATUS LAPORAN

JUDUL: NutriGuardian: Smart Health Monitoring System With IoT and AI

SESI PENGAJIAN: 2024 / 2025

Saya: Ahmad Naquiuddin Bin Mohamad

mengaku membenarkan tesis Projek Sarjana Muda ini disimpan di Perpustakaan Universiti Teknikal Malaysia Melaka dengan syarat-syarat kegunaan seperti berikut:

1. Tesis dan projek adalah hakmilik Universiti Teknikal Malaysia Melaka.
2. Perpustakaan Fakulti Teknologi Maklumat dan Komunikasi dibenarkan membuat salinan untuk tujuan pengajian sahaja.
3. Perpustakaan Fakulti Teknologi Maklumat dan Komunikasi dibenarkan membuat salinan tesis ini sebagai bahan pertukaran antara institusi pengajian tinggi.
4. * Sila tandakan (✓)

_____ SULIT

(Mengandungi maklumat yang berdarjah keselamatan atau kepentingan Malaysia seperti yang termaktub di dalam AKTA RAHSIA RASMI 1972)

_____ TERHAD

(Mengandungi maklumat TERHAD yang telah ditentukan oleh organisasi / badan di mana penyelidikan dijalankan)

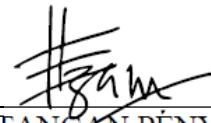
✓
_____ TIDAK TERHAD

AHMAD NAQUIUDDIN BIN
MOHAMAD

(TANDATANGAN PELAJAR)

Alamat tetap: No 22 Rumah Awam
Kos Rendah Kuala Telemong, 21210
Kuala Terengganu,
Terengganu

Tarikh: 30/08/2025



(TANDATANGAN PENYELIA)

[NAMA PENYELIA]

NOORREZAM BIN YUSOP

Nama Penyelia

Tarikh: 30/08/2025

NUTRIGUARDIAN: SMART HEALTH MONITORING
SYSTEM WITH IOT AND AI

AHMAD NAQUIDDIN BIN MOHAMAD

This report is submitted in partial fulfillment of the requirements for the
Bachelor of Computer Science (Software Development) with Honours.

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

2025

DECLARATION

I hereby declare that this project report entitled
NutriGuardian: Smart Health Monitoring System With IoT and Ai
is written by me and is my own effort and that no part has been plagiarized
without citations.

STUDENT : AHMAD NAQUIUDDIN BIN MOHAMAD Date : 30/08/2025
(STUDENT FULL NAME)

I hereby declare that I have read this project report and found
this project report is sufficient in term of the scope and quality for the award of
Bachelor of Computer Science (Software Development) with Honours.

SUPERVISOR : TS. DR NOORREZAM BIN YUSOP Date : 30/08/2025
(SUPERVISOR FULLNAME)

DEDICATION

This final year project report on NutriGuardian: Smart Health Monitoring System is dedicated to my family and friends. Their unwavering support and encouragement have been invaluable throughout this journey. I deeply appreciate their belief in me and the strength it has given me to pursue and complete this project.

I would also like to extend my heartfelt gratitude to my supervisor and lecturers for their guidance, expertise, and consistent motivation. Their mentorship has been instrumental in shaping the direction of this project and enhancing my academic and practical knowledge.

Furthermore, this project is dedicated to parents and children, whose needs for a sustainable, efficient, and personalized health monitoring solution have inspired the creation of the NutriGuardian system. It is my sincere hope that the system will not only provide convenience in tracking health and nutrition but also contribute to promoting healthier lifestyles and early health awareness among families.

Thank you to everyone who has inspired and supported the development of the NutriGuardian project.

ACKNOWLEDGEMENTS

I am immensely grateful for the unwavering support I have received throughout the development of the **NutriGuardian** application. The invaluable assistance from numerous individuals has been crucial in making this enriching and challenging project a success.

First and foremost, I extend my heartfelt thanks to my supervisor, **Ts. Dr Noorezzam Bin Yusop**. His steadfast support, expertise, and guidance have been indispensable at every stage of this project. His insightful feedback and encouragement have not only helped me navigate technical challenges but have also driven me to achieve excellence in all aspects of the NutriGuardian application. I am deeply thankful for the continuous support and patience of my friends and family. Your belief in my vision for NutriGuardian has been a constant source of inspiration. Your encouragement and understanding during late nights and difficult moments have enabled me to stay focused on creating an application that improves health awareness and supports personalized nutrition for children.

I also appreciate the online communities and forums where I found valuable advice and solutions. The implementation of essential features such as BMI calculation, IoT-based weight and height monitoring, and AI-powered food recommendations was greatly influenced by the collective knowledge and insights of fellow developers and tech enthusiasts. Your contributions have significantly improved the functionality and usability of this system.

Special thanks go to the **test users and early reviewers** who participated during the development phase of the NutriGuardian application. Your feedback and suggestions have been vital in refining the system's features and ensuring it effectively meets the needs of its users.

ABSTRACT

This project presents the development of NutriGuardian, a smart health monitoring system designed to assist parents and children in managing and tracking key health indicators, particularly body weight, height, BMI, and daily nutritional intake. The system aims to address the limitations of existing manual and non-personalized methods by offering an integrated, real-time, and user-friendly solution tailored for young users aged 5 to 12.

NutriGuardian utilizes a combination of Internet of Things (IoT) technology and Artificial Intelligence (AI) to collect, compute, and interpret health data. IoT devices, including a load cell sensor and ultrasonic sensor connected to an ESP8266 microcontroller, are used to measure the child's weight and height accurately. These values are transmitted via Wi-Fi to a local server, where they are stored and processed. The system then calculates Body Mass Index (BMI) and evaluates the child's health status. Using the Google AI Studio API, the system generates personalized food and calorie intake suggestions based on age, BMI, and activity level.

The mobile application, developed using Android Studio, provides an interactive interface that allows users to view their health data, track progress over time, and receive real-time feedback. A web-based admin control panel developed using native PHP allows supervisors or healthcare professionals to monitor and manage user data. The system runs on a local XAMPP server and stores data using MySQL, ensuring efficient and structured information handling.

The NutriGuardian system significantly improves the accessibility, accuracy, and personalization of health monitoring for children. By combining mobile technology, IoT, and AI, the project promotes early health awareness, supports informed nutritional choices, and reduces the burden of manual tracking for parents. The outcome is a reliable and scalable platform that empowers families to take a proactive role in child health management.

ABSTRAK

Projek ini membangunkan NutriGuardian, sebuah sistem pemantauan kesihatan pintar yang direka untuk membantu ibu bapa dan anak-anak dalam mengurus dan menjejak indikator kesihatan utama seperti berat badan, ketinggian, Indeks Jisim Badan (BMI), serta pengambilan kalori dan pemakanan harian. Sistem ini bertujuan untuk menangani kelemahan kaedah manual yang tidak diperibadikan dengan menyediakan satu penyelesaian yang bersepadu, masa nyata, dan mesra pengguna—terutama bagi kanak-kanak berumur antara 5 hingga 12 tahun.

NutriGuardian menggunakan gabungan teknologi Internet Pelbagai Perkara (IoT) dan Kecerdasan Buatan (AI) untuk mengumpul, mengira, dan mentafsir data kesihatan. Peranti IoT seperti sensor load cell dan sensor ultrasonik yang disambungkan kepada mikropengawal ESP8266 digunakan untuk mengukur berat dan ketinggian kanak-kanak secara tepat. Nilai ini dihantar melalui rangkaian Wi-Fi ke pelayan setempat, di mana data disimpan dan diproses. Sistem akan mengira BMI dan menilai status kesihatan kanak-kanak. Dengan menggunakan Google AI Studio API, sistem ini menjana cadangan makanan dan pengambilan kalori yang diperibadikan berdasarkan umur, BMI, dan tahap aktiviti fizikal pengguna.

Aplikasi mudah alih yang dibangunkan menggunakan Android Studio menyediakan antara muka interaktif yang membolehkan pengguna melihat data kesihatan, menjejak kemajuan, dan menerima maklum balas secara masa nyata. Panel kawalan pentadbir berasaskan web yang dibina menggunakan PHP asli pula membolehkan pentadbir atau profesional kesihatan memantau dan mengurus data pengguna. Sistem ini dijalankan di atas pelayan XAMPP setempat dan menggunakan pangkalan data MySQL untuk penyimpanan dan pengurusan data yang cekap.

Sistem NutriGuardian secara signifikan meningkatkan kecekapan, ketepatan, dan pemeribadian dalam pemantauan kesihatan kanak-kanak. Dengan menggabungkan teknologi mudah alih, IoT dan AI, projek ini menyokong kesedaran kesihatan awal, membantu dalam membuat keputusan pemakanan yang lebih bijak, dan mengurangkan beban penjejakan manual kepada ibu bapa. Hasilnya ialah satu platform yang boleh dipercayai dan boleh dikembangkan untuk memperkasakan keluarga dalam penjagaan kesihatan anak-anak secara proaktif.

TABLE OF CONTENTS

	PAGE
DECLARATION.....	II
DECLARATION.....	II
DEDICATION.....	III
ACKNOWLEDGEMENTS.....	IV
ABSTRACT	V
ABSTRAK	VI
TABLE OF CONTENTS.....	VIII
LIST OF TABLES	XIV
LIST OF FIGURES	XVI
LIST OF ABBREVIATIONS	XIX
LIST OF ATTACHMENTS.....	XX
CHAPTER 1: INTRODUCTION	21
1.1 Introduction.....	21
1.2 Problem Statement(s).....	22
1.3 Objectives	23
1.4 Scope.....	24
1.5 Project Significance	25
1.6 Expected Output.....	25
1.7 Conclusion	26
CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY.....	28

2.1	Introduction.....	28
2.2	Facts and Findings	29
2.2.1	Domain	29
2.2.2	Existing System	30
2.2.3	Technique	34
2.3	Project Methodology.....	35
2.4	Project Requirements	38
2.4.1	Software Requirements.....	38
2.4.2	Hardware Requirement.....	42
2.4.3	Other Requirement.....	43
2.5	Project Schedule and Milestones	44
2.6	Conclusion	48
	CHAPTER 3: ANALYSIS.....	49
3.1	Introduction.....	49
3.2	Problem Analysis	50
3.3	Requirement Analysis	53
3.3.1	Data Requirement	53
3.3.2	Functional Requirement.....	57
3.3.2.1	Context Diagram for NutriGuardian Application.....	58
3.3.2.2	Data Flow Diagram for NutriGuardian Application.....	58
3.3.2.3	Data Flow Diagram – Level 1 for NutriGuardian	59
3.3.3	Non-functional Requirement:	60
3.3.4	Other Requirement.....	61
3.4	Conclusion	64

CHAPTER 4: DESIGN.....	65
4.1 Introduction.....	65
4.2 High-Level Design.....	66
4.2.1 System Architecture.....	67
4.2.2 User Interface Design	69
4.2.2.1 Navigation Design	74
4.2.2.2 Input Design.....	78
4.2.2.3 Output Design.....	79
4.2.3 Database Design	81
4.2.3.1 Conceptual and Logical Database Design	82
4.2.3.2 Data Dictionary.....	86
4.3 Detailed Design.....	91
4.3.1 Software Design.....	92
4.3.1.1 BMI Calculation Module:.....	92
4.3.1.2 Calorie Tracking Calculation Module Using BMI	93
4.3.1.3 AI Recommendation Module	94
4.3.2 Physical Database Design.....	98
4.4 Conclusion	100
CHAPTER 5: IMPLEMENTATION	101
5.1 Introduction.....	101
5.2 Software Development Environment Setup.....	102
5.3 Version Control Procedure	105
5.4 Implementation Status	107

5.5	Conclusion	110
CHAPTER 6: TESTING.....		111
6.1	Introduction.....	111
6.2	Test Plan.....	111
6.2.1	Test Organization.....	112
6.2.2	Test Environment.....	113
6.2.2.1	Environment of Testing	114
6.2.2.2	Hardware Setup	114
6.2.2.3	Firmware Configuration	115
6.2.2.4	Preparation and Training Before Testing	115
6.2.3	Test Schedule.....	115
6.3	Test Strategy	118
6.3.1	Classes of Test	119
6.4	Test Design	120
6.4.1	Test Description.....	121
6.4.1.1	Test Description (User Module in Mobile Application).....	121
6.4.1.2	Test Description (Interface and Navigation Module in Mobile Application)	122
6.4.1.3	Test Description (Calorie Record Module in Mobile Application) 124	
6.4.1.4	Test Description (IOT Module in Mobile Application)	124
6.4.1.5	Test Description (Meal Plan Module in Mobile Application).....	125
6.4.1.6	Test Description (Child Module in Mobile Application)	126

6.4.1.7 Test Description (History Record Module in Mobile Application)	128
6.4.1.8 Test Description (User Authentication Module in Web Application)	128
6.4.1.9 Test Description (Interface Module in Web Application).....	129
6.4.1.10 Test Description (Parent Module In Mobile Application).....	130
6.4.1.11 Test Description (Food Module In Web Application).....	131
6.4.2 Test Data	132
6.4.2.1 Test Data for Register Page	134
6.4.2.2 Test Data for Login Page	136
6.4.2.3 Test Data for Reset Password Page	137
6.4.2.4 Test Data for Calorie Record Page	138
6.4.2.5 Test Data for IoT Measuring Page.....	138
6.4.2.6 Test Data for IoT Hardware Measuring.....	140
6.4.2.7 Test Data for Meal Plan Page	142
6.4.2.8 Test Data for Meal Suggestion By AI	143
6.4.2.9 Test Data for Manage Child User	145
6.4.2.10 Test Data for Add New Child User	149
6.4.2.11 Test Data for History Record.....	150
6.4.2.12 Test Data for User Authenticate Admin	152
6.4.2.13 Test Data for Parent Module.....	153
6.4.2.14 Test Data for Manage Food Page	156

6.4.2.15 Test Data for Add New Food.....	159
6.5 Test Result and Analysis.....	163
6.6 User Acceptance Testing (UAT)	169
6.7 Conclusion	177
CHAPTER 7: CONCLUSION	179
7.1 Observation and Weakness Strength	179
7.2 Proposition and Improvements	180
7.3 Project Contribution.....	182
7.4 Conclusion	183
REFERENCES.....	185
APPENDIX A	187

LIST OF TABLES

	PAGE
Table 1.1.1: Summary of Problem Statement	23
Table 2.1.2: Existing System Comparison	33
Table 4.1: Database Table Parent.....	86
Table 4.2: Database Table Child.....	87
Table 4.3: Database Table Record	88
Table 4.4: Database Table Calorie Record	88
Table 4.5: Database Table Recommended.....	89
Table 4.6: Database Table Food.....	89
Table 4.7: Database Table Food Intake.....	90
Table 4.8: Database Table User	90
Table 5.1: Implementation Table for Mobile Applications	107
Table 5.2: Implementation Table for Web Applications	109
Table 6.1: Test Organization.....	112
Table 6.2: Test Schedule for Mobile Applications	116
Table 6.3: Test Schedule for Web Application.....	117
Table 6.4: Test Strategy	118
Table 6.5: Classes of Testing	119
Table 6.6: Test Description for User Module	121
Table 6.7: Test Case Description for Interface and Navigation Module	123
Table 6.8: Test Case Description for Calorie Record	124
Table 6.9: Test Case Description for IOT Module	124
Table 6.10: Test Case for Meal Plan Module	125
Table 6.11 Test Description for Child Module	127
Table 6.12: Test Description for History Record Module	128
Table 6.13 Test Description for User Authentication Module.....	129

Table 6.14: Test Description for Interface Module in Web Application	129
Table 6.15: Test Description for Parent Module in Web Application	130
Table 6.16: Test Description for Food Module.....	131
Table 6.17: Test Pass And Fail Result and Analysis For All Module	163
Table 6.18: The Summary of Test Result and Analysis For All Module	168

LIST OF FIGURES

	PAGE
Figure 2.1: Agile Methodology	36
Figure 2.2: Flutter	38
Figure 2.3: Android Studio	38
Figure 2.4: Xampp	39
Figure 2.5: PhpMyAdmin	39
Figure 2.6: Figma	40
Figure 2.7: Tinkercad	40
Figure 2.8: Arduino IDE	41
Figure 2.9: Microsoft Word	41
Figure 2.10: GitHub	42
Figure 2.11: Gantt Chart	47
Figure 3.1: DFD Level 0 – Current System	51
Figure 3.2: Data Modelling	56
Figure 3.3 Context Diagram	58
Figure 3.4: DFD Level 0	58
Figure 3.5: DFD Level 1	59
Figure 4.1: System Architecture	67
Figure 4.2: Loading page	70
Figure 4.3: Leading page	70
Figure 4.4: Register page	70
Figure 4.5: Choose page	70
Figure 4.6: Homepage	71
Figure 4.7: Login page	71
Figure 4.8: Graph Daily Calorie Page	71
Figure 4.9: Daily Calorie Page	71
Figure 4.10: Child Meal Plan	72

Figure 4.11: Meal Plan.....	72
Figure 4.12: Child page.....	72
Figure 4.13: Account Page.....	72
Figure 4.14: Profile page.....	73
Figure 4.15: Add family members form	73
Figure 4.16: Record page	73
Figure 4.17: Graph record page	73
Figure 4.18: Navigation Flow	75
Figure 4.19: Bottom Bar	76
Figure 4.20: Back Button	77
Figure 4.21: Register Account	78
Figure 4.22: Register Account	78
Figure 4.23: Weight and BMI graph.....	79
Figure 4.24: Height and Calorie Graph.....	79
Figure 4.25: Entity Relationship Diagram	83
Figure 4.26 BMI Calculation Code.....	92
Figure 4.27 Calorie Calculation Code.....	93
Figure 4.28 Initialize function.....	95
Figure 4.29 Apply the function of probability age and weight.....	95
Figure 4.30 Normalize the probability	96
Figure 4.31 Decision based on probability for priority.....	96
Figure 4.32 Decision Food based on probability	96
Figure 4.33 Database Management System	99
Figure 5.1: Software Environment Architecture Diagram.....	104
Figure 6.1: Total Responder of System	133
Figure 6.2: The Percentage Category of User.....	170
Figure 6.3: The percentage of used digital health system before.....	171
Figure 6.4: The percentage of easily can understand system works	171
Figure 6.5: The percentage of accuracy of measurement	172
Figure 6.6: The percentage of automatic measurements tools.....	173
Figure 6.7: The percentage of likely would use the system regularly	173
Figure 6.8: The percentage of system user-friendly and safe to use.....	174
Figure 6.9: The percentage of the recommended system to their friend.....	175
Figure 6.10: The percentage of easily to remember to use the application.....	175

Figure 6.11: The percentage of learn applications quickly	176
Figure 6.12: The percentage of suggestion for the extra features	177

LIST OF ABBREVIATIONS

FYP - **Final Year Project**

LIST OF ATTACHMENTS

		PAGE
Appendix A	Sample of data	19
Appendix B	Analysis of data collection	78
.....		
.....		

CHAPTER 1: INTRODUCTION

1.1 Introduction

The fast-paced nature of modern life creates obstacles for families with young children to follow a healthy lifestyle. The combination of fast-food abundance and busy lives and insufficient nutrition knowledge makes it difficult for parents to provide their children with proper nutrition and physical activity. Children develop unhealthy eating habits because of this situation which produces obesity and health issues including diabetes and poor self-esteem. Developing NutriGuardian as a smart health monitoring system to assist families in making improved and healthier decisions throughout their daily routines.

The NutriGuardian system integrates a smart weight scale with a mobile application and connected website to deliver an uninterrupted health tracking experience. The system enables users to measure their weight and BMI automatically before providing AI-based personalized food recommendations through the Google AI Studio API. The main advantage of NutriGuardian lies in its user-friendly design which provides both accessibility and engagement for its users. The system features an interface that serves both children and parents by providing an easy-to-use platform that remains enjoyable throughout its use.

Through their mobile devices families can track their progress and receive personalized food recommendations and stay motivated by accessing real-time updates and historical data tracking. The NutriGuardian system functions as a supportive tool

which assists families in developing small yet significant improvements for their health. NutriGuardian uses contemporary technology to provide functional solutions which enable users to establish enduring healthy practices during childhood while promoting better nutrition awareness and wellness practices.

1.2 Problem Statement(s)

In today's digital age, childhood health issues such as obesity, poor diet, and lack of physical awareness are becoming increasingly prevalent. Many families struggle to monitor their children's health effectively due to the absence of personalized and accessible tools. While numerous mobile applications and devices exist, they often function in isolation, lack real-time data analysis, and are not tailored for children's engagement. Traditional methods of tracking weight and dietary intake are either too general, too manual, or not user-friendly. Furthermore, health applications rarely integrate AI to provide smart and adaptive suggestions, nor do they connect seamlessly across platforms. This project aims to address these gaps by introducing NutriGuardian, a smart health monitoring system that combines IoT, mobile and AI to support children and families in managing their nutrition and well-being more effectively.

PS	Problem Statement
PS ₁	Lack of an integrated and personalized system to monitor children's health and nutrition.
PS ₂	Difficulty for parents to receive accurate, real-time dietary recommendations tailored to their child.
PS ₃	Existing health monitoring tools lack engagement and are not child-friendly in design or function.
PS ₄	Limited accessibility of tools that combine IoT (smart devices) and AI to provide actionable health data.

PS ₅	Inconvenience in manually tracking weight, BMI, and food intake over time.
-----------------	--

Table 1.1.1: Summary of Problem Statement

1.3 Objectives

- To develop a smart health monitoring system using IoT technology
 - Create a system that connects a smart weight scale to a mobile app via Bluetooth or Wi-Fi to automatically track users' weight and height.
- To calculate and monitor BMI based on real-time data
 - The system will compute BMI using the data collected and present it in a way that is easy for both parents and children to understand.
- To provide AI-powered personalized food intake suggestions
 - Use Google AI Studio API to generate diet recommendations tailored to the user's age, weight, activity level, and nutritional needs.
- To implement a user-friendly interface suitable for both children and parents
 - Design a simple, engaging UI/UX that makes health tracking fun and accessible for young users, while still informative for adults.
- To enable historical tracking and progress monitoring
 - Allow users to view trends in their health data over time, such as weight history and calorie intake, through visual graphs and logs.
- To ensure cross-platform accessibility via mobile app and web integration
 - Develop both mobile and web interfaces to improve user access and convenience, allowing data to be synchronized across devices.

1.4 Scope

- Target Users:
 - i. Children (Age: 5-12 years old).
 - ii. Parent
 - The system is designed to help parents monitor and guide their children's health and for themselves, particularly focusing on early nutrition and growth during primary school years.
- Mobile Application (Android-based)
 - The system will be developed for Android smartphones, providing a platform for users to interact with their health data, get dietary suggestions, and view their BMI.
- Smart Weight and Height Scale Integration via IoT
 - The system will use a Wi-Fi-enabled smart weight and height scale to automatically send weight and height data to the mobile app. This eliminates manual input and ensures data accuracy.
- Body Mass Index (BMI) Calculation Module
 - BMI will be calculated using the collected height and weight data, providing instant feedback on whether the child's weight is within a healthy range.
- Historical Data Tracking
 - The app will store and display a history of weight, BMI, and recommended food intake so parents and children can monitor their progress over time.
- User Interface Designed for Both Children and Parents
 - The interface will be simple, visual, and friendly to engage young users, while also including detailed views and settings for parents to manage and review health data.

1.5 Project Significance

NutriGuardian system is designed to bring meaningful benefits to families, particularly parents, children and also their guardians use the system by providing a smart and user-friendly solution for storing and managing their health record. One of the main benefits of this project is parents, who often struggle to monitor their children's dietary habits and growth effectively. With NutriGuardian, they can access real-time data on weight, height and assist to find the BMI and straight receive AI-powered food recommendations based on BMI receiving. And it helps to track their members under the account to see the progress over time—all through a single mobile application. For children aged 5 to 12, the system offers an engaging and educational platform that introduces them to healthy habits from a young age, encouraging better lifestyle choices in a fun and interactive way.

Healthcare professionals can also benefit indirectly for the food suggestion, as the system enables families to guide the parent to choose the right meal and consistent health records from clinical visits, allowing for more informed consultations and early intervention when necessary. Beyond individual use, this project supports broader public health goals by promoting awareness, prevention, and healthier living. By aligning with the objectives of real-time monitoring, personalized guidance, and accessible technology, NutriGuardian serves as a practical tool that empowers families to take control of their health journey and contributes to the long-term well-being of the community.

1.6 Expected Output

The NutriGuardian project is expected to deliver a fully functional mobile application integrated with a smart weight scale, designed specifically to support

children's health monitoring and also for the parents to guide their calories by self. The system has some many features which can collect height and weight data through IoT hardware and calculate Body Mass Index (BMI) automatically. One of the key outputs is the ability of mobile applications to generate personalized food and calorie intake recommendations using Kalman Filter and Fuzzy Logic, based on user data such as age, weight, and activity level.

The system will also include a historical tracking feature that allows users to monitor their health progress over time, presented through a user-friendly interface suitable for both children and parents. Additionally, the project is expected to promote awareness about healthy eating and early weight management among families, with the goal of preventing childhood obesity. Overall, the NutriGuardian system will serve as an accessible and intelligent health companion that empowers users to make informed dietary and lifestyle decisions.

1.7 Conclusion

In conclusion, this chapter introduced the NutriGuardian project, a smart health monitoring system aimed at supporting children's nutrition and overall well-being using IoT and AI technologies. The background has highlighted the growing concern of childhood obesity and the lack of accessible, personalized health monitoring tools for families. The problem statements, objectives, scope, significance, and expected outputs have all been outlined to provide a clear understanding of the project's direction and purpose. NutriGuardian aims to fill the gap by offering a mobile application that integrates with a smart weight scale and utilizes AI to generate personalized dietary recommendations.

Moving forward, the next activities will involve conducting a literature review to explore existing technologies, systems, and research related to health monitoring, IoT, and AI in nutrition. This will be followed by the development of the project methodology, which outlines the techniques, tools, and processes that will be used to design, implement, and evaluate the NutriGuardian system.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

To ensure the successful development of the NutriGuardian system, it is important to first understand the existing technologies, approaches, and solutions related to health monitoring, dietary recommendation systems, and the integration of IoT and AI in mobile applications. The upcoming literature review will examine previous research studies, related systems, and technological frameworks that provide a foundation for this project. It will also help identify gaps, strengths, and opportunities for innovation, especially in terms of personalized health tracking for children.

Following the literature review, the project methodology will be discussed. This section will describe the development process, tools, and techniques used to build the NutriGuardian system. It will include the chosen software and hardware platforms, system architecture, design approaches, testing methods, and implementation steps. Together, these next chapters will form the technical and conceptual basis for the project's development and ensure that it is grounded in relevant knowledge and effective practices.

2.2 Facts and Findings

2.2.1 Domain

The NutriGuardian app is a part of the Health and Wellness Technology field, and its primary areas of concentration include computerized health monitoring and intelligent nutrition management. To assist and enhance the health of people as well as the public, this field makes use of technology such as mobile applications, internet of things (IoT), and artificial intelligence (AI). The rise in the prevalence of unhealthy eating habits and obesity among children has resulted in a significant demand for digital health solutions that are not only user-friendly but also intelligent and accessible to all individuals. Through the provision of tools that enable individuals and families to collect data in real time and obtain individualized insights that assist them in monitoring, managing, and enhancing their health, health and wellness technology can fulfill these requirements.

This project also has some connection to the Internet of Things (IoT) since it incorporates a smart weight scale that transmits data directly to a mobile application. The application of artificial intelligence (AI) by Google AI Studio places the project within the AI in Healthcare subdomain. This subdomain is focused on the application of AI to improve the accuracy of health-related applications in terms of forecasting outcomes, providing personalized recommendations, and assisting individuals in making intelligent choices. NutriGuardian is working in this field to assist individuals in managing their health and to contribute to the greater public health aim of reducing the number of diseases that can be prevented by early intervention and improved lifestyle choices.

2.2.2 Existing System

NutriGuardian is a system with the domain is related with Health and Wellness Technology that particularly focuses on Digital Family Health Monitoring, Personalized Nutrition Systems, and IoT-Enabled Healthcare. This domain integrates mobile technology, artificial intelligence (AI), and Internet of Things (IoT) to offer smart solutions for monitoring and improving the health of children. As health awareness rises, especially regarding childhood obesity and nutrition, this domain becomes increasingly relevant for creating solutions that are intelligent, accessible, and engaging.

To support the development of the NutriGuardian system, various research articles and projects have been reviewed. The approaches used in these studies provide valuable insights into how similar systems function, their effectiveness, and technologies used.

- i. Reading 1: Mobile-Based Nutrition and Child Health Monitoring
 - This project implemented a mobile-based system to collect real-time data on children's health and nutrition. It demonstrated how mobile apps can support public health programs through real-time updates and easy data access. This supports NutriGuardian's approach to using a mobile app as the core platform for health tracking.
- ii. Reading 2: AI Nutrition Recommendation Using Deep Generative Models
 - This article provides evidence for the effectiveness of AI in generating personalized dietary suggestions. The NutriGuardian system applies a similar method using Google AI Studio to offer tailored food intake recommendations.

iii. Reading 3: IoT Wearables in Child Health: A Scoping Review

- This review highlights how IoT devices (e.g., smart scales and wearables) benefit children's healthcare by automating monitoring and making health data more accessible. NutriGuardian uses a smart scale connected via Bluetooth/Wi-Fi to automate data collection.

iv. Reading 4: Community-Based Child Nutrition Projection and Monitoring

- This project used simulation and trend analysis for predicting nutrition outcomes. Similarly, NutriGuardian tracks historical health data and aims to help parents and children view their progress over time.

v. Case Study 5: NutriBot – AI-Powered Nutrition Chatbot

- NutriBot used an AI-based chatbot to deliver personalized food suggestions. NutriGuardian applies the same logic of user profiling and data-driven meal planning through AI, though not in chatbot form.

vi. Reading 6: Mobile Health for Pediatric Weight Management

- This scoping review analyzes various mobile health (mHealth) interventions aimed at treating childhood overweight and obesity. It confirms the growing use of smartphone-based systems to support weight management, including features like dietary tracking and physical activity monitoring, echoing NutriGuardian's mobile-first approach

- vii. Reading 7: Effectiveness of Smartphone App for Pediatric Obesity Treatment
 - This recent randomized controlled trial (M App) evaluated a 6- to 12-month app-based lifestyle intervention. Outcomes showed improved adherence and reduced drop-out rates, validating the potential of mobile apps to engage young users in healthy routines

- viii. Reading 8: AI-Assisted Dietary Assessment Tools
 - A scoping review on AI-powered dietary tools highlights the use of food image recognition and wearable sensors for real-time food intake monitoring. These match NutriGuardian's goals of integrating AI and IoT into daily nutrition tracking

- ix. Reading 9: Challenges of Diet Planning for Children Using AI
 - This study showcases the use of generative AI (GANs, reinforcement learning) to plan nutritionally compliant diets for preschool-aged children, emphasizing the importance of AI in providing balanced meal plans, closely aligned with NutriGuardian's objectives

Reading. No	Author	Method/Approach	Framework Used	Tools/Tech (IoT/AI)	Web App	Mobile App
1	Guyon. A Buback.L Knittel/B	Real-time health data collection	Custom system	Mobile, database		/
2	Papastratis.L Konstnttinidis.D Daras.P	AI-based meal generation	Deep generative models	AI algorithm, datasets		/
3	Kajal Mistry	IoT health tracking	Review study	Smart wearables, sensors	/	/
4	Larisan J Queje.B Maglangit.N	Simulation and trend analysis	Custom simulator	Data trend prediction	/	
5	Vinaj Kumar	AI chatbot diet suggestions	Rasa Framework	AI, NLP, user profiling	/	/
6	Tully.L Sorensen.J	mHealth intervention	Literature review	Health tracking		/
7	Umano.G Masino.M	App-based lifestyle coaching	Experimental trial	Mobile sensors, logging		/
8	Phalle.A Gokhale.D	AI-powered food tracking	AI tools	Food image recognition, sensors		/
9	Changhu, Lee	AI-based meal planning	GAN, Reinforcement Learning	AI logic, data rules		/
—	NutriGuardian: Smart Health Monitoring System (Proposed Project)	IoT health monitoring + AI meal suggestions	Android SDK, Google AI Studio	Smart Scale, AI	/	/

Table 2.1.2: Existing System Comparison

2.2.3 Technique

While NutriGuardian adopts a mobile-based solution integrated with IoT devices and AI-driven recommendations, there are several other technological approaches that are also applicable and relevant to child health monitoring systems. However, these alternatives were not selected due to specific limitations and mismatches with the project's goals.

One common approach is the use of a web-based health monitoring dashboard, which allows data visualization and centralized control for healthcare professionals and parents. While this method is effective for detailed health analysis and multi-user access, it is not ideal for real-time weight tracking through Bluetooth-enabled smart scales. Additionally, considering the primary users of NutriGuardian are children and parents using mobile devices, a mobile-first strategy offers greater portability, ease of use, and tighter integration with IoT devices.

Another potential approach is the integration of a chatbot-based nutrition assistant, like the NutriBot project. Chatbots can simplify interactions through conversational interfaces, using natural language processing (NLP) to suggest meals and answer health-related queries. However, this approach was not chosen because it is more suited for adult users. Children may find text-based chatbots less engaging, and the additional complexity of training an NLP system was not necessary for the project's current objectives. NutriGuardian instead focuses on a visual and intuitive interface suitable for younger users.

The use of wearable health trackers like smartwatches is also an alternative worth noting. These devices can track steps, heart rate, and other biometric data continuously. However, they are typically expensive and less accessible to families,

especially for children. In contrast, a smart weight scale is a more practical, cost-effective solution that still supports the core functionality of the project — monitoring growth through weight and BMI.

Lastly, some systems rely on manual food logging combined with calorie databases without AI. While this method is widely used in health apps, it requires users to manually search and input every food item, which is time-consuming and susceptible to error. This method is also less engaging, particularly for young users. By incorporating AI through Google AI Studio, NutriGuardian can generate personalized suggestions automatically, creating a more intelligent and user-friendly experience.

In summary, while these alternative approaches have their own benefits, they were not selected due to considerations around cost, usability for children, engagement, and technical simplicity. NutriGuardian prioritizes accessibility, automation, and child-friendly interaction, making its chosen approach the most appropriate for the intended goals and users. Give reason to justify why not.

2.3 Project Methodology

For the development of the NutriGuardian system, the Agile software development methodology was selected due to its flexibility, iterative nature, and strong focus on user feedback. Agile is particularly effective for projects that involve continuous improvements, user testing, and rapid development cycles, all of which are important for building an engaging, user-friendly health monitoring system targeted at both children and parents.

Agile promotes iterative development, where the project is broken down into small, manageable units called sprints, each typically lasting one to two weeks. At the end of every sprint, a working version of the product is reviewed, tested, and refined based on feedback. This approach ensures that any issues can be quickly identified and addressed, and that the system evolves in response to real user needs.



Figure 2.1: Agile Methodology

The Agile approach in this project is divided into several stages:

i. Planning & Requirement Gathering

In this stage, the project team identifies user needs, system requirements, and features. Key features such as smart weight scale integration, BMI calculation, and AI-powered food suggestions are prioritized. Interviews with potential users (e.g., parents) and consultation related research help shape the system's goals.

ii. Design

Using tools like Figma, the UI/UX of the mobile app is designed with a focus on being child-friendly and intuitive. The system architecture is also planned, including how

data from the IoT device (smart scale) will flow into the mobile app and connect with AI services.

iii. Development

The development process is carried out in multiple sprints. The first sprint focuses on building the login and user profile module. The second sprint develops the integration with the smart weight scale and BMI calculation. Later sprints involve linking the Google AI Studio API for food recommendation and setting up historical tracking features.

iv. Testing

At the end of each sprint, unit and user testing is performed to ensure each feature works correctly. Feedback from testers is collected to make improvements in the next sprint cycle.

v. Deployment & Review

Once all core features are complete, the system is deployed on Android devices. Final evaluations are conducted, including user experience assessments and performance analysis.

vi. Documentation & Final Report

A final documentation phase ensures that the design, code, and user manuals are properly recorded. The report reflects the entire development process and findings from user feedback.

Agile is supported by many practitioners and researchers in software development. According to Highsmith (2002), “Agile development focuses on delivering value to customers through collaboration, adaptive planning, and rapid response to change.” Similarly, a study by Dingsøyr et al. (2012) confirms that “Agile methodologies increase productivity and software quality, especially in projects requiring frequent feedback and flexibility.” By applying Agile, NutriGuardian

benefits from continuous testing and feedback, resulting in a more refined and reliable product. This approach is ideal for a system involving children, where usability and functionality need to be continuously improved based on actual user interaction.

2.4 Project Requirements

2.4.1 Software Requirements

To successfully develop and manage the NutriGuardian system, several software tools and platforms are required.

- Android Studio



Figure 2.2: Flutter



Figure 2.3: Android Studio

The primary development environment will be Android Studio, which supports building native Android applications using Java or Kotlin. This platform offers comprehensive tools for designing, coding, debugging, and testing the mobile application that will serve as the core of the system. For the AI-driven dietary

recommendation feature, Google AI Studio (integrating the Gemini API) will be used. This tool allows the development team to build, test, and deploy artificial intelligence models capable of generating personalized food suggestions based on user profile data such as age, weight, and activity level.

- Localhost: Xampp



Figure 2.4: Xampp



Figure 2.5: PhpMyAdmin

For storing user information, tracking historical data, and managing application records, a local database hosted on localhost will be used instead of a cloud-based solution. Specifically, XAMPP will serve as the local server environment, while MySQL will be used to create and manage the database. This approach is suitable for offline development and testing, allowing the project team to build and verify database operations without relying on internet connectivity. It also simplifies data handling for a student project and supports integration with Android applications via PHP scripts or REST APIs.

- Figma



Figure 2.6: Figma

In the design phase, Figma will be used to create wireframes and interactive prototypes for the mobile user interface. Figma supports collaborative design, which is essential for aligning with user feedback during Agile development cycles.

- Tinkercad



Figure 2.7: Tinkercad

To support the IoT component of the NutriGuardian systems specifically for integrating the smart weight scale and smart height measurement—two key software tools are used: Tinkercad and the Arduino IDE. Tinkercad, developed by Autodesk, is a web-based simulation platform that allows users to virtually design and test electronic circuits. It is particularly useful in the early prototyping stage, as it enables the simulation of sensor behavior without requiring physical components. In this project, Tinkercad is used to model the connection between sensors (such as load cells for weight and ultrasonic sensors for height) and the ESP8266 microcontroller, helping to visualize how data is collected and transmitted wirelessly to the backend system.

- Arduino IDE



Figure 2.8: Arduino IDE

Once the circuit logic is validated through simulation, the Arduino IDE is used to write, compile, and upload the actual program code to the ESP8266 board. The Arduino IDE provides full support for the ESP8266 through board manager extensions and supports integration with sensor libraries such as HX711 for load cells and standard ultrasonic sensor libraries. In this project, the ESP8266 is programmed to collect weight and height data from the connected sensors, process the data, and transmit it via Wi-Fi using HTTP requests to a PHP API hosted on a local server (XAMPP). The Arduino environment also includes a serial monitor for real-time testing and debugging of sensor readings and Wi-Fi connectivity. Together, Tinkercad and the Arduino IDE provide a complete workflow for designing, testing, and implementing the IoT functionality in the NutriGuardian system, enabling accurate and automated data collection directly from smart devices.

- Microsoft Word



Figure 2.9: Microsoft Word

Documentation tasks will be handled using Microsoft Word or Google Docs, which are essential for writing the proposal, system reports, user manuals, and other required documentation throughout the project. For organizing development tasks and tracking progress in an Agile environment, tools like Trello, Jira, or Notion will be used. These platforms help structure the project timeline into sprints and ensure tasks are completed on schedule.

- GitHub



Figure 2.10: GitHub

To maintain version control and support team collaboration on coding, GitHub will be used. It allows tracking changes, managing branches, and coordinating development between multiple contributors. For testing the connection and accuracy of data exchange between the app and external services like the AI API, Postman will be used. It enables developers to simulate and debug API requests efficiently.

Together, these software tools support every phase of the system development life cycle — from design and coding to testing, deployment, and maintenance — ensuring a smooth, efficient, and structured workflow for the project.

2.4.2 Hardware Requirement

- Personal Computer's/Laptop

Laptop: Asus Tuf Gaming F15

- Processor (CPU) : Intel(R) Core (TM) i5-10300H CPU @2.5GHz
- RAM (Memory) : 16.0 GB
- Storage (SDD) : 1 TB
- Operating System : Windows 11 Home Single Language
- Network Capability : Wi-Fi and LAN enabled

- Supported tools

Mobile Devices: Oppo F11

- Processor (CPU) : MediaTek Helio P70 Octa-Core

- RAM (Memory) : 4GB
- Storage : 128GB internal storage
- Operating System : ColorOS 6
- Network Capability : 4G LTE, Wi-Fi 802.11

3D Printer: Elegoo Neptune 4 pro

- Type: FDM (Fused Deposition Modeling) 3D Printer
- Build Volume: 225 x 225 x 265 mm
- Printing Speed: Up to 500 mm/s (recommended: 250 mm/s for quality)
- Layer Resolution: 0.1 – 0.4 mm
- Filament Diameter: 1.75 mm
- Nozzle Diameter: Standard 0.4 mm (can be replaced)
- Max Nozzle Temperature: 300°C
- Max Heated Bed Temperature: 100°C

- Server

Local Server (via XAMPP on PC/Laptop)

2.4.3 Other Requirement

- Discussion Room.

To make regular discussions with the supervisor more effective and focused, a discussion room is needed throughout the project implementation. This space provides a quiet and distraction-free environment where the project team can conduct weekly progress reviews, receive feedback, and clarify technical or conceptual issues directly with the supervisor. It also serves as a suitable place for group meetings, planning sessions, and collaborative troubleshooting during critical development phases. Having access to a dedicated discussion room helps improve communication, decision-making, and the overall quality of the project work.

- Library

To support both research and development activities throughout the project. It provides access to academic resources such as journals, e-books, and research papers that are valuable for the literature review and fact-finding phases. Moreover, the library offers a conducive environment for focused work, which is especially useful during system development and coding tasks. Many university libraries also provide stable internet access, power outlets, and quiet zones, making them ideal for implementing and testing parts of the system without distractions. This environment helps enhance productivity, accuracy, and concentration during key stages of the project.

2.5 Project Schedule and Milestones

The Implementation of NutriGuardian project has been following and structured in 14 weeks and being reported in timeline, organized into phases that align with fundamental project management principles such as initiating, planning, executing, monitoring, and closing. Each phase is designed to ensure the project progresses smoothly from idea to implementation and final presentation.

- First Phase: (Week 1 – Week 2)

In this phase is to focus on preparing the project proposal. During this stage, the problem is identified, objectives are defined, and the overall scope of the system is discussed with the supervisor. A written proposal is prepared and submitted for approval. This phase is crucial as it sets the foundation for all future work and ensures the project is feasible and aligned with academic expectations.

- Second Phase: (Week 2 – Week 3)

In this phase, it is to make analysis and design about the structure of the initial system that to be developed. This includes conducting a literature review, gathering requirements, analyzing existing systems, and designing the system architecture. During this stage, wireframes are created using Figma and the roles of IoT, AI, and mobile application components are clearly defined. This phase helps the developer understand the technical needs and layout of the system before writing any code.

- Third Phase: (Week 4 – Week 5)

In this phase in focusing on shifts to Report Writing Progress 1, where the initial chapters of the report are written which cover chapter 1 until 3. These chapters include the introduction, problem statement, objectives, scope, significance, and literature review and the analysis of the system that has been developed. This allows proper documentation of the research and background information before moving into the development stage. In this report it was created to cover the process of implementing the system and how the system works.

- Fourth Phase: (Week 6 – Week 12)

In this phase is the most critical, involving Application/System Development Progress. The system is developed in stages, beginning with basic features like user login and progressing to IoT integration, BMI calculation, and AI-based food suggestion. The mobile application is tested using real devices, and a web-based admin panel is developed for monitoring data. Iterative improvements are made based on test results and supervisor feedback.

- Fifth Phase: (Week 10 - 12)

Focuses on Report Writing Progress 2, where updates to the report are made to include implementation, coding descriptions, and system screenshots. This ensures that the documentation reflects the current state of development and provides a clear picture of how the system was built.

- Sixth Phase: (Week 11 – Week 13)

The final report for evaluation is prepared. This includes finalizing all chapters, formatting the document according to university standards, proofreading, and ensuring all requirements are met. It is then submitted to the supervisor and academic committee for assessment.

- Seventh Phase (Week 8 - 13)

The demonstration of the system has been conducted between supervisors. The demonstration session to showcase key features, including smart scale integration, real-time BMI updates, AI-based food suggestions, and historical tracking. This allows evaluators to see the practical functionality of the system.

- Eight Phase: (Week 14)

The project concludes with the presentation. A formal presentation is delivered to a panel of evaluators, highlighting the problem addressed, the solution developed, technical implementation, and the system's impact. This is the final step where the project is defended and assessed for its completeness and quality.

Gantt Chart

No.	Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
1	Project proposal														
2	Project progress 1 (Analysis and Design)														
3	Report writing progress 1														
4	Application/System Development Progress.														
5	Report writing progress														
6	Report for evaluation														
7	Demonstration														
8	Presentation														

Figure 2.11: Gantt Chart

2.6 Conclusion

In conclusion, this chapter has provided a comprehensive overview of the literature, existing technologies, and related projects that form the foundation for the development of the NutriGuardian system. Through the analysis of past research, it is evident that mobile health applications, AI-driven nutrition recommendations, and IoT-enabled devices play a crucial role in improving health monitoring, particularly for children. The findings from related studies have helped shape the design of NutriGuardian by highlighting the importance of usability, real-time data processing, and personalized feedback.

Additionally, the project methodology has been clearly defined, with Agile selected as the preferred development approach. Agile's iterative nature allows for continuous improvement, user feedback integration, and flexibility in managing development tasks. Each stage—from planning, design, and development, to testing and presentation—has been structured to ensure that the project meets its goals within the allocated timeframe. By combining proven development practices with insights from existing literature, this chapter has laid a strong groundwork for the successful implementation of the NutriGuardian smart health monitoring system.

Moving forward, the next chapter will focus on the analysis of each implementation phase of the project. This includes a detailed breakdown of how the system is built and integrated, covering aspects such as mobile app development, API creation with PHP, and data handling via a local database. Each component will be analyzed to demonstrate how it contributes to achieving the project's objectives and ensuring the overall functionality of the NutriGuardian system.

CHAPTER 3: ANALYSIS

3.1 Introduction

This chapter focuses on the analysis phase of the NutriGuardian project, where each system component is evaluated in terms of its functionality, interaction, and contribution to the overall system objectives. In software development, the analysis phase is crucial as it bridges the gap between user needs and technical implementation. It allows the developer to understand how the system should behave, what features must be implemented, and how the different components—hardware, software, user interfaces, and databases—should work together.

In this phase, the functional requirements of the NutriGuardian system will be clearly identified and broken down into modules such as user registration and login, smart weight and height data collection, BMI calculation, personalized food suggestion using AI, and historical data tracking. Each feature will be analyzed from both a user and system perspective to ensure it meets the intended goals. The system architecture will also be defined, outlining the interaction between the mobile application, IoT components (Arduino), local database (MySQL), and web-based admin panel. Diagrams such as case diagrams, data flow diagrams (DFD), and context diagrams will be used to visually represent the flow of information and relationships among system elements.

3.2 Problem Analysis

The current system used by parents and children to monitor health is entirely manual, involving traditional tools such as weighing scales, measuring tapes, and handwritten notes. The child's weight is measured using a traditional scale, and height is measured using a measuring tape. These values are then manually recorded by parents into a physical logbook or notebook (Process 3.0 – Book Note for Weight and Height). There is no automation or digital assistance in the data collection, storage, or processing stages.

Following this, parents manually calculate BMI (Process 4.0), using standard BMI formulas, based on the height and weight values recorded. This process is not only time-consuming but also prone to human error, particularly in calculation and data transcription. The BMI result is written again into a separate book or note (Process 5.0 – Book Note for BMI). After that, parents proceed to manually estimate calorie needs (Process 6.0) using general references or charts, often based on guesswork without any scientific personalization.

Finally, a meal is chosen manually (Process 7.0 – Manually pick meal plan), based on the calculated calorie values. This method lacks precision and does not consider deeper personal metrics such as child age, growth pattern, nutritional needs, or activity level. There is no feedback mechanism, no data visualization, and no long-term tracking or analysis of the child's health metrics. Additionally, data is scattered across multiple notebooks, making it difficult to retrieve historical data or track progress over time.

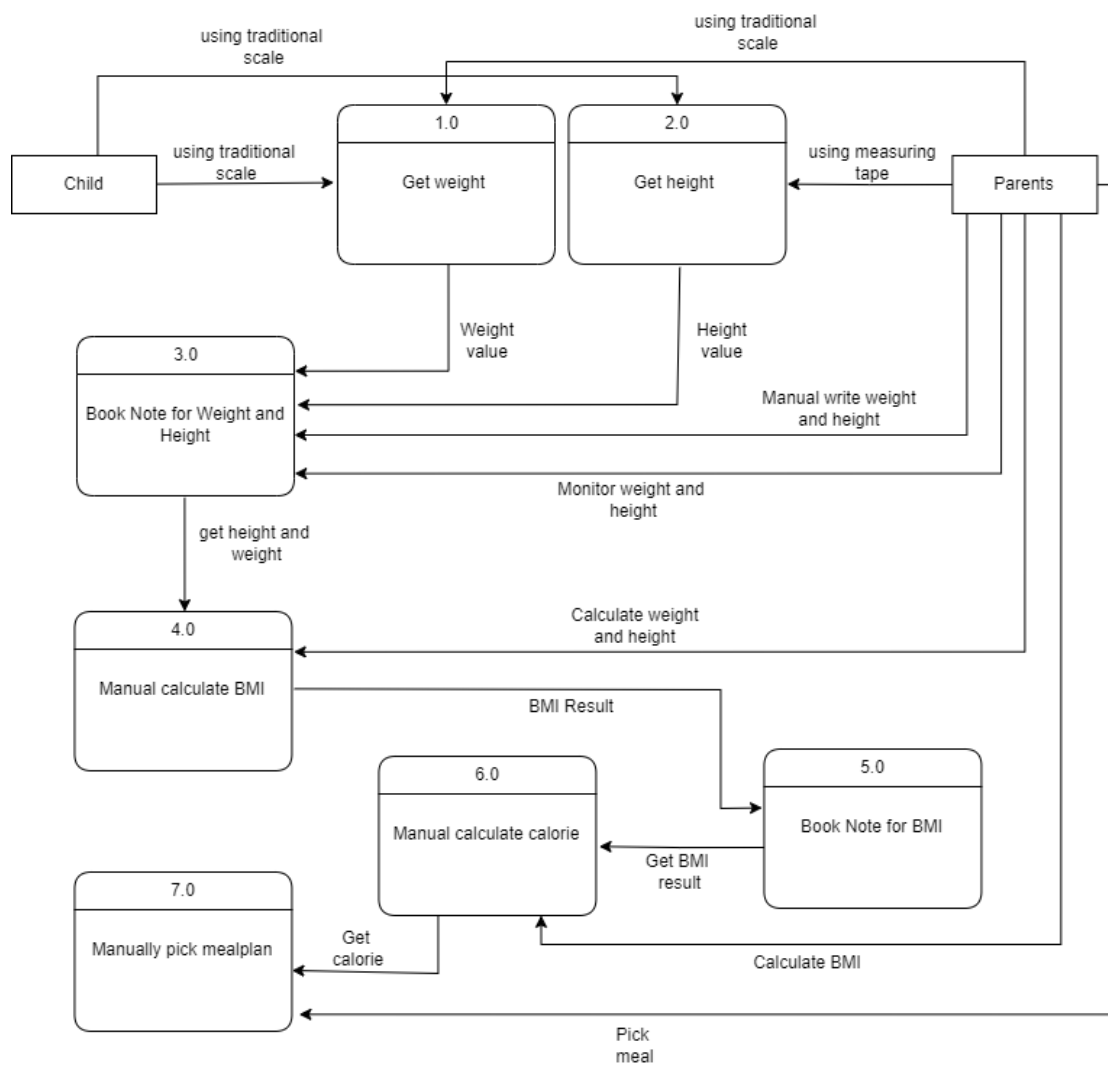


Figure 3.1: DFD Level 0 – Current System

Identified Problems (Aligned with DFD Processes)

The analysis of the current manual system, as depicted in the Data Flow Diagram (DFD Level 0), reveals several practical and technical challenges that directly correlate with the problem statements outlined in Chapter 1. These issues highlight the inefficiencies of the traditional method and the need for a more intelligent, integrated solution like NutriGuardian.

- i. Lack of an integrated and personalized system to monitor children's health and nutrition
 - There is no integrated system where parents can view, manage, and track their child's health holistically. The absence of a digital platform forces users to rely on scattered manual records, limiting the ability to provide personalized health insights.
- ii. Difficulty for parents to receive accurate, real-time dietary recommendations tailored to their child
 - Calorie needs and meal plans are determined manually, often based on assumptions or generalized information. This makes it difficult for parents to receive timely and accurate food suggestions that are tailored to their child's actual health status (e.g., age, weight, BMI).
- iii. Difficulty for parents to receive accurate, real-time dietary recommendations tailored to their child
 - Calorie needs and meal plans are determined manually, often based on assumptions or generalized information. This makes it difficult for parents to receive timely and accurate food suggestions that are tailored to their child's actual health status (e.g., age, weight, BMI).

- iv. Existing health monitoring tools lack engagement and are not child-friendly in design or function
 - The current manual system does not involve the child in any interactive or engaging way. There are no visuals, progress tracking, or feedback mechanisms that can help a child understand their own health, making the system less effective for long-term use.
- v. Limited accessibility of tools that combine IoT (smart devices) and AI to provide actionable health data
 - The existing system relies entirely on manual measurement tools and calculations. There is no use of smart scales, automated BMI calculations, or AI-based food suggestion engines that can simplify and enhance decision-making.
- vi. Inconvenience in manually tracking weight, BMI, and food intake over time
 - Parents have to write down and retrieve data from multiple books, which is time-consuming and error prone. There is no structured history or report that allows them to track progress, identify patterns, or make data-driven decisions about their child's health.

3.3 Requirement Analysis

3.3.1 Data Requirement

The NutriGuardian system is designed to function as a smart health monitoring solution that leverages both IoT and AI technologies to provide personalized and responsive nutritional feedback for children. To operate effectively, the system

depends on a variety of data inputs, processes that data to produce meaningful outputs, and stores critical information internally for ongoing analysis and improvement.

The input data in this system originates from two main sources: users and hardware sensors. From the user side, parents or guardians are required to register by entering details such as their full name, email address, contact number, and password. They also register their child by submitting key profile information including the child's name, gender, and age. In some cases, parents or users can also input food consumption manually to track calorie intake. On the hardware side, the system is integrated with IoT devices such as a load cell (for measuring weight) and an ultrasonic sensor (for measuring height). These sensors are connected through an ESP8266 microcontroller, which captures real-time physical measurements and transmits the data to the system wirelessly.

The output data generated by NutriGuardian is based on calculations and AI-driven analysis of the input data. Once the system receives the child's weight and height, it calculates the Body Mass Index (BMI) using standard health formulas. The resulting BMI value is then classified into categories such as underweight, normal, overweight, or obese. Alongside this, the system calculates a recommended daily calorie intake that is suitable for the child's body composition, age, and activity level. Using these outputs, NutriGuardian interacts with an AI-based API to generate a list of suggested meals and food items tailored to the child's nutritional needs. These suggestions are displayed in a user-friendly format within the mobile app interface for the parent and child, and also accessible through the web-based admin panel for monitoring and feedback purposes. Additional output includes motivational, or achievement messages based on health progress, which helps engage children in maintaining healthy habits.

Internally, the system stores all essential data within a structured local database built using MySQL. This database holds user login credentials, child profiles, real-time sensor readings, BMI records, calorie suggestions, AI-generated food

recommendations, and manually logged food intake. Each data record is time-stamped and associated with a unique user or child ID for traceability and progress monitoring. This historical data is crucial for enabling long-term health tracking, generating visual reports, supporting predictive analytics, and improving future recommendations. Admin users can access this data to oversee system usage and performance, while parents benefit from personalized and evolving insights based on their child's historical data.

The data model implemented ensures a clear structure and relationship between entities such as parents, children, health records, and AI outputs. This robust approach to handling data allows the NutriGuardian system to deliver accurate, real-time health monitoring while maintaining user-friendly functionality and data security. By efficiently managing inputs, processing relevant outputs, and storing data in a reliable format, NutriGuardian fulfills its goal of empowering parents and children with personalized tools for better health management.

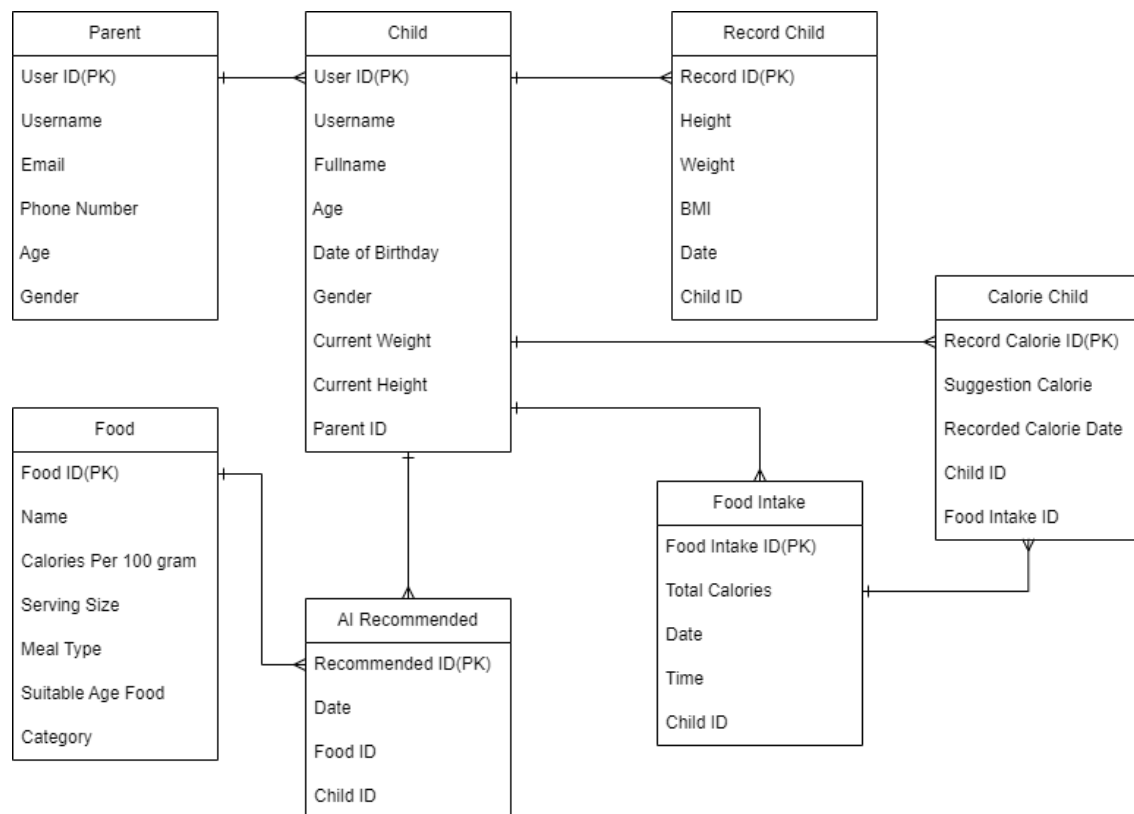


Figure 3.2: Data Modelling

3.3.2 Functional Requirement

The NutriGuardian: Healthcare Monitoring System performs a series of essential functions that involve recording user data, computing health metrics, transforming raw information into meaningful insights, and transmitting outputs back to the users. The system begins by allowing parents to register their account and register their child within the application. It then records the child's biometric data—specifically weight and height—through IoT devices such as smart scales and ultrasonic sensors. This data is collected automatically and stored in the system's local database.

Once the system receives the necessary input, it processes the data by calculating the child's Body Mass Index (BMI) using the standard BMI formula. The computed BMI is then used to estimate the child's daily caloric needs, considering factors like age and activity level. These computations are either handled internally through the application logic or externally via integration with AI services such as Google AI Studio. This stage represents how the system transforms raw data into personalized health recommendations.

The transformation process results in outputs such as categorized health status (e.g., underweight, normal, overweight), suggested calorie intake, and a tailored meal plan that fits the child's nutritional needs. These results are transmitted to both the parent and the child through their respective interfaces. Parents receive the BMI results and suggested meal plans to help them manage their child's dietary intake more effectively, while children receive simplified feedback, meal suggestions, and achievement indicators to encourage healthy behavior. Through this complete cycle of recording, computing, transforming, and transmitting, NutriGuardian provides a smart, personalized, and engaging health monitoring experience.

3.3.2.1 Context Diagram for NutriGuardian Application

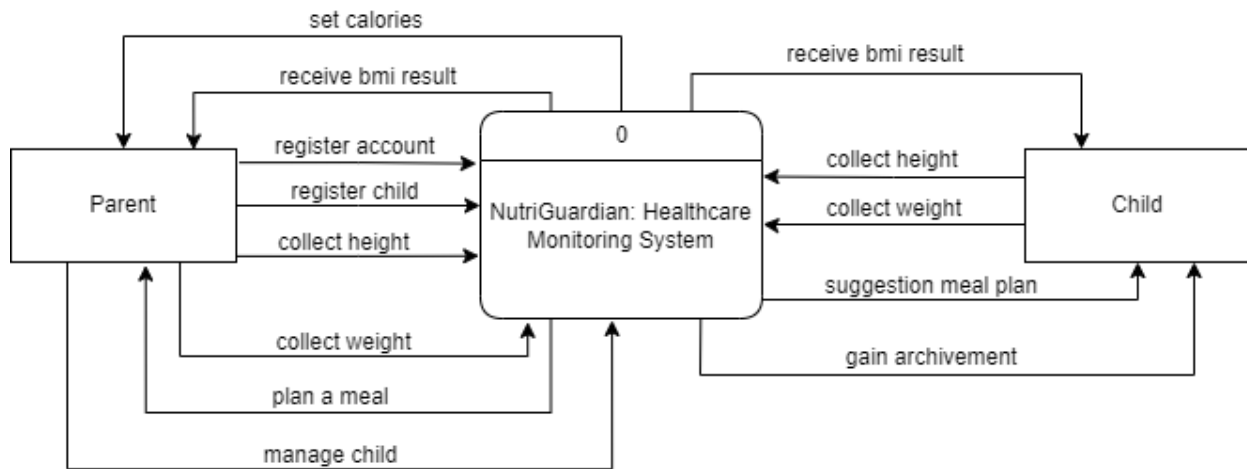


Figure 3.3 Context Diagram

3.3.2.2 Data Flow Diagram for NutriGuardian Application

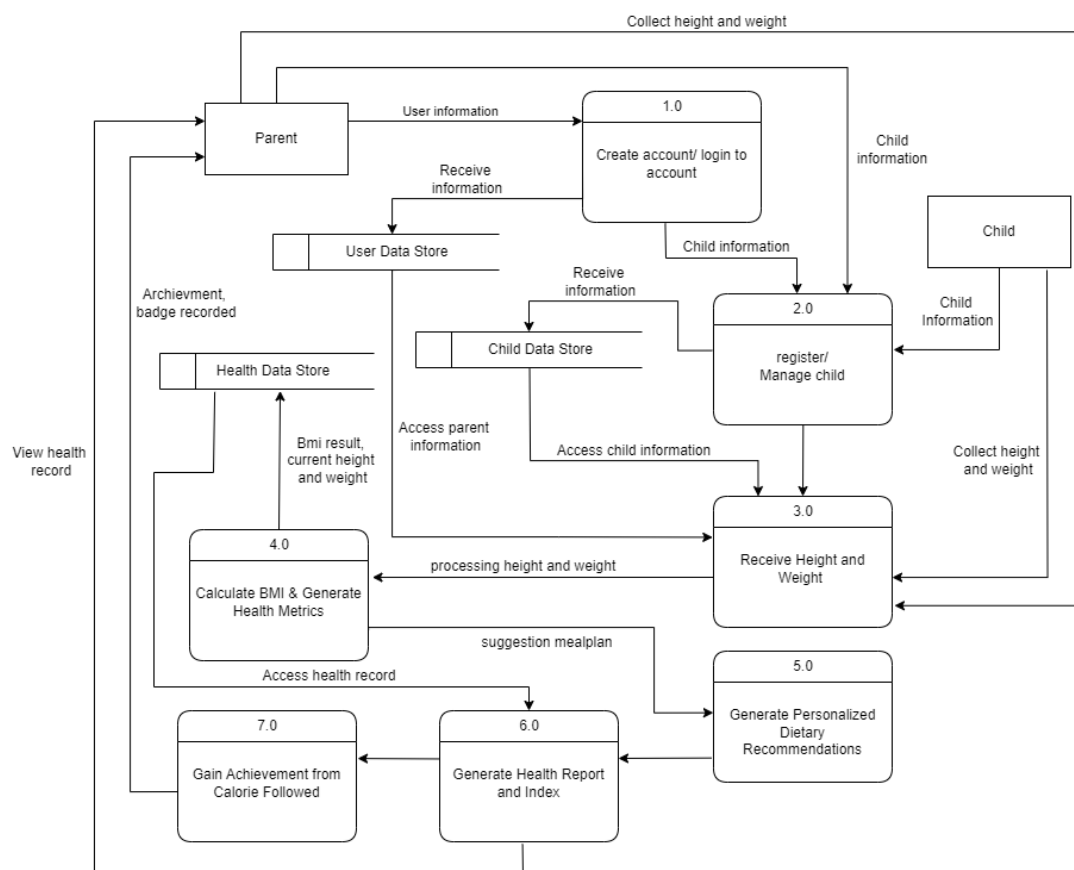


Figure 3.4: DFD Level 0

3.3.2.3 Data Flow Diagram – Level 1 for NutriGuardian

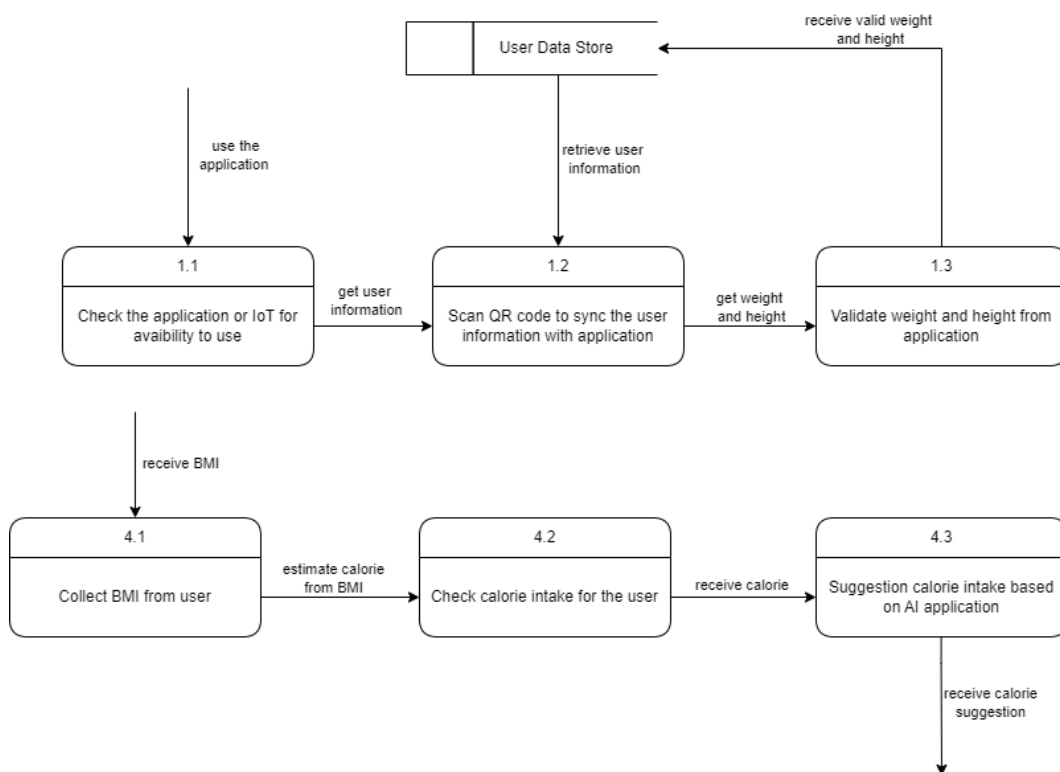


Figure 3.5: DFD Level 1

3.3.3 Non-functional Requirement:

The NutriGuardian system is designed to perform its intended functions with a high level of reliability, responsiveness, and accuracy to ensure it meets the needs of both parents and children. In terms of quality requirements, the system must provide a user-friendly interface that is intuitive for parents and engaging for children. The mobile application is optimized to run smoothly on Android devices with at least 2GB RAM and a quad-core processor, ensuring that the app performs efficiently without consuming excessive system resources. For data processing, the system can calculate BMI values with up to 99% accuracy, as it is based on proven mathematical formulas and real-time input data. Calorie and meal plan recommendations are generated using AI integration, which ensures contextual and personalized suggestions based on weight, height, and age.

In terms of performance, the system should be able to handle simultaneous input and retrieval operations without delays, with response times for displaying BMI results or food suggestions being under 2 seconds in most cases. The local database, hosted via XAMPP and MySQL, is lightweight and capable of storing and retrieving hundreds of user profiles, including their weight/height history, BMI logs, calorie records, and meal recommendations. It is estimated that the database can efficiently manage at least 500–1000 records, depending on the device's storage capacity and indexing strategy. The system also ensures data consistency and minimizes errors during transmission by validating inputs before processing. Overall, NutriGuardian is built to deliver fast, accurate, and consistent health monitoring functionality while maintaining low system overhead and a strong user experience.

3.3.4 Other Requirement

i. Software Requirements

1. Android Studio

- Used to build and test the mobile app for Android. It supports real-time features, is developer-friendly, and integrates easily with APIs and sensor data.

2. Arduino IDE

- Required to program the ESP8266 microcontroller for handling input from the load cell and ultrasonic sensor. It's lightweight and widely supported for IoT projects.

3. XAMPP (with Apache & MySQL)

- Acts as the local server and database environment. Suitable for offline development and testing. Offers easy configuration for PHP and MySQL.

4. Native PHP

- Used to develop APIs for communication between the mobile app and local database. It is simple, fast, and well-suited for small-scale backend projects.

5. Figma

- Used for UI/UX design to ensure the app is easy to use and visually engaging for both children and parents. It supports real-time collaboration and prototyping.

ii. Hardware Requirements

1. Laptop/PC (Min. Intel i5, 8GB RAM)

- Used for system development and local server hosting. Ensures smooth multitasking and efficient compiling of mobile and backend code.

2. Smartphone (e.g., OPPO F11)

- Required to run and test the NutriGuardian mobile app in a real environment. Must support Wi-Fi for IoT data communication.

3. ESP8266 NodeMCU

- IoT microcontrollers are used to send height and weight data wirelessly to the local server. Chosen for its cost-effectiveness and Wi-Fi capability.

4. Load Cell Sensor

- Measures weight accurately and transmit data through the ESP8266.

5. Ultrasonic Sensor

- Measures height without manual input. Reliable for non-contact measurements.

iii. Other Requirements

1. Discussion Room

- Needed for weekly meetings with supervisor and team to discuss progress and receive feedback.

2. Library

- Provides a quiet space for research, development, coding, and access to academic references.

To ensure the NutriGuardian system is developed efficiently and performs its intended functions, a combination of software, hardware, and facility requirements has been carefully selected. Android Studio is used for mobile app development due to its full-featured environment that supports debugging, UI building, and real-time app testing on Android devices. The Arduino IDE is essential for programming the ESP8266 microcontroller, which is responsible for collecting sensor data and transmitting it to the mobile app. XAMPP, combined with MySQL, is used as a local server to host the backend and database. It offers a lightweight, offline-friendly solution suitable for a student-level development environment. Native PHP is used for backend scripting and building RESTful APIs, due to its simplicity and compatibility with XAMPP.

On the side of the design, Figma plays an important role in designing the mobile interface. It allows the team to create wireframes, prototypes, and UI elements that cater to both parents and children in a visually appealing and user-friendly format. In terms of hardware, a laptop or PC with at least an Intel i5 processor and 8GB RAM is required to support development tools, server hosting, and testing simultaneously. A smartphone such as the OPPO F11 is used to test the actual mobile app and its connectivity with the IoT devices. The core IoT hardware includes the ESP8266 NodeMCU, which communicates with a load cell sensor (for weight) and an ultrasonic sensor (for height). These components allow the system to automatically capture health data and send it to the application for processing.

Finally, the project benefits from access to a discussion room, where regular meetings with the supervisor and group members take place, and the library, which serves as a productive environment for research, writing, and implementation. Each component selected contributes to ensuring that the system functions as intended, is cost-effective, and aligns with the educational goals of the project.

3.4 Conclusion

In summary, this chapter has analyzed the current challenges in child health monitoring and explained how the NutriGuardian system addresses these issues through a structured, technology-driven solution. The existing manual processes were reviewed using a Data Flow Diagram (DFD Level 0), revealing inefficiencies such as the lack of automation, personalized dietary guidance, and real-time data tracking. Based on this analysis, the core problems identified in Chapter 1 were reflected and supported through the current system flow.

The system's functions have been thoroughly described, highlighting how it records data from IoT devices, computes health metrics like BMI, transforms raw input into personalized nutrition recommendations, and transmits the results to both children and parents. Additionally, this chapter justified the use of each software, hardware, and facility component, demonstrating their relevance in ensuring the system performs its intended tasks efficiently, accurately, and reliably.

Looking ahead, the next activities involve the system design and development phase, where the architecture, interface layouts, database structures, and actual coding will be implemented based on the analysis completed in this chapter. This includes designing flowcharts, user interface mock-ups, and integrating the IoT devices and mobile application through APIs. The transition from planning to execution will mark the start of building a fully functional prototype of NutriGuardian, aligned with the specifications and user requirements identified throughout the analysis.

CHAPTER 4: DESIGN

4.1 Introduction

This chapter defines the outcomes of the analysis phase and presents the results of both the preliminary and detailed system design for the NutriGuardian application. It outlines how the system's components are structured, how they interact, and how each feature is implemented based on the requirements gathered in the previous chapter. The design phase involves the construction of various diagrams and models that represent the system architecture, data flow, user interaction, and component behavior. This includes the system architecture diagram, use case diagrams, data flow diagrams (DFD), entity relationship diagrams (ERD), and interface designs. These visual representations help illustrate how data moves through the system, how users interact with it, and how each module connects to achieve the system's objectives.

This chapter also discusses the logical and physical design of the system, such as database structure, module breakdown, and input/output design. The goal is to ensure the system is planned in a way that is scalable, maintainable, and functional before moving on to the implementation phase. Through this chapter, the blueprint of the NutriGuardian system is fully developed to guide its successful construction.

4.2 High-Level Design

The NutriGuardian system is structured using a modular and layered architecture, designed to ensure scalability, maintainability, and clear separation of responsibilities across components. At a high level, the system consists of three core layers: the IoT layer, the application layer, and the data layer. The IoT layer is responsible for collecting biometric input data such as weight and height using sensors connected to the ESP8266 microcontroller. This data is transmitted wirelessly to the server through HTTP requests.

The application layer includes both the mobile application (Android) and the web-based admin panel. The mobile app serves as the main interface for children and parents, displaying BMI results, health status, and AI-generated food recommendations. The admin panel, built using native PHP, allows authorized users to view records and manage user data. The data layer, running on a local MySQL database, stores user profiles, measurement history, BMI calculations, and meal suggestion logs. The system also integrates with Google AI Studio API to provide smart dietary suggestions based on BMI and other user attributes. Each component communicates through custom APIs written in PHP, hosted on a XAMPP local server. This layered and modular structure ensures that different parts of the system can be developed, tested, and refined independently, especially during PSM II—while maintaining overall coherence and functionality.

4.2.1 System Architecture

The NutriGuardian system is designed based on a modular, layered architecture that integrates IoT technology, native web APIs, and AI logic to deliver a smart and responsive healthcare monitoring solution. The architecture follows a three-tier model that separates the system into the presentation layer (mobile app and admin interface), application layer (business logic and API), and the data layer (local database using MySQL via XAMPP). This separation of concerns allows for better scalability, flexibility, and maintainability of the system. Data is collected in real time through embedded devices such as ESP8266 microcontrollers connected to load cell and ultrasonic sensors, processed via native PHP APIs, and results are displayed and interacted with through the mobile GUI and web dashboard. Furthermore, the architecture includes AI components such as Fuzzy Logic and Kalman Filters to refine decision-making and reduce data noise.

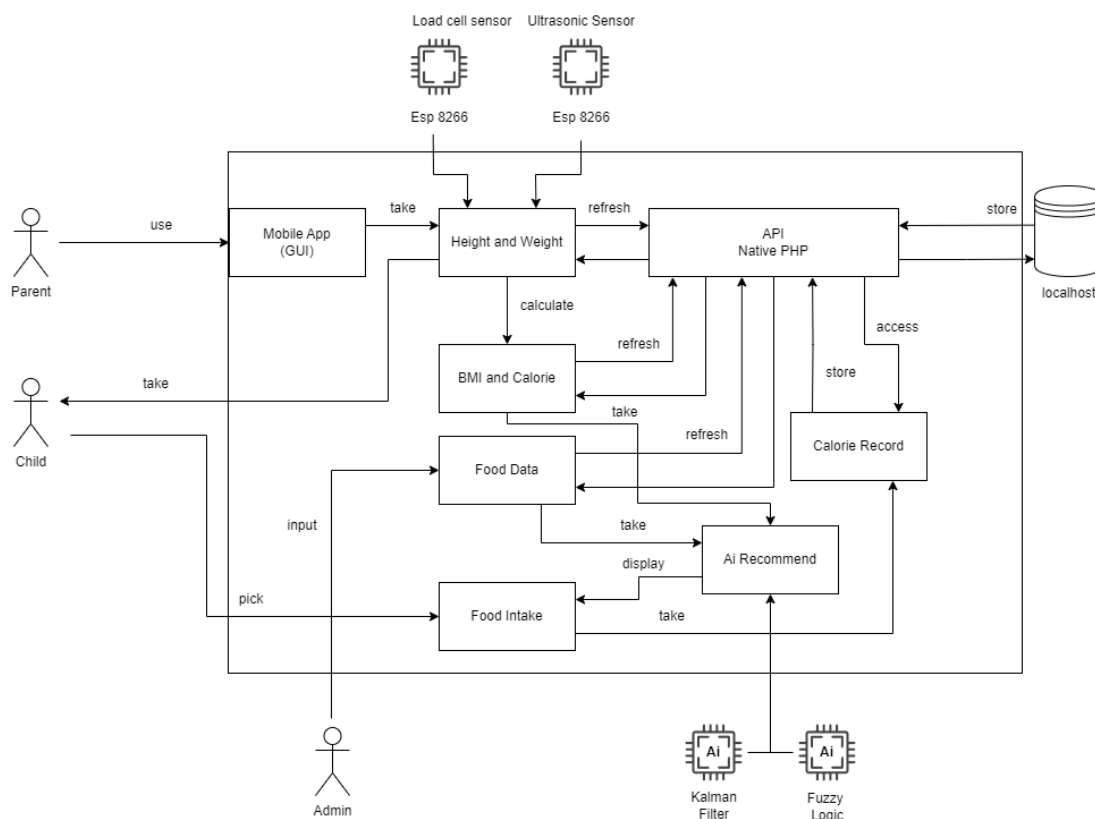


Figure 4.1: System Architecture

Figure 4.2.1 illustrates the complete flow of the NutriGuardian system, beginning with the data collection stage through two ESP8266 devices connected to a load cell sensor (for weight) and ultrasonic sensor (for height). These sensors send real-time biometric data to the “Height and Weight” module, which is accessible through the mobile application interface used by the parent and viewed by the child. Once collected, the system calculates the BMI and calorie recommendation through internal logic and passes this data to the API (Native PHP) module, which handles both data processing and communication with the backend. This API also manages the Calorie Record and ensures data is stored securely in the local MySQL database (localhost).

The Food Data and Food Intake modules are used to track and manage nutritional intake manually or automatically. The AI Recommend component plays a key role by leveraging stored Calorie that required and food history to suggest personalized meal plans using AI techniques such as Kalman Filtering for sensor smoothing and Fuzzy Logic for recommendation decisions. This AI-recommended food list is then made available for review and selection by users via the mobile app. The admin also has control and oversight over all stored data, ensuring centralized monitoring. Each connection in the diagram illustrates the interaction flow between modules, highlighting the comprehensive integration of sensors, AI, and databases to support real-time health tracking and personalized nutrition.

4.2.2 User Interface Design

The user interface (UI) design of the NutriGuardian system is thoughtfully developed to be visually appealing, user-friendly, and thematically consistent with the system's health-oriented purpose. It utilizes a calming color palette of white and light blue, which represents the water element—signifying life, nourishment, and balance—reflecting the core values of the system in promoting children's health and well-being. This dual-tone color scheme provides a modern, clean aesthetic that enhances the user experience while maintaining a professional and trustworthy appearance. To ensure clarity and readability, all text is presented in black, creating a strong contrast against the blue background and improving accessibility across different devices and lighting conditions.

The layout is structured for ease of use, with intuitive navigation, consistent alignment of buttons and forms, and logical flow between sections such as data input, health monitoring, and food recommendations. Overall, the interface not only supports functional usability but also creates an engaging and comfortable environment for both parents and children to interact with the system effectively.



Figure 4.2: Loading page

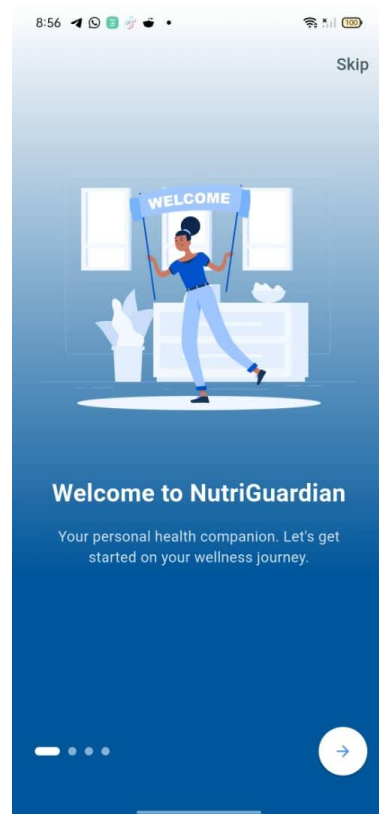


Figure 4.3: Leading page

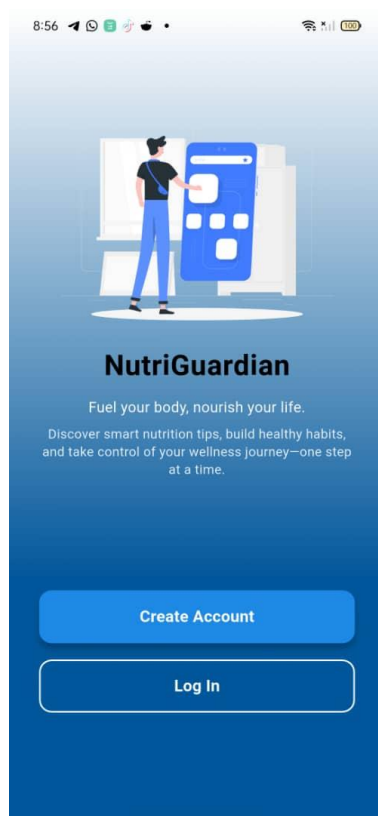


Figure 4.5: Choose page

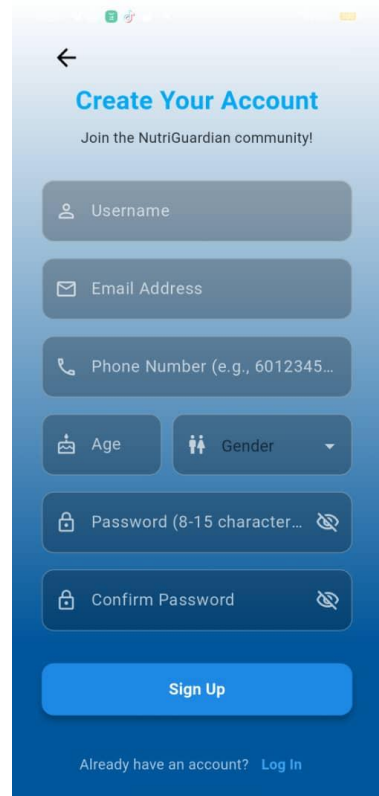


Figure 4.4: Register page

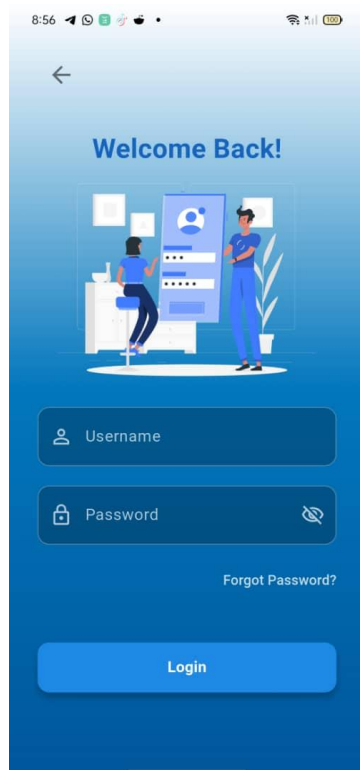


Figure 4.7: Login page



Figure 4.6: Homepage



Figure 4.9: Daily Calorie Page



Figure 4.8: Graph Daily Calorie Page



Figure 4.11: Meal Plan

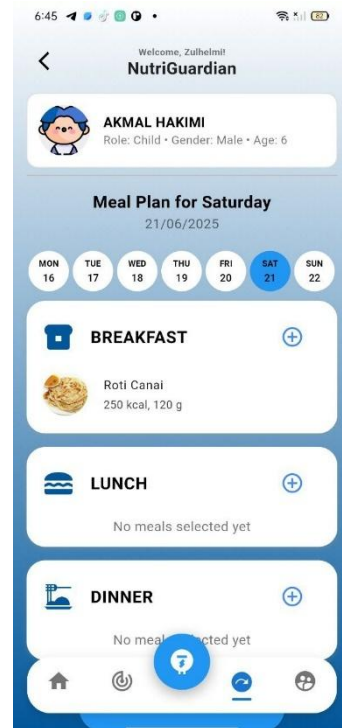


Figure 4.10: Child Meal Plan

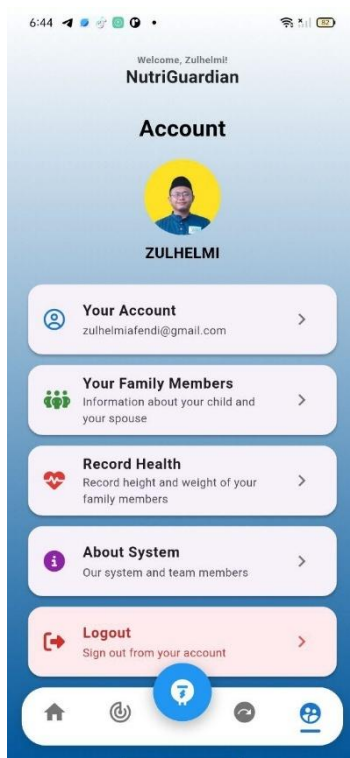


Figure 4.13: Account Page

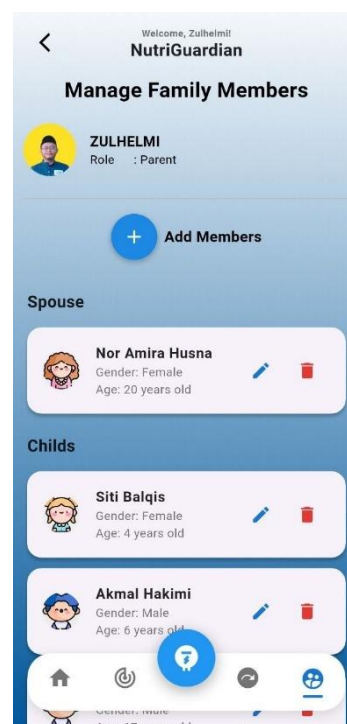


Figure 4.12: Child page

Welcome, Zulhelmi!

NutriGuardian

Manage Family Members

ZULHELMI
Role: Parent

Add New Family Member

Nickname

Full Name

Role
Select Family Role

Gender
Select Gender

Birthdate (YYYY-MM-DD)

Add Member

Figure 4.15: Add family members form

Welcome, Zulhelmi!

NutriGuardian

Profile

ZULHELMI

Change Picture

Email
zulhelmi.fendi@gmail.com

Phone
197295642

Age
22

Gender
Male

Figure 4.14: Profile page

Welcome, Zulhelmi!

NutriGuardian

Choose a Family Member

Swipe to select a member and view their health records.

Add or update family members in the Account section.

Akmal Hakimi
Child • Age: 6

View Records

Figure 4.16: Record page

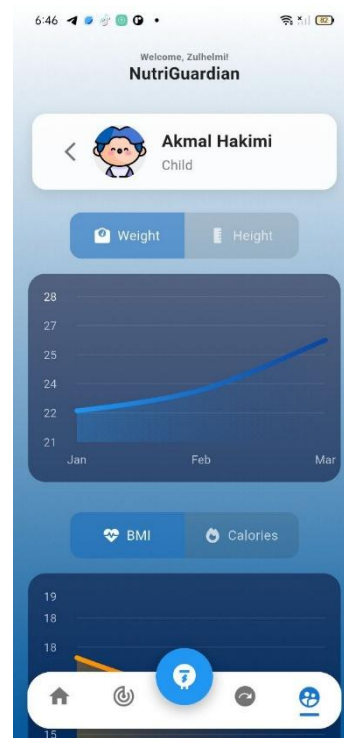


Figure 4.17: Graph record page

4.2.2.1 Navigation Design

The navigation flow of the NutriGuardian system is designed to be intuitive and user-centered, guiding users smoothly through each stage of the application from registration to monitoring and food recommendation. The main interface follows a linear and hierarchical flow, where users first land on a login or registration page and are then directed to a dashboard that presents key modules such as “Manage Family Member,” “Measure Height & Weight,” “BMI & Calorie recommended,” “AI Meal Suggestion,” and “History log.” Each module is interconnected and accessible through a bottom bar depending on the platform, ensuring quick transitions between screens without confusion.

The system implements several common navigation controls including buttons for actions like “Submit,” “Save,” and “Tare Scale,” tab bars for switching between features, sliding menus for grouped categories, and back arrows for returning to the previous screen. Icons are used alongside labels to enhance clarity, especially for child users or non-technical parents. In both the mobile and web versions, consistent placement of navigation elements and visual feedback like highlights or color changes upon selection help users understand their position within the system. This thoughtful design of navigation controls not only improves usability but also ensures that users can interact with health data in a simple, efficient, and structured way.

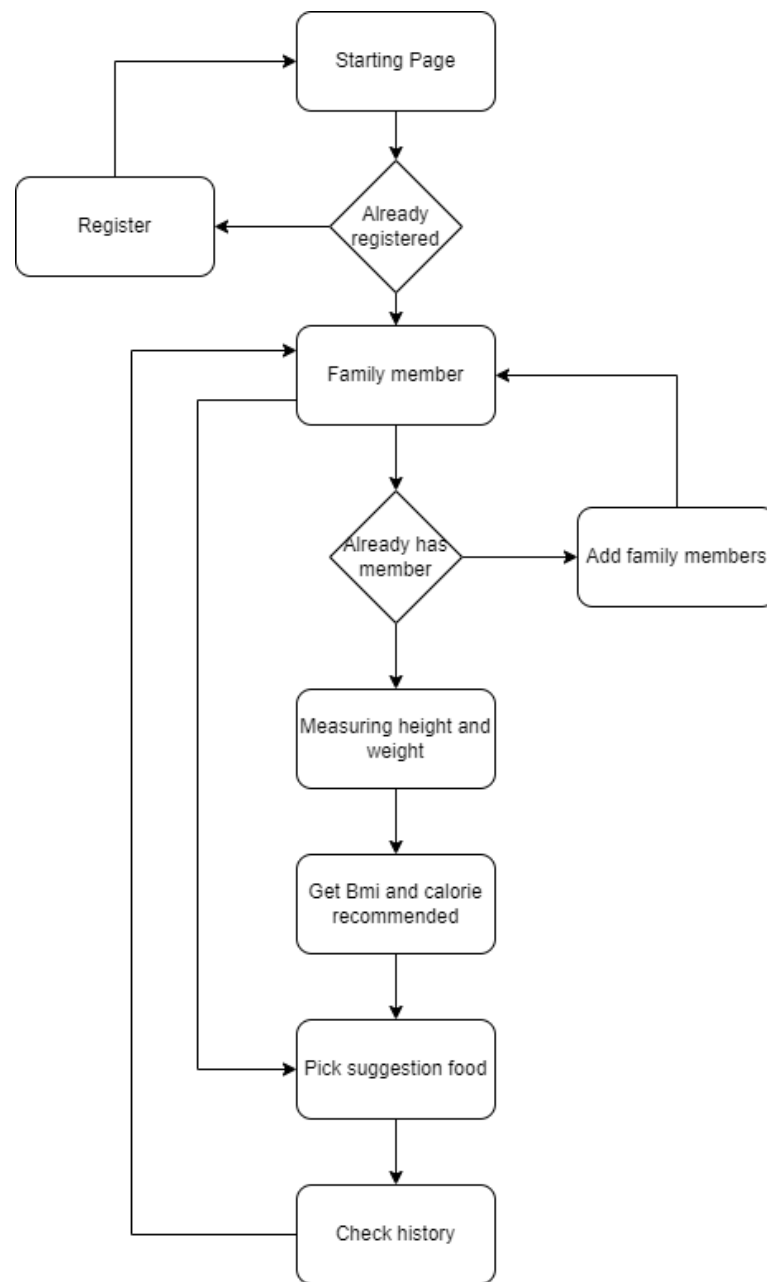


Figure 4.18: Navigation Flow

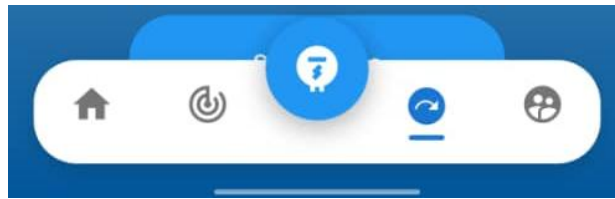


Figure 4.19: Bottom Bar

Figure 4.2.2 explains the bottom navigation bar. The navigation design used in the custom bottom bar of this application follows a tab-based navigation structure, which is commonly used in mobile applications to allow quick access to major sections of the app. The bottom bar consists of five main navigation points, each represented by an icon and structured logically to reflect the app's core features: Home, Track Calorie, Meal Plan, Account, and a central Floating Action Button (FAB) for IoT functionality. This navigation design uses visual cues such as icon color changes and an indicator line to guide users in identifying the currently active tab. The active icon turns blue, while inactive icons appear in grey. A small blue bar appears below the active icon to enhance clarity and feedback. The navigation between pages is handled using `Navigator.pushReplacement`, which replaces the current screen with the selected one, ensuring smooth and efficient page transitions without stacking multiple screens.

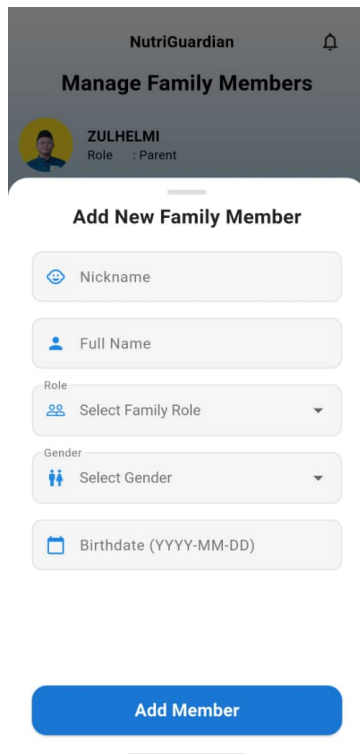
The central FAB is positioned slightly above the bar and acts as a shortcut to the IoT page. It is styled to stand out with a circular shape, shadow, and blue color, drawing user attention to its importance. When active, it changes to a darker blue and includes a white border to visually distinguish it from other tabs. Overall, this navigation design emphasizes clarity, ease of use, and aesthetic consistency. It helps users quickly identify where they are in the app and move efficiently between sections, following common mobile UX design principles for user-friendly navigation.



Figure 4.20: Back Button

This icon represents a "back" navigation pattern, which follows standard user interface guidelines, especially in mobile UX. It provides a clear and intuitive way for users to move backward in the app's navigation stack, typically managed using `Navigator.pop(context)` in Flutter. The simplicity of the icon ensures that it is universally understood by users without the need for additional labels. This navigation design supports a hierarchical structure, where users can move deeper into subpages and then return to the previous context. It contributes to a consistent and predictable user experience, helping users feel in control while navigating through different sections of the app.

4.2.2.2 Input Design



NutriGuardian

Manage Family Members

ZULHELMY
Role : Parent

Add New Family Member

Nickname

Full Name

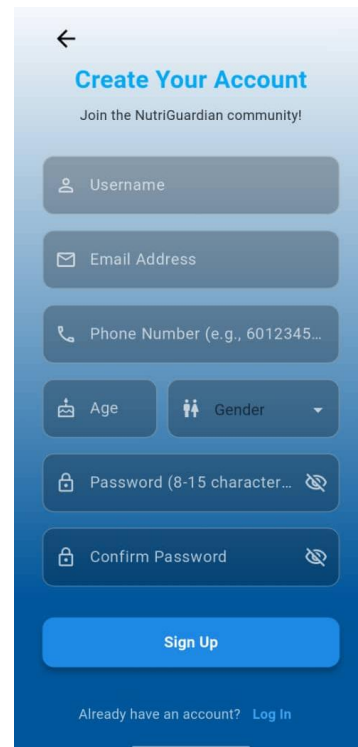
Role
Select Family Role

Gender
Select Gender

Birthdate (YYYY-MM-DD)

Add Member

Figure 4.22: Register Account



Create Your Account

Join the NutriGuardian community!

Username

Email Address

Phone Number (e.g., 6012345...)

Age Gender

Password (8-15 character...)

Confirm Password

Sign Up

Already have an account? [Log In](#)

Figure 4.21: Register Account

In figure 4.2.3 and figure 4.2.4 explain about add member forms that are designed with a structured layout and intuitive components to ensure ease of use, especially for parents managing family data. One such form that I choose to be analysis in reporting is the “Add New Family Member” screen, which collects key information such as nickname, full name, family role, gender, and birthdate. Each input is matched with the appropriate form component to reduce input error and enhance clarity. Text input fields are used for both the "Nickname" and "Full Name" sections, allowing users to type freely using alphabetically or alphanumeric characters. The "Family Role" and "Gender" fields use dropdown selection boxes that restrict choices to predefined options such as “Child” or “Parent,” and “Male” or “Female,” which improves consistency across records. A date picker input is applied for the "Birthdate" field to enforce correct formatting and prevent user error. Each of these input fields is governed by clear and necessary validation rules.

The nickname and full name fields are required, must meet minimum character lengths, and disallow special characters or numbers (for full name). The family role and gender fields are mandatory and must be selected from the dropdown lists, preventing submission if left unchosen. The birthdate must be entered in the format "YYYY-MM-DD" and validated to ensure that it is not a future date. These validation rules are implemented to ensure data accuracy, completeness, and system integrity. Furthermore, the form uses visual cues like placeholder text, icons, and responsive error messages to guide the user during data entry. Only when all inputs are valid does the system enable the “Add Member” button, ensuring that only complete and correctly formatted data is submitted into the system.

4.2.2.3 Output Design



Figure 4.24: Height and Calorie Graph



Figure 4.23: Weight and BMI graph

In figure 4.24 and figure 4.25 is showing the graph of the weight, height, BMI and calories of a child that has pick. This application primarily generates outputs in the form of graphs, which serve as the main visual representation of user data over time. These graphs include periodic tracking of Weight, displaying trends from January to March, and are updated periodically, likely daily or weekly with monthly viewing. Similarly, Height is presented in a periodic graph from January to March, reflecting less frequent but still periodic updates (monthly/quarterly). The app also provides a BMI graph, showing trends from January to March, which is periodically calculated daily or weekly and viewed monthly.

Lastly, Calories are tracked over a longer period, from January to August, with daily input reflected in a periodic graph viewed monthly, bi-monthly, or yearly. While not explicitly shown as separate documents, the app implicitly provides summary reports through the current values displayed alongside the graphs and the welcome message upon opening the application, making them available on an ad-hoc basis. Though not directly visible in the provided images, it is highly probable that the application offers detail reports, such as daily logs of weight or calorie intake, which would be generated periodically and accessible on demand for users to review specific inputs. Turnaround documents, a concept more common in traditional business processes involving external data input, are not applicable to the interactive and direct data entry nature of this personal health tracking application.

4.2.3 Database Design

The database design for the NutriGuardian system was developed to ensure efficient, structured, and scalable data management that supports the system's core functions, including user management, health tracking, and AI-based meal recommendations. The design process began with the creation of an Entity Relationship Diagram (ERD), which provides a high-level view of the database structure and illustrates the relationships between key entities such as parent, child, record child, Food, AI recommended, and calorie record. Each entity is defined with its primary key and relevant attributes, and foreign key constraints are used to establish referential integrity—for instance, each child is linked to a parent, and health records are linked to both the parent and child entities.

The ERD supports normalization principles, ensuring that data is stored without redundancy and can be efficiently queried. In addition to the ERD, a Data Flow Diagram (DFD) was constructed to illustrate how data flows through the system at different levels. The DFD Level 0 shows the interaction between external entities (such as parents, children, and sensors), system processes (such as BMI calculation and AI food recommendation), and data stores (such as the local MySQL database). At a more detailed level, the DFD Level 1 breaks down specific processes like weight and height input, food intake logging, and calorie suggestion handling. Other components such as the logical data model, data dictionary, and architecture diagram were also prepared to support implementation and documentation.

These diagrams collectively ensure that the database structure not only aligns with the functional requirements of the system but also supports performance, scalability, and maintainability. The system uses a local database hosted via XAMPP, with PHP-based APIs facilitating communication between the mobile application and backend server, allowing for real-time data entry, retrieval, and recommendation generation.

4.2.3.1 Conceptual and Logical Database Design

The Logical Data Model (LDM), commonly represented through an Entity Relationship Diagram (ERD), plays a crucial role in the design and structure of the NutriGuardian system's database. It provides a high-level visual representation of the data entities, their attributes, and the relationships between them. By mapping out the connections between elements such as children, parents, food intake, health records, and AI recommendations, the ERD ensures that the database is logically organized and supports the system's intended functionality.

This model helps developers understand how data flows within the application, how tables should be normalized, and how relationships should be defined using primary and foreign keys. Ultimately, the LDM/ERD serves as the foundation for building a reliable, scalable, and efficient database that meets the system's data storage and processing needs.

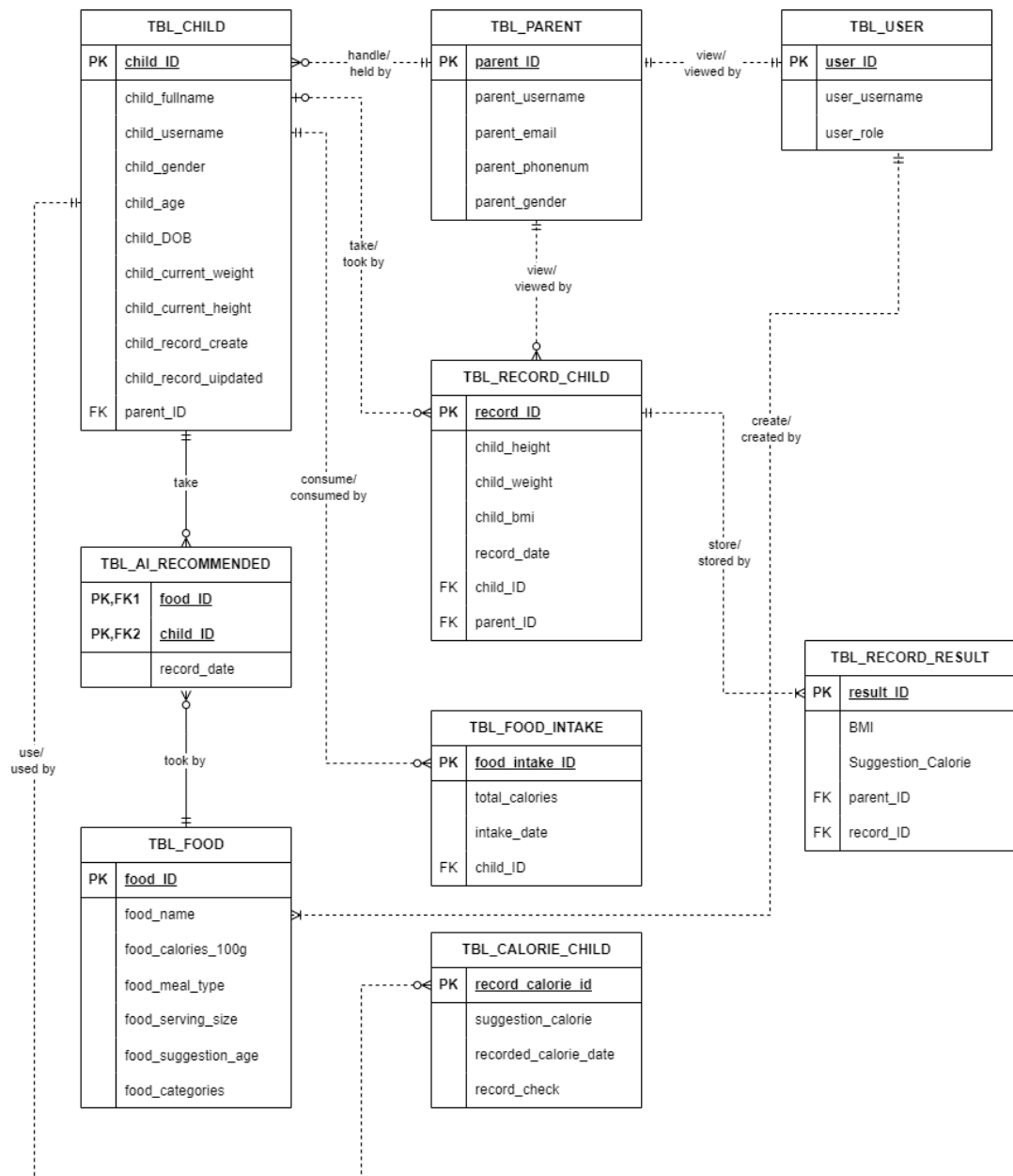


Figure 4.25: Entity Relationship Diagram

Figure 3.3.4 explain ERD of NutriGuardian system which show the structure and relationships between various entities involved in managing child health data, including users, parents, children, food intake, and AI-generated recommendations. At the core of the system is the TBL_CHILD table, which stores essential details of each registered child user such as name, age, gender, weight, height, and parent user association. This table is linked via a foreign key (parent_ID) to the TBL_PARENT entity, which contains parent user information including username, email, phone number, and gender.

Each child user has multiple health records stored in TBL_RECORD_CHILD, which logs historical data such as weight, height, BMI, and the date of record. These records are also associated with both child and parent entities for relational tracking. The results of BMI analysis and calorie recommendations are then stored in TBL_RECORD_RESULT, which connects with the corresponding record_ID and parent_ID, allowing parents to view and track health outcomes. The system also integrates AI-based food recommendations through the TBL_AI_RECOMMENDED table, which links food items (TBL_FOOD) to specific children based on their profile and record date. Each food item includes nutritional information such as calories per 100g, meal type, serving size, suitable age, and category. This allows for personalized dietary suggestions based on calculated BMI.

Lastly, to monitor the child's actual eating behavior, the TBL_FOOD_INTAKE table records calorie intake by linking food intake data to each child along with the intake date and total calories consumed. This is complemented by the TBL_CALORIE_CHILD table, which logs the system's calorie suggestions, the calories recorded, and whether they match (record check). The TBL_USER table appears to define general system users or administrators with associated roles and credentials, likely for login control and user access management. Overall, this ERD defines a normalized and relational structure that enables the NutriGuardian system to

effectively track, process, and analyze health and dietary information in a structured and scalable way.

4.2.3.2 Data Dictionary

Table 4.1: Database Table Parent

Field Name	Data Type	Data Format	Field Size	Description	Example
User_ID	Integer		11	Unique number ID for all parent or user	1234
Username	Varchar		30	Username for the user to login	Ahmad
Email	Varchar	user@mail.com	40	Email of the user to access the system	ahmad@gmail.com
PhoneNum	Integer		11	Phone number for user	01122122121
Age	Integer		11	Age of user	23
Gender	Varchar		10	Gender of user	Male
Password	Varchar		999	Password to access and login the system	32143245
Icon	Varchar		244	Image of user to use in the system	Image.jpg
Token	Varchar		20	Tokens are code that used to verify the user account	Esd34twf

Table 4.2: Database Table Child

Field Name	Data Type	Data Format	Field Size	Description	Example
Child_id	Integer		11	Unique number ID for child user like family member	1231
Child_fullname	Varchar		40	Full name of child user	Zulhe lmi
Child_username	Varchar		20	Username of child user to use in the system	Zul
Child_gender	Integer		11	Gender of the child user	Male
Child_age	Integer		11	Age of the child user	12
Child_dateofbirth	date	YYYY-MM-DD		Date of birth of child user	2012-09-03
Child_role	Varchar		10	Role of child user like child or spouse or parent	spouse
Child_current_weight	Double			Current weight of user and will change after measuring new	45
Child_current_height	Double			Current height of user and will change after measuring new	167
Child_created_at	Date	YYYY-MM-DD		Date the child user create	2010-10-01
Child_updated_at	Date	YYYY-MM-DD		Date the child user updated	2010-10-01
Child_checker	enumerable	'Yes','No'		Checkers are used to check if the child user already used the IoT or not	Yes
Parent_id	Integer		11	Unique number ID for Parent for each child user	234

Table 4.3: Database Table Record

Field Name	Data Type	Data Format	Field Size	Description	Example
Record_id	Integer		11	Unique number ID for record	123
Child_id	Integer		11	Unique number ID for child for each record	123
Child_height	Double			Weight of child user use IoT	45.3
Child_weight	Double			Height of child user use IoT	160.8
Child BMI	Double			Bmi of child user	23.3
Record_date	Datetime	YYYY-MM-DD HH:MM:SS		Date of record for the data inserted	2023-12-11 11:34:34
Parent_id	Integer		11	Unique number of child for each record	123

Table 4.4: Database Table Calorie Record

Field Name	Data Type	Data Format	Field Size	Description	Example
Record_calorie_id	Integer		11	Unique number of record calorie	123
Child_id	Integer		11	Unique number of child user	123
Suggestion_calorie	Integer		11	Suggestion calorie of each child user	2300
Recorder_calorie_at	Datetime	YYYY-MM-DD HH:MM:SS		Record date of calorie taken	2023-12-11 11:34:34
Record_check	Enumbrable	'Yes', 'No'			

Table 4.5: Database Table Recommended

Field Name	Data Type	Data Format	Field Size	Description	Example
Recommended_id	Integer		11	Unique number of recommended foods	123
Date	date	YYYY-MM-DD		Date the record create	2023-12-11
Food_id	Integer		11	Unique number Id for food	123
Child_id	Integer		11	Unique number Id for child user	123

Table 4.6: Database Table Food

Field Name	Data Type	Data Format	Field Size	Description	Example
Food_id	Integer		11	Unique number Id for food	123
Food_name	Varchar		40	Name of food	Nasi Lemak
Food_image	longtext	Name-image.jpg		Image of food	Nasi-lemak.jpg
Food_calories_100g	Integer		11	Calories of food per 100g	230
Serving_size	Integer		11	Size of serving food	250
Meal_type	Enumerable	'breakfast', 'lunch', 'dinner', 'snack'		Type of meal	Breakfast
Suitable_age_group	Varchar		30	Suitable age of food	9-12
Food_category	Enumerable	'Carb', 'Protein', 'Vegetable', 'Side'		Category of food	Side

Table 4.7: Database Table Food Intake

Field Name	Data Type	Data Format	Field Size	Description	Example
Food_intake_id	Integer		11	Unique number Id for food intake	123
Child_id	Integer		11	Unique number Id for child user	123
Total_calorie	Integer		11	Total calorie of food intake for child user	2300
date	Date	YYYY-MM-DD		Date of food intake	2023-12-11

Table 4.8: Database Table User

Field Name	Data Type	Data Format	Field Size	Description	Example
User_id	Integer		11	Unique number id for user how manage system	123
User_name	Varchar		50	Username of the user	Admin
User role	Varchar		11	Role of user	admin
User_password	Varchar		99	Password of user to access the login form	Abc12312

4.3 Detailed Design

The design of the NutriGuardian system is built upon a logical, layered architecture that aligns with the system's objectives of monitoring children's health and generating personalized food recommendations. The specification of the system is supported through detailed ERD (Entity Relationship Diagram), DFD (Data Flow Diagram), and system architecture diagrams, each of which demonstrates how data and functions interact to fulfill user needs. The ERD outlines the structural relationships between essential entities such as users (parents and children), food records, AI-generated recommendations, and health measurement records. This structure ensures that data is normalized and traceable, enabling secure and accurate storage of health history.

The DFD Level 0 and Level 1 diagrams highlight how inputs like weight and height (collected via IoT devices) flow through various processes including BMI calculation, calorie estimation, and AI-based food recommendation. These processes are logically sequenced to meet functional requirements, starting with data input, followed by processing, and then feedback to the user. For instance, a parent uses the mobile app to register their child and initiate health measurements. The system collects real-time sensor data via ESP8266 microcontrollers, sends it through a PHP-based API, and stores it in a local MySQL database using XAMPP. The backend calculates BMI and daily calorie needs, and integrates with an AI model (Fuzzy Logic and Kalman Filter) to recommend appropriate meals. This output is then displayed on the app for parents and children to follow. The logical design ensures that each function, from sensor input to AI output, works cohesively and is scalable for future enhancements. By combining clearly defined database structures, real-time processing flows, and modular architecture, the system effectively meets its goals of providing actionable, personalized nutrition guidance based on continuous health monitoring.

4.3.1 Software Design

4.3.1.1 BMI Calculation Module:

This module is responsible for calculating the Body Mass Index (BMI) of a child based on real-time sensor input data—weight from a load cell and height from an ultrasonic sensor. Once calculated, the BMI value is used to determine the child's health status (underweight, normal, overweight, obese), which will later be referenced for calorie and meal suggestions. The process is initiated after the parent triggers the measurement via the mobile app.

```

    return;
}
double bmi = weightForCalc / (heightInMetersForCalc * heightInMetersForCalc);
_showBmiDialog(context, bmi);
insertBmi(heightInMetersForCalc, weightForCalc, bmi);
if (mounted) {
    setState(() {
        _countdownToBmiStarted = false;
    });
}

```

Figure 4.26 BMI Calculation Code

Input	Sources	Format
Weight_kg	Load Cell via Esp8266 or manual	Double
Height_cm	Ultrasonic Sensor via Esp 8266 or manual	Double
Child_ID	Mobile app form	Integer

Table 4.1 Input of BMI Calculation

Output	Destination	Format
BMI_result	Localhost database	Double
BMI category	App GUI Result	String

Table 4.2 Output of BMI Result

4.3.1.2 Calorie Tracking Calculation Module Using BMI

This module is responsible for calculating the recommended daily calorie intake for a child based on their BMI, age, and gender. The calculated value is essential for monitoring the child's nutritional needs and providing personalized food suggestions through the AI engine. The calorie recommendation is updated each time a new BMI record is generated. The calculation logic applies standard nutritional formulas that relate BMI category to age-based calorie requirements.

```
// Define the getCaloriePerDay function outside the loop
function getCaloriePerDay($age, $gender, $weight, $height) {
    $activity_level = getActivityLevelByFuzzyLogic($age, $weight, $height);

    // Base calorie calculation
    if ($age < 18) {
        if ($gender == 'male') {
            $calories = 88.5 + (13.5 * $weight) + (4.2 * $height) - (5.5 * $age);
        } else {
            $calories = 135.3 + (10.0 * $weight) + (5.0 * $height) - (6.5 * $age);
        }
    } else {
        if ($gender == 'male') {
            $calories = 66.5 + (13.75 * $weight) + (5.003 * $height) - (6.75 * $age);
        } else {
            $calories = 655 + (9.563 * $weight) + (1.850 * $height) - (4.676 * $age);
        }
    }

    // Adjust calorie calculation based on activity level
    switch ($activity_level) {
        case 'low':
            $calories *= 0.9;
            break;
        case 'medium':
            $calories *= 1.0;
            break;
        case 'high':
            $calories *= 1.2;
            break;
    }
}
```

Figure 4.27 Calorie Calculation Code

Input	Sources	Format
Child_ID	Mobile App	Integer
BMI Result	BMI Calculation Module	Double
Child_Age	Child Database	Integer
Child Gender	Child Database	String(Male,Female)

Table 4.3 Input of Calorie Calculation

Output	Destination	Format
Calorie Required	Stored in Calorie Record	Double
Record Calorie Date	Auto-generate	Date

Table 4.4 Output of BMI Calculation

4.3.1.3 AI Recommendation Module

This module retrieves the child's BMI and calorie data and sends it to an AI recommendation engine that uses fuzzy logic and Kalman filtering to return a list of suitable food options. These foods are age-appropriate, calorie-balanced, and aligned with the child's current BMI category.


```
function getActivityLevelByFuzzyLogic($age, $weight, $height) {
    // Define ranges and probability weights for fuzzy logic
    $low_probability = 0;
    $medium_probability = 0;
    $high_probability = 0;
}
```

Figure 4.28 Initialize function

```
// Apply rules with weights
if ($age < 10) {
    $low_probability += 0.7;
    $medium_probability += 0.2;
    $high_probability += 0.1;
} elseif ($age >= 10 && $age < 15) {
    $low_probability += 0.2;
    $medium_probability += 0.6;
    $high_probability += 0.2;
} else {
    $low_probability += 0.1;
    $medium_probability += 0.3;
    $high_probability += 0.6;
}

if ($weight < 40) {
    $low_probability += 0.6;
    $medium_probability += 0.3;
    $high_probability += 0.1;
} elseif ($weight >= 40 && $weight < 60) {
    $low_probability += 0.2;
    $medium_probability += 0.5;
    $high_probability += 0.3;
} else {
    $low_probability += 0.1;
    $medium_probability += 0.2;
    $high_probability += 0.7;
}

if ($height < 140) {
    $low_probability += 0.5;
    $medium_probability += 0.4;
    $high_probability += 0.1;
} elseif ($height >= 140 && $height < 160) {
    $low_probability += 0.2;
    $medium_probability += 0.6;
    $high_probability += 0.2;
} else {
    $low_probability += 0.1;
    $medium_probability += 0.3;
    $high_probability += 0.6;
}
```

Figure 4.29 Apply the function of probability age and weight

```
// Normalize probabilities to make a decision
$total_probability = $low_probability + $medium_probability + $high_probability;
$low_probability /= $total_probability;
$medium_probability /= $total_probability;
$high_probability /= $total_probability;
```

Figure 4.30 Normalize the probability

```
// Choose the activity level with the highest probability
if ($low_probability > $medium_probability && $low_probability > $high_probability) {
    return 'low';
} elseif ($medium_probability > $high_probability) {
    return 'medium';
} else {
    return 'high';
}
```

Figure 4.31 Decision based on probability for priority

```
function fuzzyLogicForMealPlan($bmi, $age) {
    if ($age < 6) {
        // For children 5-6 years old
        if ($bmi < 18.5) {
            return ['Breakfast' => 400, 'Lunch' => 700, 'Dinner' => 600];
        } elseif ($bmi < 24.9) {
            return ['Breakfast' => 350, 'Lunch' => 600, 'Dinner' => 500];
        } else {
            return ['Breakfast' => 300, 'Lunch' => 500, 'Dinner' => 450];
        }
    } else if ($age < 12) {
        // For children 7-12 years old
        if ($bmi < 18.5) {
            return ['Breakfast' => 450, 'Lunch' => 800, 'Dinner' => 700];
        } elseif ($bmi < 24.9) {
            return ['Breakfast' => 400, 'Lunch' => 650, 'Dinner' => 550];
        } else {
            return ['Breakfast' => 350, 'Lunch' => 600, 'Dinner' => 500];
        }
    } else {
        // If age > 12, use a different logic, e.g., adult meal planning
        return ['Breakfast' => 300, 'Lunch' => 500, 'Dinner' => 400];
    }
}
```

Figure 4.32 Decision Food based on probability

Input	Sources	Format
BMI Result	Mobile App	Integer
Child_Age	Child Database	Integer
Child Gender	Child Database	String(Male,Female)
Child Calorie	Calorie Calculation Module	Float

Table 4.5 Input of AI Recommended

Output	Destination	Format
Recommended Food list	Mobile App, Stored in AI List	JSON Array

Table 4.6 Output of AI Recommended

4.3.2 Physical Database Design

The database structure for the NutriGuardian system has been carefully translated from the logical design into a physical implementation using a relational database management system (RDBMS), specifically MySQL. The design consists of multiple base tables such as Parent, Child, Record_Child, Calorie_Record, and AI_Recommended, which represent the core data entities of the system. Each table is created using Data Definition Language (DDL) statements that define the table structure, primary keys, foreign keys, data types, and validation rules. For example, the Child table is linked to the Parent table using a foreign key constraint, enforcing referential integrity between parent and child records. Data validation is implemented using rules such as NOT NULL for required fields, ENUM for restricted values like gender, and CHECK constraints to ensure values like weight, height, and calorie input are valid and within logical limits. To enhance data accuracy and consistency, the BMI is calculated automatically using a generated column within the Record_Child table based on the weight and height values entered by the sensors.

The system also uses a date field with default values for logging records and maintaining historical data. All fields are indexed either by default (via primary/foreign keys) or manually for optimization on frequently queried fields such as intake dates or parent-child relationships. The InnoDB storage engine is used to support ACID compliance and foreign key constraints. Additionally, Data Control Language (DCL) is used to manage user access privileges by defining roles such as "parent user" with limited access to SELECT and INSERT permissions, and "admin user" with full control over the database. These access levels help maintain security and data privacy, especially in a system that handles sensitive health data. Overall, this database design ensures the NutriGuardian system is reliable, efficient, and scalable, and it directly supports the data flow shown in the system's ERD and DFD diagrams.

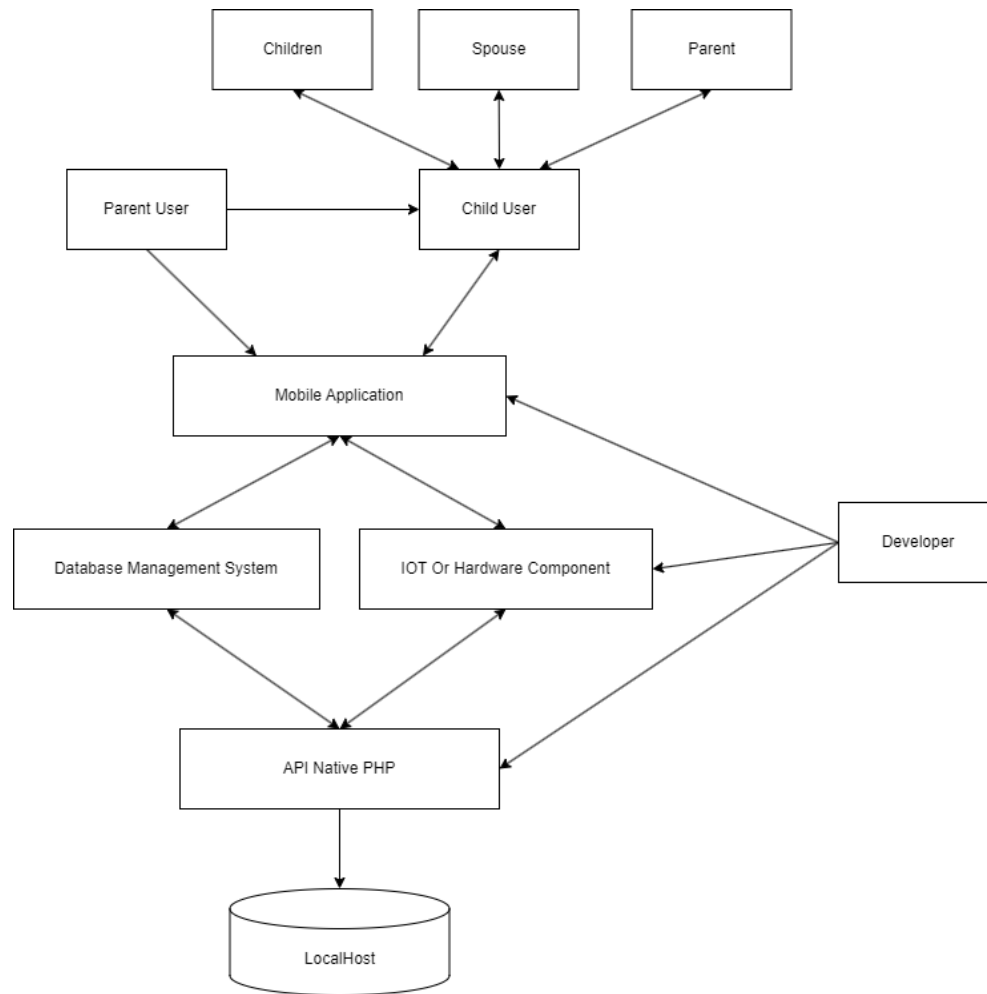


Figure 4.33 Database Management System

4.4 Conclusion

In this chapter, the database design for the NutriGuardian system has been thoroughly detailed, covering the translation of logical models into a physical database structure using MySQL as the DBMS. The implementation included the creation of base tables with appropriate field types, relational constraints, validation rules, and indexing strategies to ensure data integrity, accuracy, and performance. Supporting components such as the Entity Relationship Diagram (ERD), Data Flow Diagrams (DFD), and Data Dictionary were used to visualize the structure and flow of information across system modules. The application of DDL and DCL further solidified the system's backend logic by defining access roles and enforcing security controls on sensitive data.

Following the completion of this chapter, the next phase in the system development process will focus on the implementation and testing stage. This includes coding the application based on the system design, integrating IoT devices for height and weight input, configuring APIs for real-time communication, and incorporating AI recommendation logic. Functional testing, validation testing, and user interface testing will also be conducted to ensure the system performs as expected under different scenarios. These activities are crucial to transitioning from a working design into a fully functional and reliable application.

CHAPTER 5: IMPLEMENTATION

5.1 Introduction

This chapter outlines the implementation phase of the NutriGuardian system, where all planned designs and functionalities are brought to life in a working application. It marks the transition from the development stage to the actual deployment and testing of the system. The purpose of this chapter is to demonstrate how the system components including the IoT hardware, mobile application, admin web panel, and AI integration are integrated and function as a complete solution.

During this phase, various tasks were carried out, such as:

- Installing and configuring development tools (e.g. XAMPP, PHP, Arduino IDE).
- Integrating IoT hardware components like the HX711 module for weight and ultrasonic sensor for height measurement.
- Developing and linking mobile applications with local database and API.
- Creating the admin control panel (web platform) to manage user records and view measurement history.
- Connecting AI-based API for food suggestion, based on BMI results.
- Testing functionality and correcting bugs to ensure proper operation.

5.2 Software Development Environment Setup

The development environment for the NutriGuardian system was carefully configured to ensure smooth integration between all components including the mobile application, IoT devices, and the web-based admin control panel. This environment supports both client-side (user mobile app) and server-side (PHP server with MySQL database) development, as well as the setup required for Internet of Things (IoT) devices.

Development Environment Setup

The following tools and platforms were used in this project:

- i. XAMPP (Localhost Server):
 - Used to run Apache server and MySQL for backend operations.
 - Hosted all PHP API endpoints for communication between the mobile app and the database.
 - Default port: 80, Database Port: 3306.
- ii. PHP (Native):
 - Used to build RESTful APIs to connect mobile applications and store user measurements into the MySQL database.
- iii. MySQL Database:
 - Used for storing user data, BMI records, weight and height history, calorie intake, meal record and admin data control.
 - Database name: db_sihatselalu.

iv. Android Studio:

1. Flutter is used to build the cross-platform mobile application interface.
2. Developed using Dart language within Android Studio.
3. UI components, form validation, and HTTP request integration are implemented using libraries like http, shared_preferences, and provider.

v. Arduino IDE + Tinkercad:

- Used to design and build the IoT components such as the HX711 (weight) and ultrasonic sensor (height).

vi. Admin Web Panel (Browser-based):

- Developed using HTML, CSS, JavaScript, and PHP to allow admins to monitor and manage user health data.

vii. Network Configuration:

- All devices connect via the same WiFi router using IP-based communication.
- The ESP8266 sends data to the API hosted on <http://192.168.x.x/api/...> (based on local IP address).

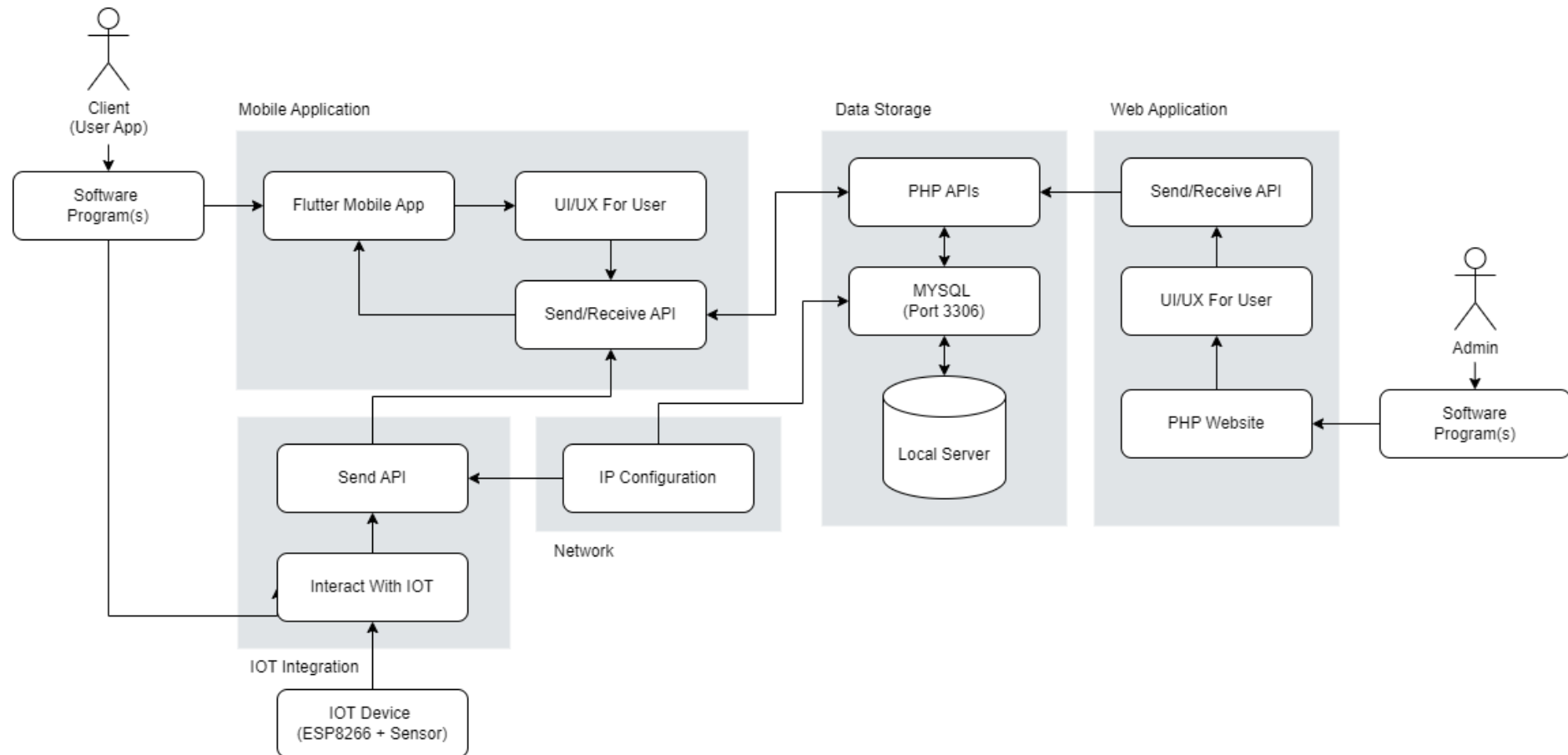


Figure 5.1: Software Environment Architecture Diagram

5.3 Version Control Procedure

To ensure effective tracking, collaboration, and management of the source code during the development of the NutriGuardian system, a version control system was employed. This system plays a vital role in organizing code changes, managing collaborative efforts, and maintaining a secure backup of the entire project source.

The version control for this project was managed using Git and GitHub. Git is a distributed version control system that allows multiple developers to work on different features simultaneously without overwriting each other's work. GitHub is an online repository hosting platform that provides centralized cloud storage for the project and supports remote collaboration.

Version Control Workflow:

1. Repository Initialization:

- A Git repository was initialized at the beginning of the development process for both mobile and backend components.

2. Branching Strategy:

- Separate branches were created for new features, bug fixes, and experimental tasks (e.g., main, feature-login, iot-integration, bugfix-api).
- This helped isolate work and avoid conflicts with the main project version.

3. Commits and Descriptive Messages:

- Each code update was committed with meaningful messages describing the changes, e.g., Add login form validation or Fix height sensor reading accuracy.

4. Regular Push to GitHub:

- Changes were regularly pushed to a private GitHub repository to maintain an updated cloud backup and enable easy recovery in case of local failures.

5. Review and Merge:

- Once a feature was complete and tested, it was merged back into the main branch after reviewing changes to ensure consistency and avoid conflicts.

6. Backup and Rollback:

- Git's ability to revert to previous commits allowed the team to easily roll back changes if bugs or issues were discovered in newer versions.

5.4 Implementation Status

The implementation phase of the NutriGuardian system involved the development of multiple core modules, each playing a critical role in the functionality of the application. Below is a summary of the development status of each major component, including brief descriptions, the time taken to develop, completion dates, and approximate codebase size.

Table 5.1: Implementation Table for Mobile Applications

Module / Component	Description	Duration	Completion Date	Size/ Tools
Module for User Interface (Homepage, Account Page, Front Page)	Frontend mobile UI design with responsive layout and navigation for users.	3 Week	5 April 2025	~3000 lines of Dart code, Flutter SDK
Module for Smart Weight Sensor (IoT)	Integration of HX711 with ESP8266 for capturing weight and sending via API.	1 Week	3 July 2025	93 lines of C++ code, Arduino IDE
Module for Smart Height Sensor (IoT)	Integration of ultrasonic sensors with ESP8266 to measure height via API.	1 Week	4 July	91 lines of C++ code, Arduino IDE
Module for Registration & Login	Handle interface and database integration for registration of the user and login form that are	1 Week	12 April 2025	~ 920 lines of Dart code, Android Studio and ~

	used data from local server			120 lines of PHP code, VS Code
Module for Track and Analysis Calorie	Handle interface for analysis and track the calories of each child user by recording each of the calories and show in the graph	1 Week	22 April 2025	720 lines of Dart code, Android Studio and ~100 lines of PHP code, VS Code
Module for Measurement and Display Weight/Height	Handle interface for the measurement weight and height based on IOT integration and record it in database	1 Week	11 July 2025	~530 lines of Dart code, Android Studio and 53 lines of PHP code, VS Code
Module for Choose and Record Meal Plan	Handle interface for user to choose the meal plan based on the suggestion on AI that has created and save it in database and can be recorded to show back the meal plan that has pick	2 Week	6 June 2025	~960 lines of Dart code, Android Studio and 66 lines of PHP code, VS Code
Integration of Artificial Intelligent (Kalman Filter	Handle the code to make suggestions based on calculation calories that has being taken and calories food per serving	1 Week	4 June 2025	~140 lines of PHP code, VS Studio

and Fuzzy Logic)	and make analysis based on suitable age.			
Module for Register, Manage, and Remove Child User	Handle interface for register the child user and manage the recorded information that interact with database.	2 Week	20 June 2025	~1190 lines of Dart code, Android Studio and ~180 lines of PHP code,

Table 5.2: Implementation Table for Web Applications

Module / Component	Description	Duration	Completion Date	Size/ Tools
Module for Manage User	Handle the interface to manage user that show all the record about user and can modify based on recorded user in database.	1 Week	27 June 2025	255 lines of PHP code and 543 lines of CSS code, VS Code
Module for Manage Food	Handle the interface for manage the food, modifying and remove the food and record it in database	1 Week	3 July 2025	~390 lines of PHP code and 543 lines of CSS code, VS Code

5.5 Conclusion

This chapter describes the implementation phase of the NutriGuardian system, covering all the major activities such as environment setup, version control, and the development progress of each module. The mobile app, IoT integration, API communication, and admin dashboard have been developed systematically, with most components achieving their intended functionalities within the expected timeline. Tools such as Flutter, PHP, MySQL, and Arduino IDE have been effectively utilized to build and connect both mobile and IoT aspects of the system.

The successful completion of the implementation phase marks a critical milestone in the project. Moving forward, the next activities will involve system testing, including functionality testing, user acceptance testing, and performance validation. This ensures the system operates reliably under real-world conditions before final deployment. Any detected bugs or usability issues will be addressed during the testing phase, followed by final documentation and project evaluation in the concluding chapters.

CHAPTER 6: TESTING

6.1 Introduction

This chapter outlines the testing phase of the NutriGuardian system, which is crucial to ensure the developed application meets its functional and non-functional requirements. System testing is conducted to validate the overall behavior of the system, identify any bugs or inconsistencies, and ensure that all modules—including the mobile app, IoT integration, and admin dashboard—work as expected in real-life scenarios. The testing process involves a series of methods such as unit testing, integration testing, and user acceptance testing (UAT) to evaluate system performance, accuracy of data (e.g., weight, height, calorie calculation), API responses, and the overall user experience. This chapter also highlights the strategies used to verify the system's reliability, usability, and responsiveness before final deployment.

6.2 Test Plan

Test plan is a structured document that describes the overall approach, scope, goals, and techniques for testing a software system. It acts as a roadmap for the testing process, detailing which features will be tested, the methods to be used, the required tools and resources, and the timeline of testing activities. The test plan also defines the testing environment, the criteria for success, and the responsibilities of each team member involved in testing. By having this plan, the testing process becomes more organized, systematic, and aligned with the project's objectives. It also helps in identifying and reducing risks, tracking progress, and ensuring that the final product meets the desired quality standards.

6.2.1 Test Organization

Test organization refers to the structured coordination and management of all testing activities in a software development project. It includes defining roles, responsibilities, processes, and resources required to properly plan, execute, and monitor testing tasks. A well-organized testing strategy ensures that testing objectives are clearly defined, responsibilities are distributed appropriately, and all activities are aligned with the overall development process. Test organization also covers documenting test cases, preparing test environments, and maintaining effective communication channels between team members and stakeholders. By following this structured approach, the testing process becomes more efficient, transparent, and accountable, ultimately ensuring the delivery of a stable and high-quality software product.

Table 6.1 represents the test organization table, which includes detailed information on the individuals engaged in the test stages. They will oversee managing, verifying, and executing the test.

Table 6.1: Test Organization

Tester ID	Name	Role	Responsibilities
TP-01	Ahmad Naquiuddin Bin Mohamad	Software Developer	Person responsible for system development
TP-02	Muhammad Zulhelmi Bin Noor Afendi	Software Tester	Acts as an end-user, interacting with the

			system from a consumer's perspective
TP-03	Muhammad Arif Aiman Bin Karim	Individual View Tester	Individual user testing and using the system from a personal perspective
TP-04	Wan Sakiah Binti Wan Long	Parental View Tester	Parent user testing and using the system through a parent account
TP-05	Ahmad Naquiuddin Bin Mohamad	Admin View Tester	Person which are responsible for testing the admin control panel

6.2.2 Test Environment

Test environment defines the setup, location, and technical requirements needed to carry out the testing process for the system. It ensures that the testing activities are conducted in a controlled and consistent manner, enabling accurate verification of the system's functionality, performance, and reliability. The test environment covers the physical location, hardware setup, firmware configuration, and the necessary preparations before testing begins.

6.2.2.1 Environment of Testing

Testing process will be conducted in a controlled environment that replicates the real operating conditions of the system. The environment includes a laboratory or discussion room equipped with stable internet connectivity, sufficient electrical supply, and space for IoT device setup such as smart scale, smart height sensor, and mobile devices. This setup ensures minimal interruptions and provides a suitable environment for reliable testing outcomes.

6.2.2.2 Hardware Setup

The hardware required for testing includes:

- **IoT Devices:** Smart scale and height measurement devices connected via ESP8266/ESP32 with HX711 load cell amplifier and ultrasonic sensors.
- **Mobile Device:** Oppo F11 (Android smartphone) to run the Flutter-based mobile application.
- **Server / Localhost:** XAMPP server running MySQL database to store and manage test data.
- **Laptop/PC:** Development machine with required IDEs and tools to monitor logs and run backend services.

This hardware setup replicates the real system environment and allows comprehensive testing across all modules.

6.2.2.3 Firmware Configuration

Firmware configuration involves programming the ESP8266/ESP32 microcontrollers with Arduino IDE to interface with IoT sensors (HX711 load cell, ultrasonic sensor) and transmit data to the server. The firmware is configured to connect to Wi-Fi, handle API communication, and ensure accurate data capture from the sensors. This step guarantees that the IoT devices are fully prepared for system integration testing.

6.2.2.4 Preparation and Training Before Testing

Before the testing phase, several essential preparations and training activities are conducted to ensure accuracy and efficiency. The IoT devices are properly connected, calibrated, and synchronized with the mobile application and backend server, while the required tools such as Flutter SDK, XAMPP, and Arduino IDE are installed and configured. Additionally, initial calibration of the smart scale is performed to guarantee precise measurements, and testers are briefed on how to operate the mobile application, IoT devices, and the administrative web panel. These steps collectively ensure that both the testing team and the environment are well-prepared, minimizing potential errors during the execution phase.

6.2.3 Test Schedule

Test schedule is a structured timetable that outlines when specific testing activities will be conducted for the Smart Healthcare Monitoring System. It defines the start and end dates for different testing phases, including unit testing, integration testing, system testing, and user acceptance testing. The schedule also assigns

resources, establishes dependencies between tasks, and sets milestones to monitor progress effectively. By aligning testing activities with the overall project timeline, the test schedule ensures that sufficient time is allocated for validating IoT device integration, mobile application features, data synchronization, and alert notifications. A well-organized test schedule is essential to minimize project risks, meet deadlines, and deliver a reliable and high-quality healthcare monitoring solution. Table 6.2 presents the planned timeline for completing testing activities across the system's modules.

Table 6.2: Test Schedule for Mobile Applications

Test Module	Test Page	Test ID	Start Date	End Date	Duration
User Module	Registration Page	TC_UM_01	1/8	2/8	1 day
	Login Page	TC_UM_02	2/8	3/8	1 day
	Reset Password Page	TC_UM_03	3/8	4/8	1 day
Interface and Navigation Module	Started Page	TC_INM_01	5/8	5/8	1 day
	Navigation Button	TC_INM_02	5/8	5/8	1 day
	Homepage	TC_INM_03	6/8	6/8	1 day
	Setting page	TC_INM_04	6/8	6/8	1 day
	Logout	TC_INM_05	6/8	6/8	1 day
	Choose Child Page	TC_INM_06	6/8	6/8	1 day
	About System Page	TC_INM_07	6/8	6/8	1 day
Calorie Record Module	Calorie Record Page	TC_CRM_01	10/8	11/8	1 day

IOT Module	IOT Measuring Page	TC_IOT_01	11/8	12/8	1 day
	IOT Measuring Hardware	TC_IOT_02	13/8	14/8	1 day
Meal Plan Module	Meal Plan Page	TC_MP_01	16/8	17/8	1 day
	AI Suggestion Meal Plan	TC_MP_02	18/8	21/8	3 days
Child Module	Child Page (Managing Child)	TC_CM_01	22/8	23/8	1 day
	Registration Child	TC_CM_02	24/8	25/8	1 day
History Record Module	History Record Child Page	TC_HR_01	26/8	27/8	1 day

Table 6.3: Test Schedule for Web Application

Test Module	Test Page	Test ID	Start Date	End Date	Duration
User Authentication Module	Login Page (Admin Control Panel)	TC_UAA_01	28/8	28/8	1 day
Interface Module	Homepage	TC_IA_01	28/8	28/8	1 day
Parent Module	Parent Page	TC_PA_01	28/8	28/8	1 day
Food Module	Food Page	TC_FA_01	28/8	28/8	1 day
	Add Food	TC_FA_02	29/8	29/8	1 day

6.3 Test Strategy

A test plan is a comprehensive document that outlines the methodology, objectives, and guidelines for testing procedures within the NutriGuardian project. It serves as a structured strategy to ensure the system delivers high-quality performance across its IoT components, AI features, mobile application, and web platform. The test plan defines the scope of testing, testing objectives, entry and exit criteria, and the levels of testing to be performed, including unit testing, integration testing, system testing, and user acceptance testing. It also specifies the required testing environment, resources, test automation plans, and potential risk assessments.

In addition, the plan identifies project stakeholders, clarifies their roles and responsibilities, and establishes communication channels throughout the testing phase. By providing a well-defined framework for test planning, execution, and evaluation, the strategy ensures alignment with project objectives, client expectations, and software quality standards. A clearly communicated and structured test plan enables the team to effectively manage resources, minimize risks, and maintain reliability, ultimately enhancing the overall quality of the NutriGuardian Project. Table 6.4 presents a detailed explanation of the testing strategies applied in this project etc. White-Box Testing and Black-Box Testing.

Table 6.4: Test Strategy

Aspect	White-Box Testing	Black-Box Testing
Definition	Testing that examines the internal structure, logic, and source code of the system.	Testing that evaluates the external behavior of the system without looking at the code.
Focus	Internal code paths, conditions, loops, and data flow.	Functional requirements, inputs, outputs, and user interactions.

Approach	Developers/testers design test cases based on the system's internal code structure.	Testers provide inputs and observe outputs to check if the system behaves as expected.
Usage in Project	Verifying authentication logic, data encryption, and processing algorithms for IoT health data.	Validating mobile app features, user login, health record display, and real-time alerts.
Example in System	Ensuring the heart rate calculation formula and data synchronization with the server work correctly.	Checking that abnormal heart rate triggers an alert notification on the user's app.
Advantage	Ensures code correctness, security, and efficient performance.	Ensures the system meets user requirements and is easy to use.
Limitation	Requires knowledge of the source code; time-consuming.	Cannot detect hidden code errors or security vulnerabilities.

6.3.1 Classes of Test

Test classes represent the type and various of testing techniques they are implementing in testing stage within the system. A test strategy's common kinds of tests may include as depicted in table 6.5:

Table 6.5: Classes of Testing

Test Class	Definition	Application in NutriGuardian
Unit Testing	Tests individual components or modules in	Verifying functions such as user login, calorie

	isolation to ensure each works correctly.	tracking, meal plan suggestion, and IoT device data collection.
Integration Testing	Tests the interaction between combined modules to ensure data flows correctly.	Ensuring IoT devices send data correctly to the mobile app and backend server, and that processed data is displayed accurately.
System Testing	Evaluates the complete system as a whole to check compliance with requirements.	Testing full functionality includes real-time monitoring, data storage, account management, and system performance.
User Acceptance Testing (UAT)	Final testing phase where end-users validate the system against real-world requirements.	Individual, parents, and healthcare staff use the app and answer the survey to confirm data accuracy, alert reliability, and system usability.

6.4 Test Design

Test design is the process of creating detailed test cases and scenarios that align with the testing objectives and requirements defined in the test strategy. It specifies the exact inputs, expected outputs, and testing conditions required to thoroughly evaluate the Smart Healthcare Monitoring System. The purpose of test design is to ensure comprehensive coverage of all system functionalities, including IoT device data collection, health data processing, real-time monitoring, alert notifications, user account management, and web panel administration. By addressing different characteristics and usage scenarios, test design helps identify potential issues early and

verifies that the system meets both functional and non-functional requirements. Effective test design is essential for detecting errors systematically, ensuring reliability and accuracy of healthcare data, and achieving high system performance within the project's timelines and objectives.

6.4.1 Test Description

A test description is a comprehensive document that outlines the details of a specific test case or scenario in software testing. It defines the unique test identifier, objective, prerequisites, input data, expected outcomes, and any special steps required for execution. By providing a structured and systematic overview, the test description ensures clarity of purpose, consistency in execution, and alignment with the overall testing goals. Within the Smart Healthcare Monitoring System, a well-prepared test description is crucial for verifying functionalities such as IoT data collection, real-time health monitoring, alert notifications, and user account operations. This approach supports effective quality assurance by detecting potential issues early and ensuring the system's reliability, security, and performance.

6.4.1.1 Test Description (User Module in Mobile Application)

Table below represents a test description of unit testing for user module which contain several scenarios that need to be tested: registration page, login page and reset password page. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.6: Test Description for User Module

Test Case ID	Test Case	Expected Result
--------------	-----------	-----------------

TC_UM_01_01	One of the user information not filled	An error message in toast show “Please fill all the form”
TC_UM_01_02	The user has filled in an invalid email address.	An error message in toast show “Please input valid email address”
TC_UM_01_03	The user has input invalid format of phone number that is 11 number	An error message in toast show “Please input valid phone number”
TC_UM_01_04	The password and reenter password doesn’t match	An error message in toast show “Password doesn’t match”
TC_UM_02_01	The username and password don’t exist	An error message “Failed to login”
TC_UM_02_02	The username exists but wrong password	An error message “Wrong password”
TC_UM_02_03	The username and password empty	An error message “Please fill username and password”
TC_UM_03_01	The email has empty	An error message “Please enter your email”
TC_UM_03_02	The email not valid	An error message “Please enter valid email”
TC_UM_03_03	The email does not exist in database	An error message “Email has not exist”

6.4.1.2 Test Description (Interface and Navigation Module in Mobile Application)

Table 6.7 below represents a test description of unit testing for interface and navigation modules which contain several scenarios that need to be tested: started page, navigation button, homepage, settings page, logout and choose child page. In

this description, it consists of the test case identification, test case and expected result for user. In this testing has use the integration testing.

Table 6.7: Test Case Description for Interface and Navigation Module

Test Case ID	Test Case	Expected Result
TC_INM_01_01	Click button “Create Account”	Navigate to registration page
TC_INM_01_02	Click button “Log In”	Navigate to login page
TC_INM_02_01	Click home icon in navbar	Navigate to homepage
TC_INM_02_02	Click calorie icon in navbar	Navigate to calorie record page
TC_INM_02_03	Click scale icon in navbar	Navigate to IOT page
TC_INM_02_04	Click arrow icon in navbar	Navigate to meal plan page
TC_INM_02_05	Click person icon in navbar	Navigate to settings page
TC_INM_03_01	Navigate to homepage	Show the step of use the system
		Show the calories remaining taken by days
		Show the current weight and height for child
TC_INM_04_01	Navigate to setting page	Show the profile image of the user and button to navigate to next page
TC_INM_04_02	Click “Your Account”	Navigate to account page
TC_INM_04_03	Click “Your Family Members”	Navigate to child user page
TC_INM_04_04	Click “Record Health”	Navigate to health record page
TC_INM_04_05	Click “About System”	Navigate to about system page
TC_INM_04_06	Click “Logout”	Sign out your account

6.4.1.3 Test Description (Calorie Record Module in Mobile Application)

Table below represents a test description of unit testing for calorie record module which contains a scenario that need to be tested: calorie record page. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.8: Test Case Description for Calorie Record

Test Case ID	Test Case	Expected Result
TC_CR_01_01	Choose the child to check the calorie record	Show the graph of calories per date for each child
TC_CR_01_02	Check the accurate calories based on calculation	The calories are calculate wall based on BMI result

6.4.1.4 Test Description (IOT Module in Mobile Application)

Table below represents a test description of unit testing for IOT module which contains several scenarios that need to be tested: IOT measuring page and IOT measuring features. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.9: Test Case Description for IOT Module

Test Case ID	Test Case	Expected Result
TC_IOT_01_01	Choose the child to measure the weight and height	Show the child information like name, gender

TC_IOT_01_02	Child steps on the weight scale	The number moves from 0.0 and increase
TC_IOT_01_03	Child stand straight on height sensor	The number moves from 0.0 and increase
TC_IOT_01_04	Child stay on scale in 6 second	The weight and height being recorded and automatically calculate the BMI
TC_IOT_02_01	Weight scale being measured accurately	The weight of user will be measuring but little bit different from real weight
TC_IOT_02_02	Height sensor being measured accurately	The height of user will be measuring but little bit different from real height
TC_IOT_02_03	Check accurate BMI on apps	The BMI has being calculate accurately

6.4.1.5 Test Description (Meal Plan Module in Mobile Application)

Table below represents a test description of unit testing for meal plan modules which contain several scenarios that need to be tested: meal plan page and AI suggestion meal plan page. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.10: Test Case for Meal Plan Module

Test Case ID	Test Case	Expected Result
TC_MP_01_01	Choose the child to plan a meal for each type of meal.	The information of child has been display
TC_MP_01_02	Choose the other date to check previous meal	The information of the meal that has record been display in each of the meal type.

TC_MP_01_03	Choose the current date to pick a meal	Message show, “No meals selected yet”
TC_MP_01_04	Add a meal follow by type of meal	The meal has been show based on meal type
TC_MP_01_05	Record the meal that has being selected	The meal that has pick being record
TC_MP_02_01	Choose a different age of user to check the suggestion food based on AI: Child	The meal has will being display based on age and gender based on AI suggestion
TC_MP_02_02	Choose a different age of user to check the suggestion food based on AI: Teen	The meal has will being display based on age and gender based on AI suggestion
TC_MP_02_03	Choose a different age of user to check the suggestion food based on AI: Adult	The meal has will being display based on age and gender based on AI suggestion
TC_MP_02_04	Choose a different age of user to check the suggestion food based on AI: Senior Citizen	The meal has will being display based on age and gender based on AI suggestion

6.4.1.6 Test Description (Child Module in Mobile Application)

Table below represents a test description of unit testing for child modules which contain several scenarios that need to be tested: child page and registration child page. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.11 Test Description for Child Module

Test Case ID	Test Case	Expected Result
TC_CM_01_01	Navigate to child page	The information of parent user and child user will be displayed
TC_CM_01_02	Add new child user under the parent user with the duplicate nickname	The dialog box will be displayed for register new child user
TC_CM_01_03	Add new child user with duplicate full name	The error message being displayed
TC_CM_01_04	Add new child user with over limit date	The error message being displayed
TC_CM_01_05	Add new child user with under birth age	The error message being displayed
TC_CM_01_06	Add new child user with one of the form not filled	The error message being displayed
TC_CM_01_07	Edit child information with duplicate nickname	Error message and child record not been saved
TC_CM_01_08	Edit child information with duplicate full name	Error message and child record not been saved
TC_CM_01_09	Edit child information with Birthdate under 1 year	Error message and child record not been saved
TC_CM_01_10	Edit child information with one of the form not filled	Error message and child record not been saved
TC_CM_01_11	Delete the member but cancel in confirmation	Child will not be deleted
TC_CM_01_12	Delete the member but delete in confirmation	Child will be deleted
TC_CM_02_01	Not fill in one of the information of child	Error message "Please fill all the form"
TC_CM_02_02	Not fill all the form	Error message "Please fill all the form"

TC_CM_02_03	Fill all the information of child	Message “Successful added”
-------------	-----------------------------------	----------------------------

6.4.1.7 Test Description (History Record Module in Mobile Application)

Table below represents a test description of unit testing for history record module which contain a scenario that need to be tested: history record page. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.12: Test Description for History Record Module

Test Case ID	Test Case	Expected Result
TC_HR_01_01	Choose child to check health record	Child information has been display
TC_HR_01_02	Check the weight of child and check based on date	Graph for the weight record has been displayed
TC_HR_01_03	Check the height of child and check based on date	Graph for the height record has been displayed
TC_HR_01_04	Check the BMI of child and check based on date	Graph for the BMI record has been displayed
TC_HR_01_05	Check the calorie of child and check based on date	Graph for calories record has been displayed

6.4.1.8 Test Description (User Authentication Module in Web Application)

Table below represents a test description of unit testing for user authentication module which contains a scenario that needs to be tested: login page. In this

description, it consists of the test case identification, test case and expected result for user.

Table 6.13 Test Description for User Authentication Module

Test Case ID	Test Case	Expected Result
TC_UAA_01_01	Username of admin empty	An error message “Please fill all form”
TC_UAA_01_02	Password not valid with username	An error message “Invalid password”
TC_UAA_01_03	Username does not exist	An error message “Username doesn’t exist”
TC_UAA_01_04	Insert right username and password	Successful login

6.4.1.9 Test Description (Interface Module in Web Application)

Table below represents a test description of unit testing for user modules which contain a scenario that needs to be tested: user page. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.14: Test Description for Interface Module in Web Application

Test Case ID	Test Case	Expected Result
TC_IA_01_01	Navigate to homepage	The index of the total of the user, total food
TC_IA_01_02	Click the manage parent navigation button	Navigate to parent page
TC_IA_01_03	Click the manage food navigation button	Navigate to manage food page

6.4.1.10 Test Description (Parent Module In Mobile Application)

Table below represents a test description of unit testing for parent module which contain several scenarios that need to be tested: registration page, login page and reset password page. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.15: Test Description for Parent Module in Web Application

Test Case ID	Test Case	Expected Result
TC_PA_01_01	Choose the parent to view	Found the parent
TC_PA_01_02	Search the not existing parent in search box	Not showing anything
TC_PA_01_03	Search the existing parent in search box	List of parent showing followed by searching.
TC_PA_01_04	Edit the parent information with duplicate username	Error message and information about parent will not saving
TC_PA_01_05	Edit the parent information with invalid phone number format	Error message and information about parent will not saving
TC_PA_01_06	Edit the parent information with one of the form empty	Error message and information about parent will not saving
TC_PA_01_07	Edit the form with the right information to edit	Information about parent will saving
TC_PA_01_08	Delete the parent but click cancel on confirmation box	Data about parent not deleted
TC_PA_01_09	Delete the parent but click delete on confirmation box	Data about parent being deleted

6.4.1.11 Test Description (Food Module In Web Application)

Table below represents a test description of unit testing for food module which contain several scenarios that need to be tested: food page and add food. In this description, it consists of the test case identification, test case and expected result for user.

Table 6.16: Test Description for Food Module

Test Case ID	Test Case	Expected Result
TC_FA_01_01	Search for the food that is not available in search box	Food not showing
TC_FA_01_02	Search for the food that is available in search box	Food showing followed by the searching
TC_FA_01_03	Click the add food button	Show dialog box for add food
TC_FA_01_04	Click the edit food button	Show dialog box for edit food
TC_FA_01_05	Not fill one of the form for edit food	Error message and food not saving
TC_FA_01_06	Fill all the form for edit food	No error message and saving successful
TC_FA_01_07	Click delete button	Show confirmation popup
TC_FA_01_08	Click cancel on confirmation	Food not deleting
TC_FA_01_09	Click delete on confirmation	Food deleting
TC_FA_02_01	Add new food but with the duplicate name	Error message and food not being added

TC_FA_02_02	Add new food but invalid serving size format	Error message and food not being added
TC_FA_02_03	Add new food but invalid calorie format	Error message and food not being added
TC_FA_02_04	Not fill one of the form	Error message
TC_FA_02_05	Fill all the form	Successful add new food

6.4.2 Test Data

Test data refers to a predefined set of inputs, values, or scenarios that are deliberately selected to evaluate how a software system behaves under various conditions. It plays a crucial role in the software testing process by ensuring comprehensive coverage of different cases, including valid inputs, invalid inputs, boundary values, edge cases, and typical usage scenarios. The primary objective of test data is to identify and correct defects, errors, or unexpected behaviors, thereby ensuring the system operates accurately, effectively, and securely in real-world situations. Well-constructed and diverse test data is essential for improving the overall quality, reliability, and safety of the NutriGuardian System.

Test data for the NutriGuardian System was also collected through surveys and personal testing involving different categories of users, namely individuals, parents, family members, and hospital staff. Demonstration videos and real testing sessions were conducted with these groups to obtain direct feedback, which served as a valuable source for evaluating the system's functionality and usability. In addition, live demonstrations were carried out in a hospital setting to gather insights from healthcare staff regarding the system's workflow and its effectiveness in addressing real-world problems, particularly in assisting users to record and monitor their height and weight.

A total of 11 respondents, consisting of individuals, family members, and hospital staff, participated in the survey. The survey was conducted over a period of two weeks, from 20 July 2025 to 2 August 2025, to collect information and feedback on system performance, user experience, and potential improvements. The combination of survey responses, demonstrations, and real-world testing ensured that the test data captured a diverse range of perspectives, supporting a more reliable and user-centered evaluation of the system.

Category of User
11 responses

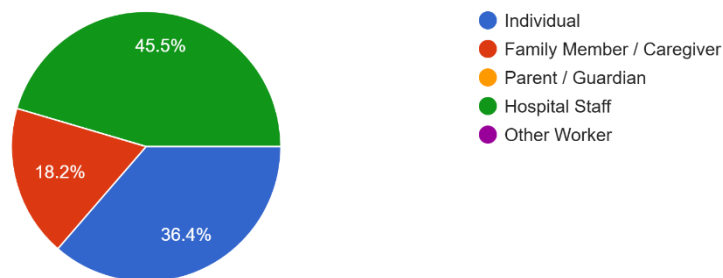


Figure 6.1: Total Responder of System

6.4.2.1 Test Data for Register Page

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : Register Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_UM_01_01	Register to login the account	One of the user information not filled	1. Fill the right username. 2. Fill the valid email address. 3. Not fill the phone number 4. Fill the right age 5. Fill the password 6. Fill the same password	Username: Ahmad Email Address: Ahmad@gmail.com Phone number: - Age: 25 Gender: Male Password: Abc@123 Confirm Password: Abc@123	Error message: "Please fill all the form".	Error message: "Please fill all the form".
TC_UM_01_02	Register to login the account	The user has filled in an invalid email address.	1. Fill the right username. 2. Fill the invalid email address. 3. Fill the right phone number	Username: Ahmad Email Address: Ahmadgmail.com Phone number: 01335432343	Error message: "Invalid email address"	Error message: "Invalid email address"

			4. Fill the right age 5. Fill the password	Age: 25 Gender: Male Password: Abc@123 Confirm Password: Abc@123		
TC_UM_01_03	Register to login the account	The user has input invalid format of phone number that is 11 number	1. Fill the right username. 2. Fill the right email address. 3. Fill the invalid phone number 4. Fill the right age 5. Fill the password 6. Fill the same password	Username: Ahmad Email Address: Ahmad@gmail.com Phone number: 01335432343 Age: 25 Gender: Male Password: Abc@123 Confirm Password: Abc@123	Error message "Invalid phone number"	Error message "Invalid phone number"
TC_UM_01_04	Register to login the account	The password and reenter password doesn't match	1. Fill the right username. 2. Fill the right email address. 3. Fill the right phone number 4. Fill the right age 5. Fill the password	Username: Ahmad Email Address: Ahmad@gmail.com Phone number: 123213 Age: 25 Gender: Male Password: Abc@123	Error message "Password doesn't match"	Error message "Password doesn't match"

			6. Fill the not same password	Confirm Password: Abc123		
--	--	--	-------------------------------	-----------------------------	--	--

6.4.2.2 Test Data for Login Page

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : Login Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 2/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_UM_02_01	Login to homepage	The username and password don't exist	1. Fill the wrong username. 2. Fill the wrong password.	Username: Abdul Password: 1234567	Error message: "Incorrect username or password".	Error message: "Incorrect username or password".
TC_UM_02_02	Login to homepage	The username exists but wrong password	1. Fill the right username. 2. Fill the wrong password.	Username: Ahmad Password: 1234567	Error message: "Incorrect username or password".	Error message: "Incorrect username or password".

TC_UM_02_03	Login to homepage	The username and password empty	1. Fill the empty username. 2. Fill the empty password.	Username: Password:	Error message: “Please fill the login form”	Error message: “Please fill the login form”
-------------	-------------------	---------------------------------	--	------------------------	--	--

6.4.2.3 Test Data for Reset Password Page

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : Reset Password Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 3/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_UM_03_01	Reset password to login	The email has empty	1. Fill the empty email address.	Email Address:	Error message: “Please enter your email”.	Error message: “Please enter your email”.
TC_UM_03_02	Reset password to login	The email not valid	1. Fill the invalid email address	Email Address: Ahmad.gmail.com	Error message: “Invalid email address”	Error message: “Invalid email address”
TC_UM_03_03	Reset password to login	The email does not exist in database	1. Fill the not existing email address	Email Address: Abdul@gmail.com	Error message: “Email doesn’t exist”	Error message: “Email doesn’t exist”

6.4.2.4 Test Data for Calorie Record Page

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : Calorie Record Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 10/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_CR_01_01	Parents to check calorie record child	Choose the child to check the calorie record	1. Choose the child	Name: Zulhelmi Age: 15 Gender: Male	The calories showing follow by the child	The calories showing follow by the child
TC_CR_01_02	Parents to check calorie record child	Check the accurate calories based on calculation	1. Choose one of the calories that has been recorded 2. Self-calculate the calories based on age, gender and BMI.	Calorie: 2450 kcal Age: 15 Gender: Male BMI: 21	The calculation accurately	The calculation accurately

6.4.2.5 Test Data for IoT Measuring Page

System Name : NutriGuardian Version : V1

(Smart Healthcare Monitoring System)

Module/Unit : IoT Measuring

Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad

Date : 5/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_IOT_01_01	Parents measure their child weight, height and calculate BMI.	Choose the child to measure the weight and height	1. Choose the child	Name: Zulhelmi Age: 15 Gender: Male	The information of child being display and IoT ready to measure the height and weight	The information of child being display and IoT ready to measure the height and weight
TC_IOT_01_02	Parents measure their child weight, height and calculate BMI.	Child steps on the weight scale.	1. Child left anything that can manipulate weight 2. Child step on the weight scale or IoT. 3. Parent check the numbering of weight on apps	Weight Child: 34 KG	The apps show the numbering increased from 0.0	The apps show the numbering increased from 0.0

TC_IOT_01_03	Parents measure their child weight, height and calculate BMI.	Child stand straight on height sensor	1. Child step on the height sensor. 2. Child stan straight on sensor 3. Parent check the numbering of height on apps	Height Child: 145 CM	The apps show the numbering increased from 0.0	The apps show the numbering increased from 0.0
TC_IOT_01_04	Parents measure their child weight, height and calculate BMI.	Measuring process has been cancel	1. Child does not stay on sensor and scale along 6 second	Second Stand: 4 second	The apps show the countdown stop numbering.	The apps show the countdown stop numbering.

6.4.2.6 Test Data for IoT Hardware Measuring

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : IoT Hardware Measuring Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
---------	---------------	-----------	------------	-----------	-----------------	---------------

TC_IOT_02_01	Parents check accurate measurement of their child weight, height and calculation of BMI.	Weight scale being measured	1. Child step on weight scale 2. Scale being measuring 3. Parent check on their app their child's weight.	Real weight of child: 35 KG Weight on app: 32.0 KG	The IoT measuring accurately	The IoT measuring little bit not accurately
TC_IOT_02_02	Parents check accurate measurement of their child weight, height and calculation of BMI.	Height sensor being measured	1. Child stand on height sensor 2. Sensor has been measuring 3. Parent check on their app their child's height	Real height of child: 145 cm Height on app: 1.45 M	The IoT measuring accurately	The IoT measuring little bit not accurately
TC_IOT_02_03	Parents check accurate measurement of their child weight, height and calculation of BMI.	Check accurate BMI on apps	1. Child measure their height and weight 2. Child stand on measuring along 6 second 3. Parent check on their chill BMI	Height: 1.45 CM Weight: 35 KG BMI: 16.65	The apps calculate accurately for BMI	The apps calculate accurately for BMI

6.4.2.7 Test Data for Meal Plan Page

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : Meal Plan Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_MP_01_01	Parent choose meal plan for child	Choose the child to plan a meal for each type of meal.	1. Choose the child	Name: Akmal Hakimi	Directed to meal plan page and show the type of meal and date	Directed to meal plan page and show the type of meal and date
TC_MP_01_02	Parent choose meal plan for child	Click the date of meal plan Choose the other date to check previous meal	1. Click the other date	Date: Sun 31	Button to add meal disable to click and show the previous meal	Button to add meal disable to click and show the previous meal
TC_MP_01_03	Parent choose meal plan for child	Choose the current date to pick a meal	1. Click on the current date	Date: Mon 01	Button to add meal able to click	Button to add meal able to click

TC_MP_01_04	Parent choose meal plan for child	Add a meal follow by type of meal	1. Choose the type of meal 2. Click the button add 3. Choose the meal 4. Click the meal 5. Click button done	Meal 1: Roti Bakar Meal 2: The Tarik	The selected meal will being showing in the list of the meal in meal page	The selected meal will being showing in the list of the meal in meal page
TC_MP_01_05	Parent choose meal plan for child	Record the meal that has being selected	1. Check the meal that has being selected 2. Scroll down to bottom of the page 3. Click button save meals	Breakfast: Meal 1: Roti Bakar Meal 2: The Tarik	The selected meal will recorded	The selected meal will recorded

6.4.2.8 Test Data for Meal Suggestion By AI

System Name : NutriGuardian Version : V1
(Smart Healthcare Monitoring System)

Module/Unit : Suggestion Meal Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
---------	---------------	-----------	------------	-----------	-----------------	---------------

TC_MP_02_01	Parent choose a different age of child user	Choose a different age of user to check the suggestion food based on AI: Child	1. Choose user from child age 2. Check the meal that will list	Name: Siti Balqis Age: 4 Years	The meal will be listed based on age for child	The meal will be listed based on age for child
TC_MP_02_02	Parent choose a different age of child user	Choose a different age of user to check the suggestion food based on AI: Teen	1. Choose user from teen age 2. Check the meal that will list	Name: Arif Aiman Age: 14 Years	The meal will be listed based on age for teen	The meal will be listed based on age for teen
TC_MP_02_03	Parent choose a different age of child user	Choose a different age of user to check the suggestion food based on AI: Adult	1. Choose user from adult age 2. Check the meal that will list	Name: Ahmad Nuriman Age: 30 Years	The meal will be listed based on age for adult	The meal will be listed based on age for adult

TC_MP_02_04	Parent choose a different age of child user	Choose a different age of user to check the suggestion food based on AI: Senior Citizen	1. Choose user from senior citizen age 2. Check the meal that will list	Name: Muhammad Sedek Age: 69 Years	The meal will be listed based on age for senior citizen	The meal will be listed based on age for senior citizen
-------------	---	---	--	---------------------------------------	---	---

6.4.2.9 Test Data for Manage Child User

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : Manage Child Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_CM_01_01	Parent to manage child user	Navigate to child page.	1. Click the child page button		Display all list of the child user under parent user	Display all list of the child user under parent user

TC_CM_01_02	Parent to manage child user	Add new child user under the parent user with the duplicate nickname	1. Click add child button 2. Fill the same nickname as other 3. Fill the right full name 4. Choose the family role 5. Choose the gender 6. Fill the right birthday date	Nickname: Akmal Full Name: Adam Hakim Role: Child Gender: Male Birthdate: 23/8/2005	Error message: “Duplicate Nickname”	Error message: “Duplicate Nickname”
TC_CM_01_03	Parent to manage child user	Add new child user with duplicate full name	1. Click add child button 2. Fill right nickname 3. Fill same full name with other 4. Choose the family role 5. Choose the gender - Fill the right birthday date	Nickname: Adam Full Name: Akmal Hakimi Role: Child Gender: Male Birthdate: 23/8/2005	Error message: “Duplicate Fullname”	Error message: “Duplicate Fullname”
TC_CM_01_04	Parent to manage child user	Add new child user with over limit date	1. Click add child button 2. Fill right nickname 3. Fill right full name 4. Choose the family role 5. Choose the gender 6. Fill the over date birthdate	Nickname: Adam Full Name: Adam Hakimi Role: Child Gender: Male Birthdate: 23/8/2026	Error message: “Invalid Birthdate”	Error message: “Invalid Birthdate”

TC_CM_01_05	Parent to manage child user	Add new child user with under birth age	1. Click add child button 2. Fill right nickname 3. Fill right full name 4. Choose the family role 5. Choose the gender 6. Fill the birthdate with under age like below 1 year	Nickname: Adam Full Name: Adam Hakimi Role: Child Gender: Male Birthdate: 23/3/2025	Error message: “Under Age”	Error message: “Under Age”
TC_CM_01_06	Parent to manage child user	Add new child user with one of the form not filled	1. Click add child button 2. Fill right nickname 3. Fill right full name 4. Not choose the family role 5. Choose the gender Fill the birthdate with under age like below 1 year	Nickname: Adam Full Name: Adam Hakimi Role: - Gender: Male Birthdate: 23/3/2025	Error message: “Please fill all form”	Error message: “Please fill all form”
TC_CM_01_07	Parent to manage child user	Edit child information with duplicate nickname	1. Click edit button 2. Navigate to edit page 3. Fill new nickname but duplicate with other 4. Click save button	Nickname: Adam	Error message: “Duplicate nickname”	Error message: “Duplicate nickname”

TC_CM_01_08	Parent to manage child user	Edit child information with duplicate full name	1. Click edit button 2. Navigate to edit page 3. Fill new full name but duplicate with other 4. Click save button	Full Name: Adam Hakimi	Error message: "Duplicate full name"	Error message: "Duplicate full name"
TC_CM_01_09	Parent to manage child user	Edit child information with Birthdate under 1 year	1. Click edit button 2. Navigate to edit page 3. Fill new birthdate but under 1 year 4. Click save button	Birthdate: 2/8	Error message: "Under age"	Error message: "Under age"
TC_CM_01_10	Parent to manage child user	Edit child information with one of the form not filled	1. Click edit button 2. Navigate to edit page 3. Keep full name empty 4. Click save button	Full name: -	Error message: "Please fill all form"	Error message: "Please fill all form"
TC_CM_01_11	Parent to manage child user	Delete the member but cancel in confirmation	1. Choose the member 2. Click button delete 3. Click cancel in confirmation box	Full name: Adam Hakimi Confirmation: Cancel	Information about child user not being deleted	Information about child user not being deleted
TC_CM_01_12	Parent to manage child user	Delete the member but	1. Choose the member 2. Click button delete	Full name: Adam Hakimi Confirmation: Delete	Information about child	Information about child

		delete in confirmation	3. Click delete in confirmation box		user being deleted	user being deleted
--	--	------------------------	-------------------------------------	--	--------------------	--------------------

6.4.2.10 Test Data for Add New Child User

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : Add New Child User Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_CM_02_01	Parent to manage child user	Not fill in one of the information of child	1. Fill the nickname 2. Fill the right full name 3. Not choose the family role 4. Choose the gender 5. Fill the right birthday date 6. Click button save	Nickname: Akmal Full Name: Adam Hakim Role: - Gender: Male Birthdate: 23/8/2005	Error message: "Please fill all the form".	Error message: "Please fill all the form".

TC_CM_02_02	Parent to manage child user	Not fill all the form	1. Not fill the nickname 2. Not fill the right full name 3. Not choose the family role 4. Not choose the gender 5. Not fill the right birthday date 6. Click button save	Nickname: - Full Name: - Role: - Gender: - Birthdate: -	Error message: "Please fill all the form".	Error message: "Please fill all the form".
TC_CM_02_03	Parent to manage child user	Fill all the information of child	1. Fill the nickname 2. Fill the right full name 3. Not choose the family role 4. Choose the gender 5. Fill the right birthday date 6. Click button save	Nickname: Akmal Full Name: Adam Hakim Role: Child Gender: Male Birthdate: 23/8/2005	Message: "Successful add new member" and data being recorded	Message: "Successful add new member" and data being recorded

6.4.2.11 Test Data for History Record

System Name : NutriGuardian

Version : V1

(Smart Healthcare Monitoring System)

Module/Unit : History Record

Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad

Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_HR_01_01	Parent check health history record of child	Choose child to check health record	1. Choose the child 2. Swipe to the left or right to choose other child 3. Click the view record	Nickname: Adam Role : Child Age: 4 years	The record of child being display in bar graph	The record of child being display in bar graph
TC_HR_01_02	Parent check health history record of child	Check the weight of child and check based on date	1. Click the weight 2. Choose the date on the graph 3. Click the graph	Date: 1/8 Weight: 34 KG	The weight of child being display at the graph	The weight of child being display at the graph
TC_HR_01_03	Parent check health history record of child	Check the height of child and check based on date	1. Click the height 2. Choose the date on the graph 3. Click the graph	Date: 1/8 Height: 156 CM	The height of child being display at the graph	The height of child being display at the graph

TC_HR_01_04	Parent check health history record of child	Check the BMI of child and check based on date	1. Scroll down 2. Click the BMI 3. Choose the date on the graph 4. Click the graph	Date: 1/8 BMI: 21	The BMI of child being display at the graph	The BMI of child being display at the graph
TC_HR_01_05	Parent check health history record of child	Check the calorie of child and check based on date	1. Scroll down 2. Click the calorie 3. Choose the date on the graph 4. Click the graph	Date: 1/8 Calorie: 2530 Kcal	The calorie of child being display at the graph	The calorie of child being display at the graph

6.4.2.12 Test Data for User Authenticate Admin

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : User Authenticate Admin Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
---------	---------------	-----------	------------	-----------	-----------------	---------------

TC_UAA_01_01	Login to account	Username of the form for admin empty	1. Not fill the username. 2. Fill the password 3. Click Log in button	Username: - Password: Abc@123	Error message: "Please fill all the form".	Error message: "Please fill all the form".
TC_UAA_01_02	Login to account	Password not valid with username	1. Fill the right username. 2. Fill the password but randomly 3. Click Log in button	Username: Admin Password: dssdf	Error message: "Wrong Username or Password".	Error message: "Wrong Username or Password".
TC_UAA_01_03	Login to account	Username does not exist	1. Fill the wrong username. 2. Fill the right password 3. Click Log in button	Username: Ahmad Password: Abc@123	Error message: "Wrong Username or Password".	Error message: "Wrong Username or Password".
TC_UAA_01_04	Login to account	Insert right username and password	1. Fill the right username. 2. Fill the right password 3. Click Log in button	Username: Admin Password: Abc@123	Successful login and navigate to homepage	Successful login and navigate to homepage

6.4.2.13 Test Data for Parent Module

System Name : NutriGuardian

Version : V1

(Smart Healthcare Monitoring System)

Module/Unit : Parent Module

Revision : -

Process By : Ahmad Naqiuddin Bin Mohamad

Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_PA_01_01	Admin to check the parent user	Choose the parent to view	1. Navigate to manage parent page 2. Choose the parent to view 3. Scroll the list until found the parent	Parent Name: Zulhelmi Age: 36	The parent found in the list of the parent	The parent found in the list of the parent
TC_PA_01_02	Admin to check the parent user	Search the not existing parent in search box	1. Click on the search box 2. Write the name randomly to be search 3. Click button search	Parent Name: Zulnaim	The list of name will not display	The list of name will not display
TC_PA_01_03	Admin to check the parent user	Search the existing parent in search box	1. Click on the search box 2. Write the name to be search 3. Click button search	Parent Name: Zulhelmi	The list of name will display	The list of name will display
TC_PA_01_04	Admin to edit the parent user	Edit the parent information	1. Click the edit button.	Username: Naqiuddin	Error message: "Username duplicate" and	Error message: "Username duplicate" and

		with duplicate username	2. Change the username but duplicate with the other username 3. Click the save button		information no being save	information no being save
TC_PA_01_05	Admin to edit the parent user	Edit the parent information with invalid phone number format	1. Click the edit button. 2. Change the phone number but invalid phone number format 3. Click the save button	Phone number: 0234434	Error message: "Invalid phone number" and information no being save	Error message: "Invalid phone number" and information no being save
TC_PA_01_06	Admin to edit the parent user	Edit the parent information with one of the form empty	1. Click the edit button. 2. Change the phone number to empty 3. Click the save button	Phone number: -	Error message: "Empty form! Please fill all" and information no being save	Error message: "Empty form! Please fill all" and information no being save
TC_PA_01_07	Admin to edit the parent user	Edit the form with the right information to edit	1. Click the edit button. 2. Change the right phone number 3. Click the save button	Phone number: 0197285465	Successful save the information	Successful save the information

TC_PA_01_08	Admin to delete the parent user	Delete the parent but click cancel on confirmation box	1. Choose the parent 2. Click button delete 3. Click the cancel button	Parent name; Zulhelmi	The parent information not being deleted	The parent information not being deleted
TC_PA_01_09	Admin to delete the parent user	Delete the parent but click delete on confirmation box	1. Choose the parent 2. Click button delete 3. Click the delete button	Parent name; Zulhelmi	The parent information being deleted	The parent information being deleted

6.4.2.14 Test Data for Manage Food Page

System Name : NutriGuardian Version : V1
 (Smart Healthcare Monitoring System)
 Module/Unit : Manage Food Revision : -
 Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
---------	---------------	-----------	------------	-----------	-----------------	---------------

TC_FA_01_01	Admin to check the food of meal	Choose the food to view	1. Navigate to manage food page 2. Choose the parent to view 3. Scroll the list until found the parent	Food Name: Nasi Lemak	The found found in the list of the food	The parent found in the list of the food
TC_FA_01_02	Admin to check the food of meal	Search the not available food in search box	1. Click on the search box 2. Write the food name randomly to be search 3. Click button search	Food Name: Laksa	The list of food will not display	The list of food will not display
TC_FA_01_03	Admin to check the food of meal	Search the available food in search box	1. Click on the search box 2. Write the name to be search 3. Click button search	Food Name: Nasi Lemak	The list of food will display	The list of food will display
TC_fa_01_04	Admin to edit the food of meal	Edit the food information with duplicate food name	1. Click the edit button. 2. Change the username but duplicate with the other username 3. Click the save button	Food Name: Nasi Lemak	Error message: "Food name duplicate" and information no being save	Error message: "Food name duplicate" and information no being save
TC_FA_01_05	Admin to edit the food of meal	Edit the food information with invalid	1. Click the edit button.	Calories: Asd Serving size:	Error message: "Invalid Calories and	Error message: "Invalid Calories and

		calorie and serving size format	2. Change the calorie and serving size but invalid format 3. Click the save button	2v	Serving size” and information not being save	Serving size” and information not being save
TC_FA_01_06	Admin to edit the food of meal	Edit the food information with one of the form empty	1. Click the edit button. 2. Change the serving size to empty 3. Click the save button	Serving size: -	Error message: “Empty form! Please fill all” and information no being save	Error message: “Empty form! Please fill all” and information no being save
TC_PA_01_07	Admin to edit the food of meal	Edit the form with the right information to edit	4. Click the edit button. 5. Change the right serving size 6. Click the save button	Serving size: 150	Successful save the information	Successful save the information
TC_PA_01_08	Admin to delete the food of meal	Delete the food but click cancel on confirmation box	1. Choose the food 2. Click button delete 3. Click the cancel button	Food name: Nasil Lemak	The food information not being deleted	The food information not being deleted
TC_PA_01_09	Admin to edit the food of meal	Delete the food but click delete on	4. Choose the food 5. Click button delete 6. Click the delete button	Food name; Nasi Lemak	The food information being deleted	The food information being deleted

		confirmation box				
--	--	---------------------	--	--	--	--

6.4.2.15 Test Data for Add New Food

System Name : NutriGuardian (Smart Healthcare Monitoring System) Version : V1

Module/Unit : Add New Food Revision : -

Process By : Ahmad Naquiuddin Bin Mohamad Date : 1/8/2025

Test ID	Test Scenario	Test Case	Test Steps	Test Data	Expected Result	Actual Result
TC_FA_02_01	Admin to add new food of meal	Add new food but with the duplicate name	1. Click the add new food button 2. Fill the food name but duplicate with other food 3. Choose the right food category 4. Choose the right food type	Food name: Nasi Lemak Category: Carb Meal type: Breakfast Calorie; 340 Serving size; 200	Error message: "Duplicate food name" and food not being recorded	Error message: "Duplicate food name" and food not being recorded

			5. Fill the right serving size of food 6. Fill the right Calorie per meal 7. Click button add food			
TC_FA_02_02	Admin to add new food of meal	Add new food but invalid serving size format	1. Click the add new food button 2. Fill the right food name 3. Choose the right food category 4. Choose the right food type 5. Fill the invalid serving size format of food 6. Fill the right Calorie per meal 7. Click button add food	Food name: Nasi Minyak Category: Carb Meal type: Breakfast Calorie; 345 Serving size; fd	Error message: "Invalid format for serving size" and food not being recorded	Error message: "Invalid format for serving size" and food not being recorded
TC_FA_02_03	Admin to add new food of meal	Add new food but invalid calorie format	1. Click the add new food button 2. Fill the right food name 3. Choose the right food category	Food name: Nasi Minyak Category: Carb Meal type: Breakfast Calorie; sd	Error message: "Invalid format for calorie" and	Error message: "Invalid format for calorie" and

			4. Choose the right food type 5. Fill the right serving size of food 6. Fill the invalid Calorie per meal format 7. Click button add food	Serving size; 200	food not being recorded	food not being recorded
TC_FA_02_04	Admin to add new food of meal	Not fill one of the form	1. Click the add new food button 2. Fill the right food name 3. Choose the right food category 4. Choose the right food type 5. Fill the right serving size of food 6. Not fill the Calorie per meal 7. Click button add food	Food name: Nasi Minyak Category: Carb Meal type: Breakfast Calorie; - Serving size; 200	Error message: "Please fill all the form" and food not being recorded	Error message: "Please fill all the form" and food not being recorded
TC_FA_02_05	Admin to add new food of meal	Fill all the form	1. Click the add new food button 2. Fill the right food name	Food name: Nasi Minyak Category: Carb	Successful add new food	Successful add new food

			3. Choose the right food category 4. Choose the right food type 5. Fill the right serving size of food 6. Fill the right Calorie per meal 7. Click button add food	Meal type: Breakfast Calorie; 243 Serving size; 200		
--	--	--	--	--	--	--

6.5 Test Result and Analysis

Test results and analysis refer to the process of examining and evaluating the outcomes of software testing activities. After executing test cases, the results are documented, emphasizing any defects or inconsistencies identified during testing. These outcomes are then reviewed by the testing team to determine the severity of the issues, analyze their root causes, and prioritize them for resolution. This process enables project stakeholders to assess whether the software meets required quality and functionality standards, and to decide if it is ready for release or requires further improvements. Effective test result analysis is crucial for promptly addressing software defects, improving overall product quality, and delivering a dependable and satisfying user experience.

Table 6.17: Test Pass And Fail Result and Analysis For All Module

Module/ Component	Result		
Test Case ID	Test Case	Pass	Fail
TC_UM_01_01	One of the user information not filled	/	
TC_UM_01_02	The user has filled in an invalid email address.	/	
TC_UM_01_03	The user has input invalid format of phone number that is 11 number	/	
TC_UM_01_04	The password and reenter password doesn't match	/	
TC_UM_02_01	The username and password don't exist	/	
TC_UM_02_02	The username exists but wrong password	/	

TC_UM_02_03	The username and password empty	/	
TC_UM_03_01	The email has empty	/	
TC_UM_03_02	The email not valid	/	
TC_UM_03_03	The email does not exist in database	/	
TC_CR_01_01	Choose the child to check the calorie record	/	
TC_CR_01_02	Check the accurate calories based on calculation	/	
TC_IOT_01_01	Choose the child to measure the weight and height	/	
TC_IOT_01_02	Child steps on the weight scale	/	
TC_IOT_01_03	Child stand straight on height sensor	/	
TC_IOT_01_04	Child stay on scale in 6 second	/	
TC_IOT_02_01	Weight scale being measured accurately		/
TC_IOT_02_02	Height sensor being measured accurately		/
TC_IOT_02_03	Check accurate BMI on apps	/	
TC_MP_01_01	Choose the child to plan a meal for each type of meal.	/	
TC_MP_01_02	Choose the other date to check previous meal	/	
TC_MP_01_03	Choose the current date to pick a meal	/	
TC_MP_01_04	Add a meal follow by type of meal	/	

TC_MP_01_05	Record the meal that has been selected	/	
TC_MP_02_01	Choose a different age of user to check the suggestion food based on AI: Child	/	
TC_MP_02_02	Choose a different age of user to check the suggestion food based on AI: Teen	/	
TC_MP_02_03	Choose a different age of user to check the suggestion food based on AI: Adult	/	
TC_MP_02_04	Choose a different age of user to check the suggestion food based on AI: Senior Citizen	/	
TC_CM_01_01	Navigate to child page	/	
TC_CM_01_02	Add new child user under the parent user with the duplicate nickname	/	
TC_CM_01_03	Add new child user with duplicate full name	/	
TC_CM_01_04	Add new child user with over limit date	/	
TC_CM_01_05	Add new child user with under birth age	/	
TC_CM_01_06	Add new child user with one of the form not filled	/	
TC_CM_01_07	Edit child information with duplicate nickname	/	

TC_CM_01_08	Edit child information with duplicate full name	/	
TC_CM_01_09	Edit child information with Birthdate under 1 year	/	
TC_CM_01_10	Edit child information with one of the form not filled	/	
TC_CM_01_11	Delete the member but cancel in confirmation	/	
TC_CM_01_12	Delete the member but delete in confirmation	/	
TC_CM_02_01	Not fill in one of the information of child	/	
TC_CM_02_02	Not fill all the form	/	
TC_CM_02_03	Fill all the information of child	/	
TC_HR_01_01	Choose child to check health record	/	
TC_HR_01_02	Check the weight of child and check based on date	/	
TC_HR_01_03	Check the height of child and check based on date	/	
TC_HR_01_04	Check the BMI of child and check based on date	/	
TC_HR_01_05	Check the calorie of child and check based on date	/	
TC_UAA_01_01	Username of admin empty	/	
TC_UAA_01_02	Password not valid with username	/	
TC_UAA_01_03	Username does not exist	/	
TC_UAA_01_04	Insert right username and password	/	
TC_PA_01_01	Choose the parent to view	/	

TC_PA_01_02	Search the existing parent in search box	/	
TC_PA_01_03	Search the existing parent in search box	/	
TC_PA_01_04	Edit the parent information with duplicate username	/	
TC_PA_01_05	Edit the parent information with invalid phone number format	/	
TC_PA_01_06	Edit the parent information with one of the form empty	/	
TC_PA_01_07	Edit the form with the right information to edit	/	
TC_PA_01_08	Delete the parent but click cancel on confirmation box	/	
TC_PA_01_09	Delete the parent but click delete on confirmation box		/
TC_FA_01_01	Search for the food that is not available in search box	/	
TC_FA_01_02	Search for the food that is available in search box	/	
TC_FA_01_03	Search for the food that is available in search box	/	
TC_FA_01_04	Click the edit food button	/	
TC_FA_01_05	Not fill one of the form for edit food	/	
TC_FA_01_06	Fill all the form for edit food	/	
TC_FA_01_07	Click delete button	/	
TC_FA_01_08	Click cancel on confirmation	/	
TC_FA_01_09	Click delete on confirmation	/	

TC_FA_02_01	Add new food but with the duplicate name	/	
TC_FA_02_02	Add new food but invalid serving size format	/	
TC_FA_02_03	Add new food but invalid calorie format	/	
TC_FA_02_04	Not fill one of the form	/	
TC_FA_02_05	Fill all the form	/	
Total Passes		72/75	
Total Fail		3/75	

Table 6.18: The Summary of Test Result and Analysis For All Module

Test Module	Total Test Case	Pass Rate	Fail Rate
User Module	10	10/10	0/10
Calorie Record Module	2	2/2	0/2
IOT Module	7	5/7	2/7
Meal Plan Module	9	9/9	0/9
Child Module	15	15/15	0/15
History Record Module	5	5/5	0/5
User Authentication Module	4	4/4	0/4
Parent Module	9	8/9	1/9
Food Module	14	14/14	0/14
Total	75	72/74	3/75

6.6 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) is a critical phase in the software development lifecycle where end users or stakeholders validate whether a system or application truly meets their requirements and expectations. This stage takes place after functional, integration, and system testing, but before the software is officially deployed into the production environment. Unlike earlier testing phases, which focus on technical correctness and system stability, UAT emphasizes real-world usability and business alignment.

During UAT, the intended users or representatives from the client side evaluate the software by executing predefined test scenarios and practical use cases that reflect real business operations. Their focus is not only on verifying that the system functions correctly, but also on ensuring that it supports their day-to-day workflows, meets business goals, and provides a smooth user experience. This process helps uncover any mismatches between the documented requirements and the delivered product.

UAT serves as the final quality checkpoint before release. Successful completion of UAT demonstrates that the software is ready for production deployment, minimizes the risk of post-release issues, and builds confidence among stakeholders. Beyond technical validation, UAT also plays a key role in driving user satisfaction, ensuring smoother adoption, and ultimately confirming that the software delivers the expected business value.

In this UAT, the focus is on several categories of users, namely Individuals, Family Members or Caregivers, Parents or Guardians, Hospital Staff, and Other Workers. For the respondents, the feedback collected provides valuable insights into how each group interacts with the system and the unique perspectives they bring. In the figure 6.2 explained about the participants, Hospital Staff formed the largest group, representing 31.6% of responses. Their

involvement is crucial as they interact with the system in a professional healthcare environment, ensuring that the application is practical, reliable, and supportive of their daily medical tasks. Parents or Guardians contributed 26.3% of responses, offering feedback that emphasizes ease of use, reliability, and suitability for family-related needs. Their input ensures that the system can cater to managing dependents effectively.

Both Individuals and Family Members or Caregivers each accounted for 21.1% of the responses. Individuals provided perspectives based on direct personal usage, highlighting usability and accessibility, while caregivers focused on how the system supports them in monitoring and assisting others. Interestingly, there were no responses from the “Other Worker” category, which indicates that the primary feedback was drawn from direct users and those closely associated with healthcare or caregiving roles. Overall, the diversity of respondents ensures that the UAT results are balanced and representative of both professional and personal use cases, strengthening the system’s readiness for deployment.

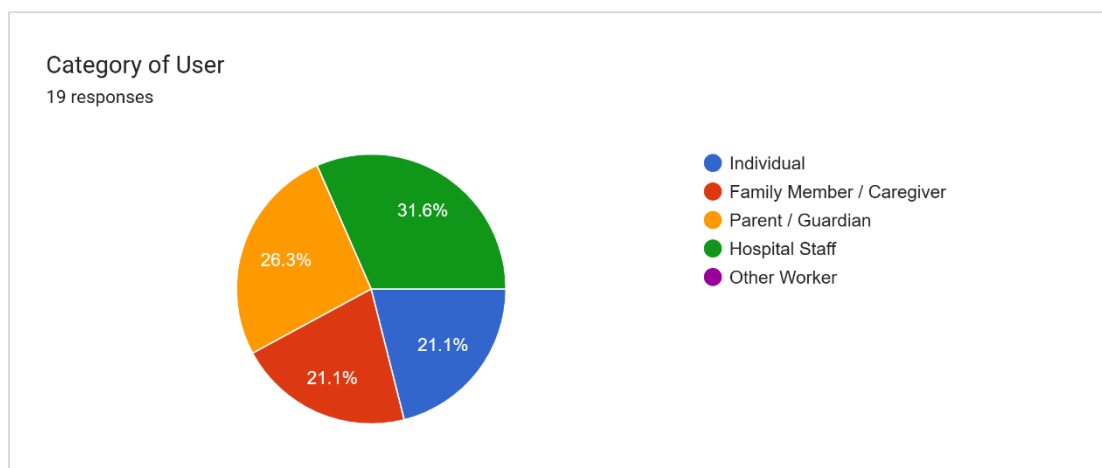


Figure 6.2: The Percentage Category of User

Figure 6.3 below illustrates the percentage of respondents who have previously used a digital health monitoring system. Based on the responses, approximately 63% of participants

reported having prior experience with such systems before being introduced to NutriGuardian, while 36.8% indicated they had not used one before.

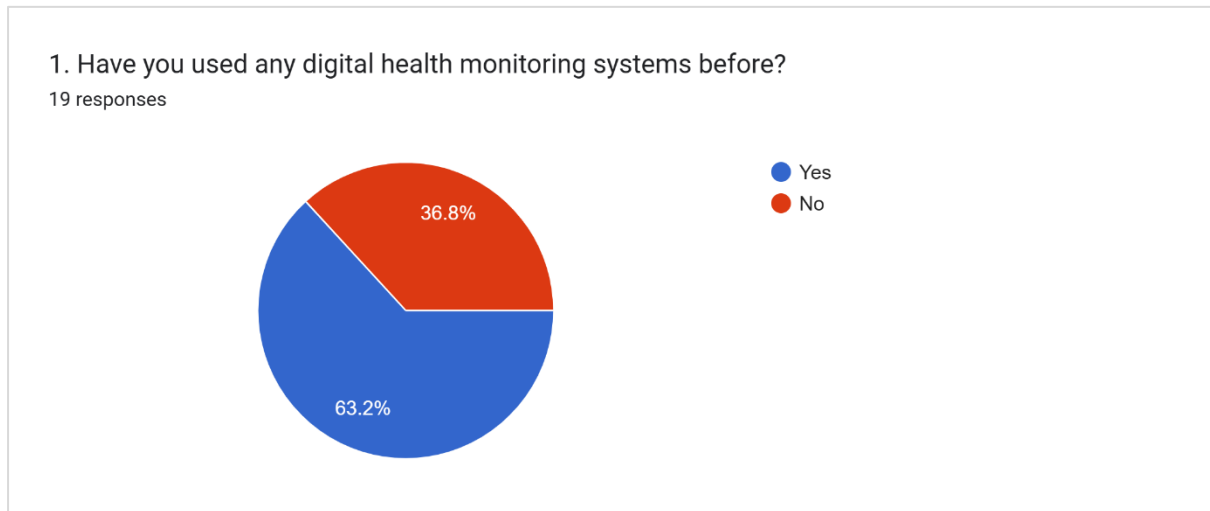


Figure 6.3: The percentage of used digital health system before

Figure 6.4 below shows the percentages of easy to understand the flow of the system and how the system might work. It was found that the 73.7% strongly agree about the system and beside that 21.1% just agree and other is just natural. Which mean that the system can easily understand the system.

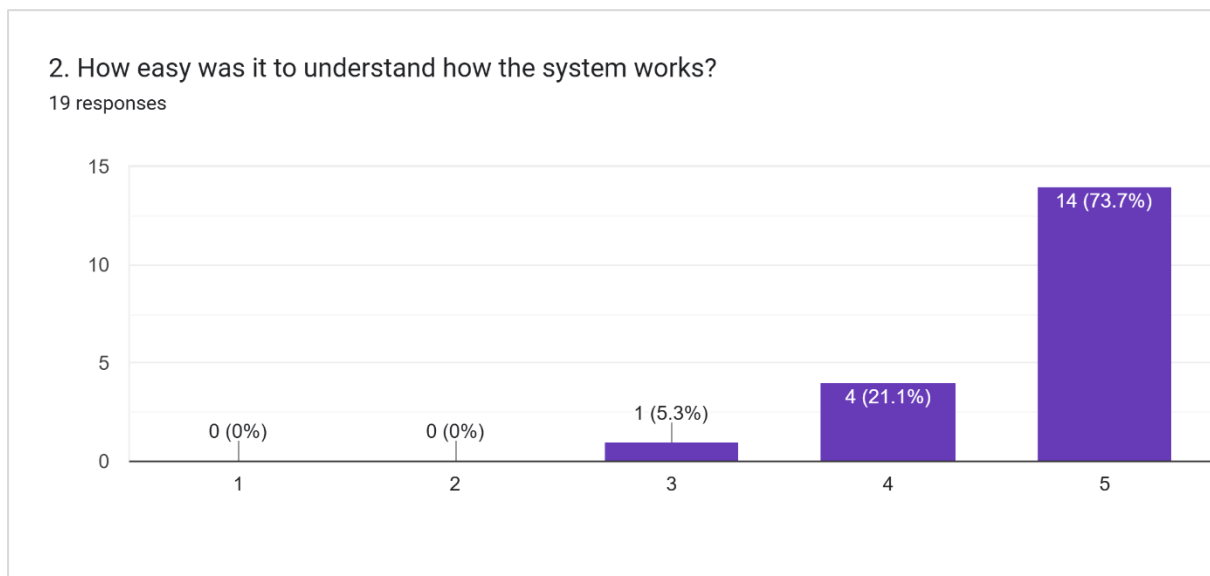


Figure 6.4: The percentage of easily can understand system works

Figure 6.5 below shows the percentage of respondents which satisfied with the accuracy of the height and weight of measurement applications that could be developed. In the respondents that can found that 52.6% of respondents strongly agree that they want the accuracy and 42.1% just agree with the accuracy. It explain that most of them are

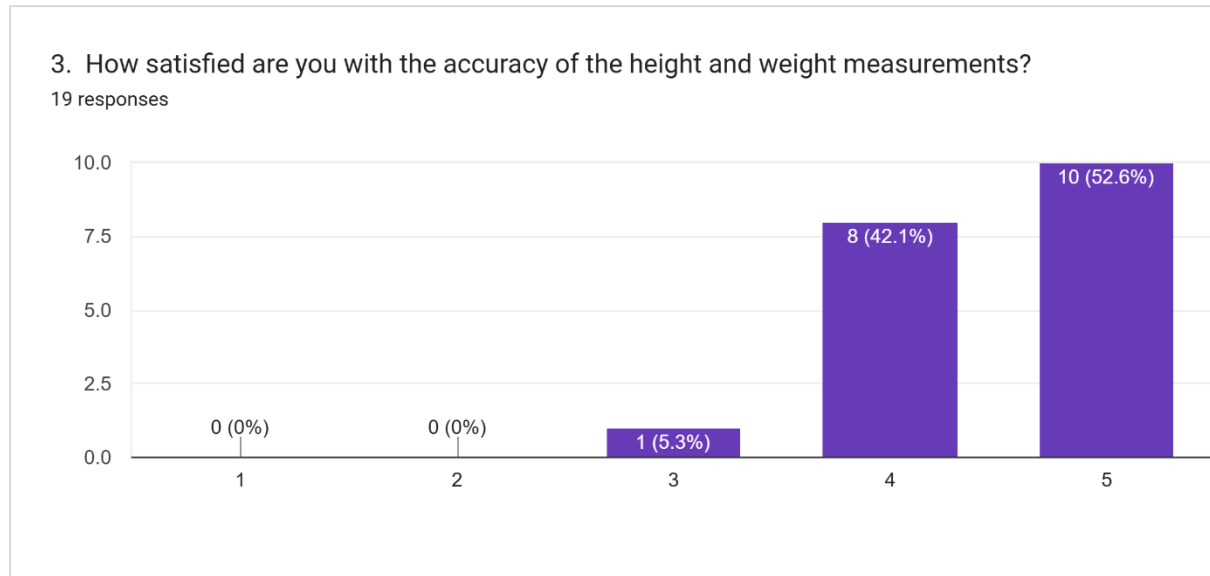


Figure 6.5: The percentage of accuracy of measurement

Figure 6.6 below shows the percentage of respondents which agree about automatic measurement process can save time compared to manual measurement method. In percentage of respondents, it can found that over 89% of the respondents strongly agree about that beside around 10% just agree with that. It can explain that most of the respondents just want to use measurement tools that can automatically work to measure the height and weight.

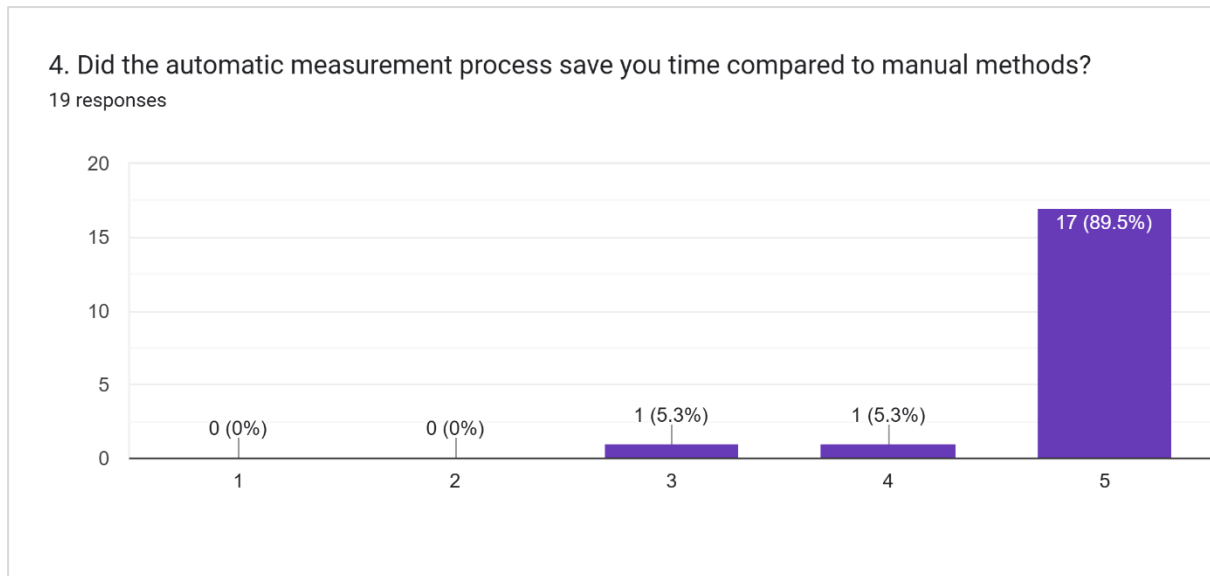


Figure 6.6: The percentage of automatic measurements tools

Figure 6.7 below shows the percentage of respondents which likely would be use the system regularly at home or in workspace. It was found that around 63% of respondents strongly agree about that and 31% just agree. It explain that most of that want to used the system in anything at their place

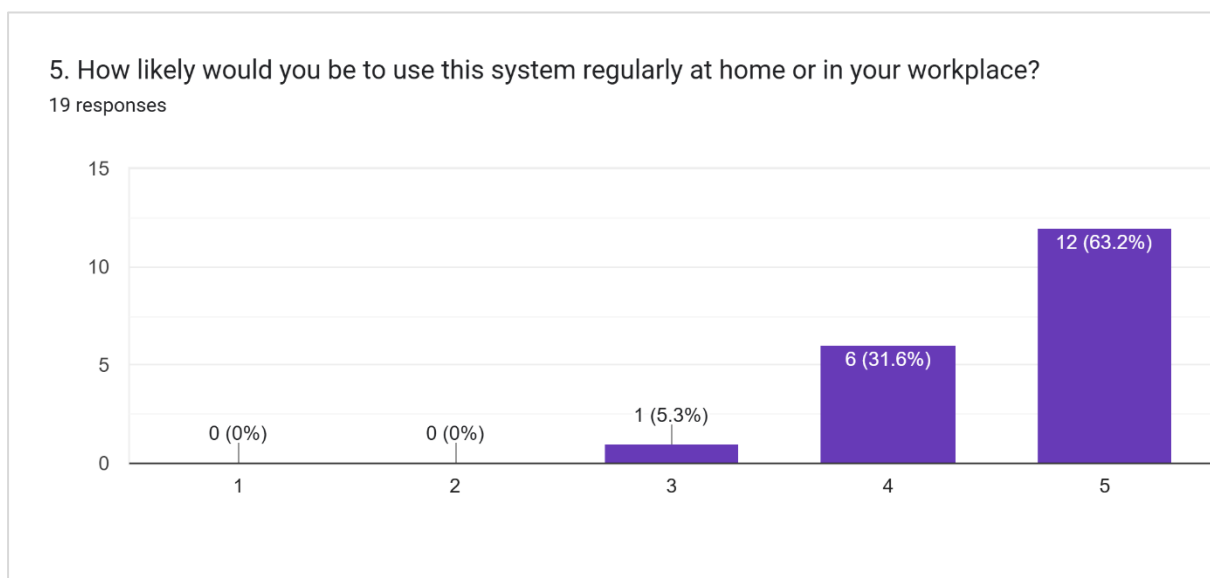


Figure 6.7: The percentage of likely would use the system regularly

Figure 6.8 below shows the percentage of respondents which agree that the design of the system is user-friendly and safe to use. From the percentage, most of the respondents agree about 78.9% and others disagree and are not sure about the system. That explains most of the respondents agree than disagree about the system is user-friendly and safe to use.

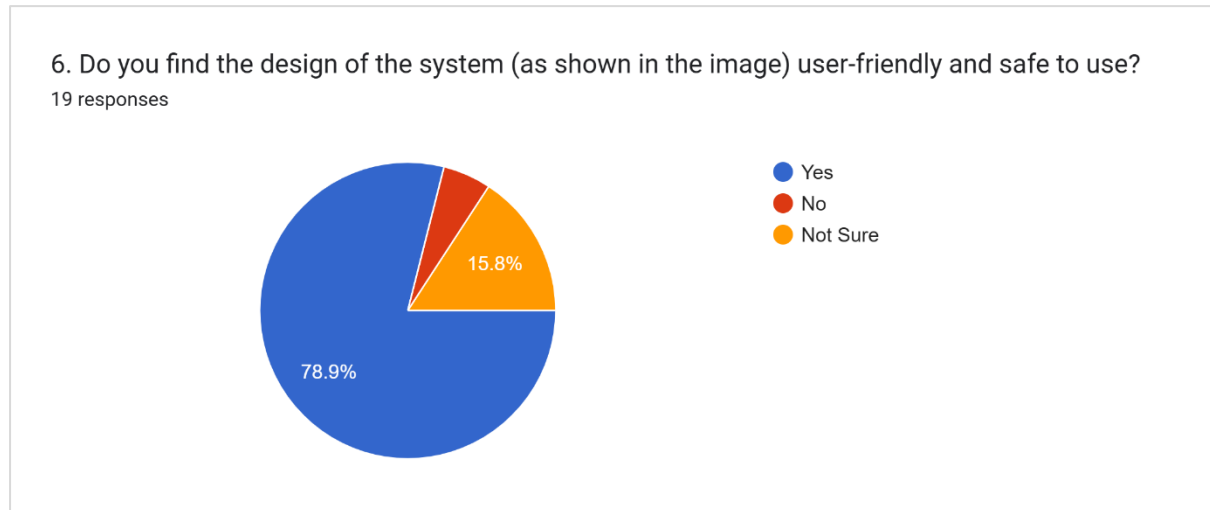


Figure 6.8: The percentage of system user-friendly and safe to use

Figure 6.9 below shows the percentage of respondents which are agree to recommend this application to their friends and family members. In this percentage, it can found that most of the respondents strongly agree about the system are good to be recommended to their friend and others just agree to this system.

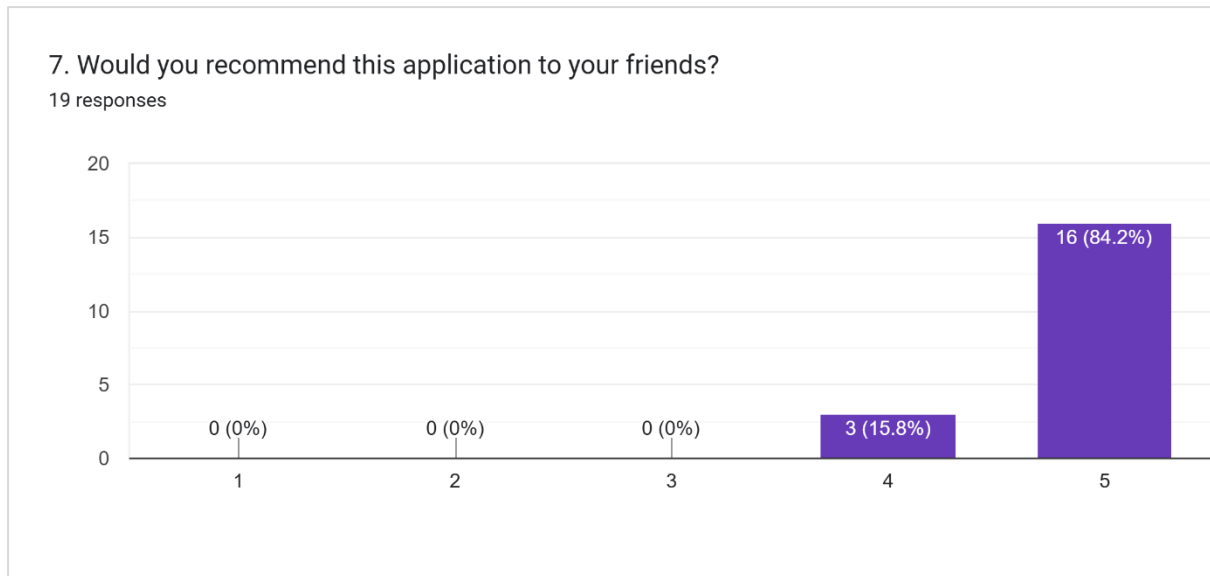


Figure 6.9: The percentage of the recommended system to their friend

Figure 6.10 below illustrates the percentage of respondents' feedback regarding the ease of remembering how to use the application. The results indicate that most respondents strongly agreed that the application is simple to understand and easy to recall when using it.

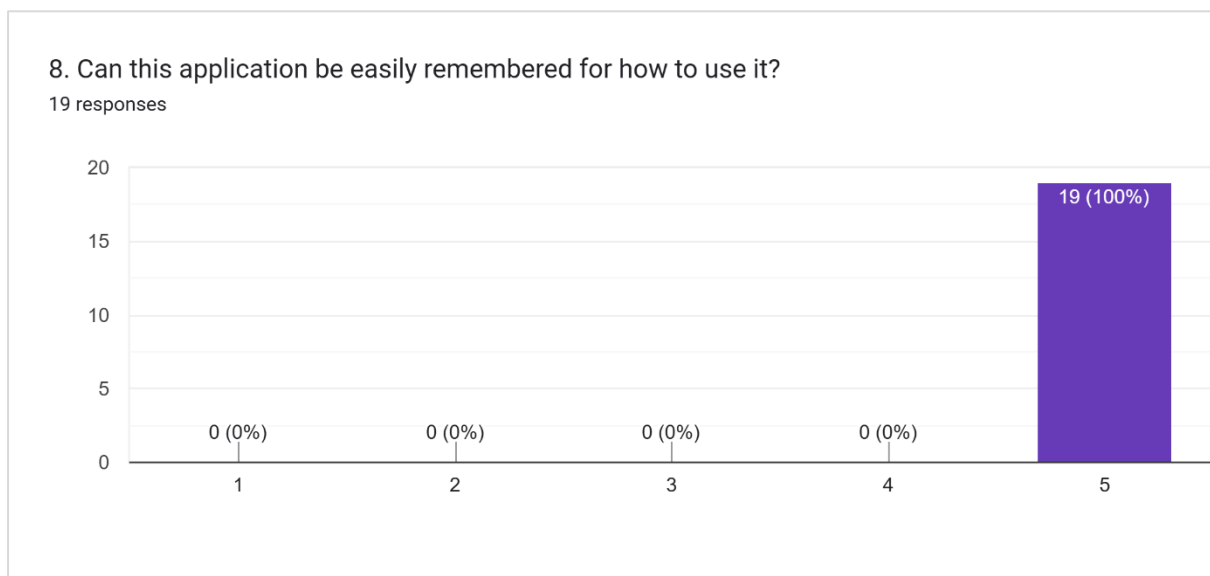


Figure 6.10: The percentage of easily to remember to use the application

Figure 6.11 below shows the percentage of respondents that agree on the system will learn this application quickly. In the percentage of respondents, it can found that the most of the respondents around 72% strongly agree about the system can be learn quickly. And besides, 27% are just agree about the system

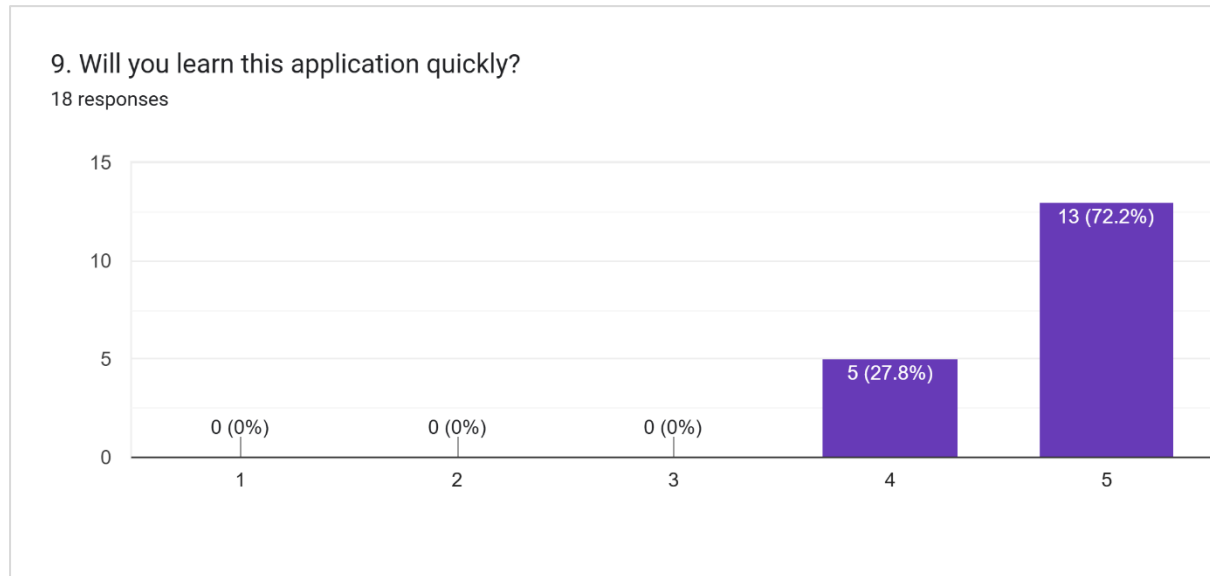


Figure 6.11: The percentage of learn applications quickly

Figure 6.12 shows the percentage of the respondent regarding additional features that users would like to see integrated into the Smart Healthcare Monitoring System. The majority of respondents, representing 26.3%, suggested the inclusion of a voice output feature, highlighting the importance of accessibility and user-friendliness, particularly for individuals who may have difficulty reading or interpreting text-based data. Another 15.8% of respondents also emphasized voice-related functionality, which reinforces the demand for audio-based interaction in the system. In addition, 10.5% of respondents recommended the implementation of a BMI calculator, while another 10.5% suggested combining features such as mobile

notifications with existing functionalities. Smaller groups of respondents (5.3% each) proposed enhancements including mobile notifications, self-calculator functions, general notifications.

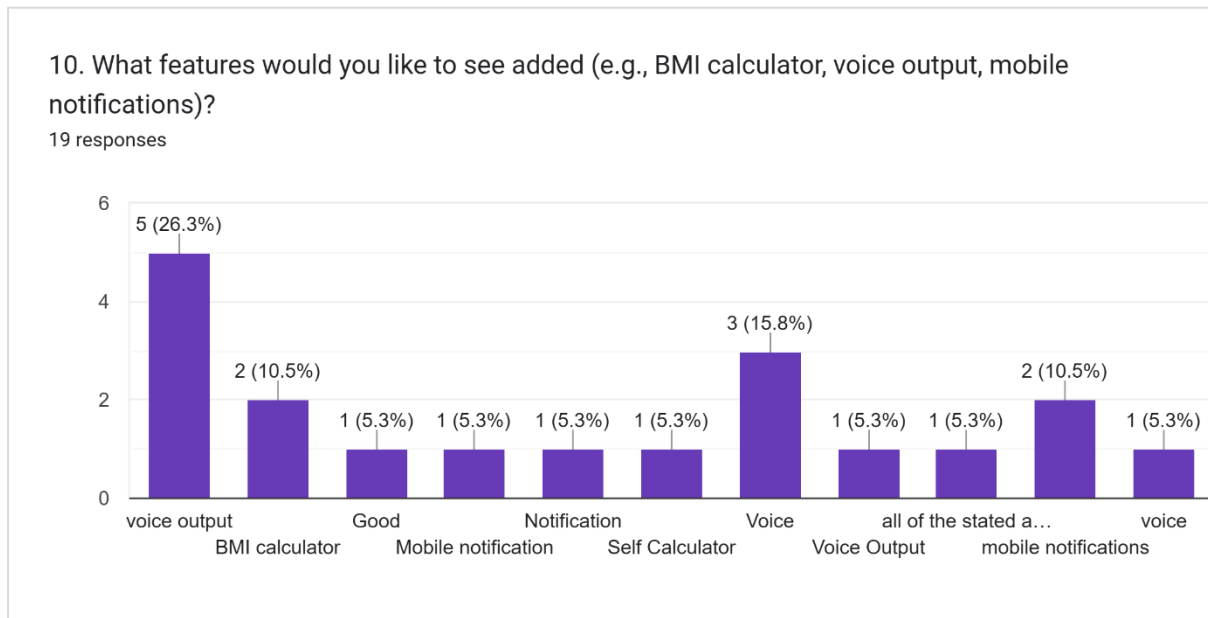


Figure 6.12: The percentage of suggestion for the extra features

6.7 Conclusion

Based on the comprehensive testing and analysis conducted in this chapter, it can be concluded that the NutriGuardian system has been successfully validated and proven reliable through a structured and rigorous testing process. Software testing plays a crucial role in the development lifecycle, ensuring not only that the application meets all functional requirements, but also that it delivers a consistent, accurate, and user-friendly experience. The testing efforts for this system were designed to evaluate its functionality, performance, data accuracy, security, and overall usability prior to final deployment.

A multi-layered testing methodology was applied, beginning with unit testing to confirm the correctness of individual components, followed by integration testing to ensure smooth interaction between modules. System testing was then performed to validate the complete functionality of the application as a whole. Finally, User Acceptance Testing (UAT) provided valuable real-world insights, allowing end-users—including individuals, family members, and hospital staff—to evaluate the practicality and effectiveness of the system in addressing everyday health monitoring needs.

The outcomes of UAT were highly positive, with respondents affirming that the system is user-friendly, efficient, and suitable for regular use. Users particularly valued the automatic measurement feature, noting that it significantly reduces the time required for recording health data. Additional feedback also highlighted potential feature enhancements, such as voice output and mobile notifications, which provide direction for future system improvements.

In conclusion, the testing process confirms that the NutriGuardian system successfully fulfills its intended purpose: offering a reliable, accurate, and accessible solution for recording and monitoring health information. The strong performance across all testing phases, combined with encouraging user feedback, demonstrates that the system is well-prepared for deployment and capable of making a meaningful impact in real-world healthcare monitoring.

CHAPTER 7: CONCLUSION

7.1 Observation and Weakness Strength

The NutriGuardian project has demonstrated several clear strengths throughout its development and testing phases. One of the most prominent strengths is its user-friendliness, which was strongly validated during the User Acceptance Testing (UAT). The system's interface is intuitive, simple to navigate, and designed with the end-user in mind, making it especially suitable for individuals, families, and healthcare staff who require a reliable yet easy-to-use health monitoring solution. This aspect is particularly critical, as a health monitoring system that is overly complex or difficult to operate would fail to achieve widespread adoption. Another significant strength lies in the automatic measurement feature, which was consistently highlighted by users as a time-saving innovation. By streamlining the process of recording height and weight, the feature reduces the burden of manual input and enhances the overall practicality of the system. Moreover, the rigorous testing approach—including unit testing, integration testing, and system testing—ensured that the system's core functionality is validated, reliable, and accurate, reinforcing its credibility as a dependable health monitoring tool.

Despite these strengths, the project also revealed certain weaknesses and limitations that must be acknowledged. While the testing process did not uncover any major flaws, minor bugs and usability issues were identified, suggesting areas that require refinement before full-scale deployment. These weaknesses do not undermine the project's success but highlight the importance of continuous improvement in achieving a flawless final product. Additionally, feedback gathered from the UAT pointed toward potential enhancements, such as the integration of voice output, BMI calculators, and mobile notifications. These requested features indicate that while the current system successfully addresses its core objectives, future iterations can be improved to better meet user expectations and broaden the scope of functionality.

The observations from users during UAT further emphasize the system's strengths and relevance. Respondents consistently reported a positive user experience, confirming that the design and flow of the system are well-aligned with its target audience. Many users also affirmed that the system is suitable for regular use, a key requirement for any health monitoring application intended to integrate seamlessly into daily routines. The efficiency and convenience of the automatic measurement feature was frequently praised, with users acknowledging it as a major contributor to the system's appeal. These insights collectively demonstrate that the NutriGuardian project not only fulfills its intended purpose but also holds strong potential for real-world application.

In conclusion, the NutriGuardian system can be regarded as a technically sound and user-validated solution that effectively addresses the problem of simplifying and improving health monitoring. While some refinements and additional features may be required to enhance its performance further, the project has successfully established a solid foundation. The combination of technical robustness, user-friendliness, and innovative functionality ensures that NutriGuardian provides both value and practicality, marking it as a promising solution within the domain of digital health monitoring.

7.2 Proposition and Improvements

To further enhance the NutriGuardian system and ensure its long-term effectiveness, several improvements can be proposed to expand its functionality, increase user engagement, and provide greater value for both individual users and healthcare professionals. The suggestions below focus on transforming the system

from a basic health monitoring tool into a more comprehensive and intelligent wellness platform.

One potential improvement is the integration with wearable technology such as smartwatches and fitness trackers. While the current system excels with its automatic measurement feature, wearable integration would allow NutriGuardian to collect a broader range of real-time health data, including heart rate, sleep quality, daily physical activity, and calorie expenditure. Combining this information with height, weight, and nutritional tracking would give users a holistic view of their overall health. This enhancement would not only improve accuracy but also enable the system to deliver personalized health recommendations, positioning NutriGuardian as a one-stop solution for daily health monitoring.

Another proposed improvement is the addition of personalized meal planning and recipe suggestions. By analyzing user data such as body metrics, dietary preferences, health goals (e.g., weight loss, muscle gain, or maintenance), and restrictions (e.g., allergies or medical conditions), the system could generate weekly meal plans tailored to each individual's needs. A built-in recipe database aligned with these goals would provide practical guidance for healthier eating habits, making NutriGuardian more than just a monitoring tool—it would act as a proactive digital health coach. This would empower users to not only track their progress but also make informed dietary decisions that support their long-term wellness.

Lastly, introducing social and community-driven features could significantly boost user motivation and engagement. By allowing users to connect with peers, share their progress, exchange tips, and participate in health-related challenges or leaderboards, NutriGuardian could create a supportive and interactive environment. Features such as discussion forums, recipe-sharing platforms, or group challenges

would foster a sense of accountability and belonging, which are powerful drivers for maintaining healthy habits. This community aspect would transform NutriGuardian into not just a tool but also a platform where users feel supported and inspired to achieve their health and wellness goals together.

7.3 Project Contribution

The NutriGuardian project represents a meaningful contribution to the field of health technology, with impacts that extend to the university, faculty, and individual users. From an academic perspective, this project demonstrates the practical application of theoretical knowledge in areas such as IoT, AI, and mobile development. By transforming classroom concepts into a fully functional system, NutriGuardian stands as evidence of how innovative student research can address real-world challenges. This not only strengthens the reputation of the university and faculty for producing impactful, application-driven projects but also serves as a reference point for future research and development in smart healthcare systems.

On an individual level, the contribution of NutriGuardian is even more tangible. The system provides users with a practical and convenient solution for monitoring their health, particularly in tracking nutritional data, height, and weight with minimal effort. By automating data collection and integrating it into a user-friendly mobile interface, the project empowers individuals to take greater control of their health, make informed dietary decisions, and adopt healthier lifestyles. The positive outcomes from User Acceptance Testing further validate the system's usefulness, highlighting its potential to improve daily health management and contribute to long-term well-being.

Additionally, this project contributes valuable insights into the growing integration of IoT and mobile applications in healthcare, which can inspire further innovations in the field. Beyond its technical contributions, NutriGuardian also fosters awareness about preventive health monitoring, emphasizing the importance of convenience and accessibility in encouraging consistent user engagement.

To complement this contribution, a comprehensive User Manual has been prepared and is included in the appendices of this report. The manual provides clear step-by-step instructions, supported by visual aids such as images of the testers, IoT components, demonstration setups, and system screenshots. This ensures that users can operate the system effectively and fully benefit from its features, thereby extending the impact of the project from research into real-world application.

7.4 Conclusion

Based on the evidence and findings presented throughout this report, it can be confidently concluded that the NutriGuardian project has successfully achieved its intended objectives. The development and implementation of the system have demonstrated the effective integration of IoT devices, mobile applications, and data processing to create a reliable and user-friendly health monitoring solution. The testing phase, which included unit testing, integration testing, system testing, and User Acceptance Testing (UAT), provided strong validation of the system's functionality and performance. In particular, the overwhelmingly positive feedback from UAT respondents highlights the system's strengths in terms of user-friendliness, suitability for daily use, and the efficiency of its automatic measurement feature. These outcomes confirm that NutriGuardian not only fulfills its core purpose of simplifying the monitoring of height, weight, and nutritional data but also delivers genuine value to end-users by saving time and providing practical health insights.

Ultimately, the NutriGuardian project stands as a testament to how theoretical knowledge in IoT and AI can be applied to solve real-world challenges in the healthcare domain. By combining innovation with practicality, the system has proven its potential as a valuable tool for personal health management, empowering individuals to take a proactive role in maintaining their well-being. Furthermore, the project contributes to the broader field of smart health technology, offering a foundation that can be further enhanced with additional features such as wearable device integration, personalized meal planning, and community-driven engagement. In conclusion, NutriGuardian not only validates its viability as a working system but also paves the way for future research and development, demonstrating the important role of technology in shaping healthier and more sustainable lifestyles.

REFERENCES

- Hayat, F., Rehman, A. U., Arif, K. S., Wahab, K., & Abbas, M. (2019, July). The influence of agile methodology (Scrum) on software project management. In 2019 20th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD) (pp. 145-149). IEEE.
- Abdulmalek, Suliman, et al. "IoT-Based Healthcare-Monitoring System towards Improving Quality of Life: A Review." *Healthcare*, vol. 10, no. 10, 2022, p. 1993. *Mdpi*, pmc.ncbi.nlm.nih.gov/articles/PMC9601552/.
- "Add Nutrition Data." *Google for Developers*, 2024, developers.google.com/fit/scenarios/add-nutrition-data. Accessed 26 June 2025.
- Guyon, Agnes, et al. "Mobile-Based Nutrition and Child Health Monitoring to Inform Program Development: An Experience from Liberia." *Global Health: Science and Practice*, vol. 4, no. 4, 23 Dec. 2016, pp. 661–670, <https://doi.org/10.9745/ghsp-d-16-00189>.
- "IoT Based Health Monitoring System | Arduino Project." *Projecthub.arduino.cc*, projecthub.arduino.cc/rajeshjiet/iot-based-health-monitoring-system-arduino-project-27f2ba.
- Lirasan, Jamesrell L, et al. *Community-Based Child Nutrition Projection and Monitoring*. Vol. 8, no. 1, 1 Aug. 2019, pp. 1–24, www.researchgate.net/publication/364308227_Community-Based_Child_Nutrition_Projection_and_Monitoring.
- Mistry, Kajal, and Georgios Dafoulas. "IoT Wearables in Child Health: A Comprehensive Scoping Review and Exploration of Ubiquitous Computing."

Internet of Things, vol. 31, May 2025, p. 101556,
<https://doi.org/10.1016/j.iot.2025.101556>.

“NutrAI - Personal Nutrition Assistant | Gemini API Developer Competition | Google AI for Developers.” *Google AI for Developers*, 2025, ai.google.dev/competition/projects/nutrai-personal-nutrition-assistant. Accessed 26 June 2025.

“NutriBot: An AI-Powered Personalized Nutrition Recommendation Chatbot Using Rasa.” *Ku.edu*, 16 May 2025, eecs.ku.edu/nutribot-ai-powered-personalized-nutrition-recommendation-chatbot-using-rasa.

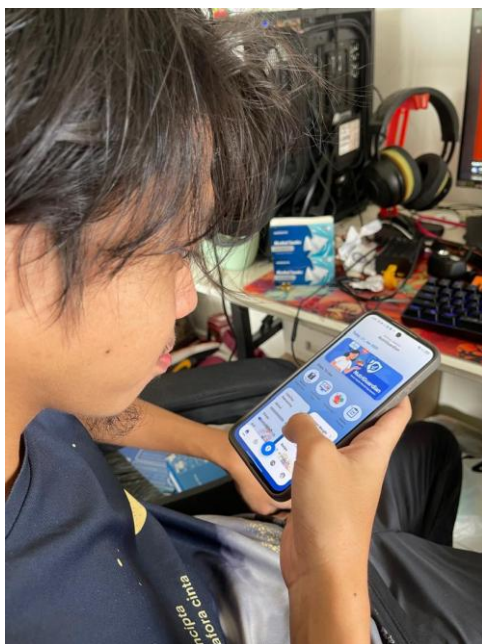
Papastratis, Ilias, et al. “AI Nutrition Recommendation Using a Deep Generative Model and ChatGPT.” *Scientific Reports*, vol. 14, no. 1, 25 June 2024, <https://doi.org/10.1038/s41598-024-65438-x>.

Phalle, Anuja, and Devaki Gokhale. “Navigating Next-Gen Nutrition Care Using Artificial Intelligence-Assisted Dietary Assessment Tools—a Scoping Review of Potential Applications.” *Frontiers in Nutrition*, vol. 12, 23 Jan. 2025, pmc.ncbi.nlm.nih.gov/articles/PMC11798783/, <https://doi.org/10.3389/fnut.2025.1518466>.

Vishal Dharmadhikari. “CalCam: Transforming Food Tracking with the Gemini API.” *Googleblog.com*, 5 Mar. 2025, developers.googleblog.com/en/calcam-transforming-food-tracking-with-the-gemini-api/. Accessed 26 June 2025.

APPENDIX A

	
The plate of the weight scale	The complete of the IoT
	
The height sensor	Tester 1 of the system: Individual, IT Student



Tester test how the system can be help a person to store their weight and height



Tester 2 of the system: Hospital Staff, Medical Engineer



Test the accuracy of the weight and height and calculation of BMI



Tester 3 of the system: Parent, Housewife



Test how the system can manage the
young child